

Porque Docker Compose sabe de forma inteligente que el volumen `~/dev/roadtok8s/py/dev` no está administrado por Docker Compose. Esto significa que Docker Compose no intentará eliminarlo.

Permitamos que Docker Compose administre los volúmenes en nuestro nombre. Esto le otorga a Docker Compose mayor control sobre el entorno y, además, simplifica la gestión de las dependencias de nuestros contenedores.

6.3.3 Gestión de volúmenes integrada con Docker Compose

Docker Compose tiene una sección de nivel superior llamada volúmenes , donde podemos declarar los distintos volúmenes que queremos que Docker Compose administre. El nombre del volumen declarado depende de nosotros (al igual que con los servicios), por lo que usaré nombres obvios para que sea fácil distinguirlos.

El Listado 6.7 muestra un ejemplo de un archivo `compose.yaml` donde llamamos a un volumen `mywebdata` y lo asignamos a `/app/data` en nuestro contenedor. En este caso, Docker Compose administrará `mywebdata` porque está declarado en la sección de volúmenes ; no existe en mi proyecto local y nunca existirá. Si bien existe una forma de acceder a este volumen directamente en nuestro equipo host, no es necesario, por lo que permitir que Docker Compose administre `mywebdata` es la mejor opción en este caso. Además, este volumen se eliminará al ejecutar `docker compose down -v`, ya que lo administra Docker Compose; su directorio autoadministrado no se eliminará con este comando.

Listado 6.7 Gestión de volúmenes con Docker Compose

```
servicios: web:

  construir:
    contexto: .
    dockerfile: Dockerfile

  Puertos:
    - Entorno "8080:8080":

    - PUERTO=8080

  volúmenes:
    - mywebdata:/app/data

volúmenes:
  mywebdata:
```

Ahora, si ejecutamos `docker compose up` y `docker compose down -v`, veremos la salida en la figura 6.6.

```
% docker compose down -v
[+] Running 3/3
✓ Container roadtok8s-py-web-1    Removed    10.2s
✓ Volume roadtok8s-py_mywebdata   Removed    0.0s
✓ Network roadtok8s-py_default    Removed    0.0s
```

Figura 6.6 Administración de volúmenes con Docker Compose

La figura muestra que Docker Compose eliminará un volumen, destruyendo todos los archivos y datos que contiene. Les mostré primero la otra opción, ya que destruir volúmenes que no deben destruirse es terrible y es algo que queremos evitar.

La última forma en que analizaremos la gestión de contenedores es declarando un volumen externo directamente en `compose.yaml`. Los volúmenes externos no son administrados por Docker Compose y, por lo tanto, no se eliminarán al ejecutar `docker compose down -v`, lo que los convierte en candidatos ideales para los datos que...

Debe persistir independientemente del estado de Docker o Docker Compose. Usar la configuración `external: true` permite usar una carpeta relativa a `compose.yaml` como volumen externo que Docker Compose no intentará administrar. La siguiente lista muestra cómo hacerlo.

Listado 6.8 Gestión de volúmenes con Docker Compose

```
servicios:
  web:
    compilación:
      contexto: .
      dockerfile: Dockerfile
    Puertos:
      - Entorno "8080:8080":

      - PUERTO=8080
    volúmenes:
      - mywebdata:/app/data -
      myprojectdata:/app/other-data

volúmenes:
  mywebdata:
  myprojectdata:
    externo: verdadero
```

Si ejecuta Docker Compose sin crear la carpeta "myprojectdata", recibirá un error que indica que no se encontró el volumen externo "myprojectdata". Esto indica que Docker Compose no está administrando esta carpeta y debe crearla.

Otro aspecto clave a tener en cuenta es que puedes combinar los distintos tipos de volúmenes, así como tener varios volúmenes diferentes asociados a tu contenedor. Esto significa que puedes tener un volumen administrado por Docker Compose, un volumen administrado por Docker Compose pero externo, y un volumen no administrado por Docker.

Compose en absoluto. Si los datos del volumen son importantes y deben persistir, recomiendo no permitir que Docker Compose los administre, sino adjuntar directorios que usted mismo administre. Dejar que Docker Compose administre un volumen significa que este puede destruirse en cualquier momento, incluso accidentalmente mediante `docker compose down -v`; esto no es algo que desee que le suceda a sus datos críticos.

Ahora que comprendemos cómo usar volúmenes, veamos una razón práctica para usarlos: las bases de datos. Para usar las bases de datos eficazmente con Docker Compose, debemos comprender un concepto clave adicional: la red.

6.4 Fundamentos de redes en Docker Compose

Docker Compose es nuestro primer paso hacia la orquestación de múltiples contenedores, y comprender cómo se comunican entre sí es fundamental. Hasta ahora, hemos usado nuestro host local para acceder a nuestras aplicaciones Python o Node, independientemente del puerto y de si la aplicación usa un contenedor o Docker Compose. El host local suele estar asignado a la dirección IP 127.0.0.1, una dirección IP especial que siempre apunta a la máquina actual. Usar localhost nos permite comunicarnos con aplicaciones alojadas localmente.

Docker Compose nos permite modificar esto al proporcionar una red para que nuestros contenedores se comuniquen entre sí.

Esto significa que podemos usar el nombre del servicio en lugar de localhost o 127.0.0.1 para acceder a nuestras aplicaciones. Docker Compose crea esta red automáticamente, pero también podemos crear nuestras propias redes si es necesario.

En esta sección, analizaremos algunos fundamentos de la red con Docker Compose en relación con la comunicación entre contenedores. Mediante este tipo de comunicación, podemos conectar aplicaciones contenedorizadas con estado, como bases de datos, directamente con aplicaciones contenedorizadas sin estado, como nuestras aplicaciones Python o Node.js. Una vez comprendidos estos fundamentos, sentaremos las bases para el futuro de Kubernetes.

6.4.1 Comunicación de contenedor a contenedor simplificada

Con Docker Compose, los contenedores se comunican entre sí mediante el nombre del servicio . Este nombre se convierte en nuestro nombre de host. Esto significa que `http://localhost:8080` se convierte en `http://web:8080` , donde "web" es el nombre del servicio.

En el archivo `compose.yaml` de nuestra aplicación Python, añadiremos un nuevo servicio que ejecuta la imagen pública del contenedor NGINX. Esta imagen es fácil de ejecutar en Docker Compose y, si se ejecuta correctamente, devolverá contenido HTML estándar por defecto.

El Listado 6.9 muestra cómo agregar este nuevo servicio a nuestro archivo `compose.yaml`. Usaremos el nombre de servicio `my-nginx-service` para la imagen NGINX, simplemente para distinguirla de nuestro servicio `web` .

Listado 6.9 del servicio NGINX en Docker Compose

```
servicios: mi-
  servicio-nginx: imagen: nginx
    web: compilación:

    contexto: .
    dockerfile: Dockerfile

  Puertos:
    - Entorno "8080:8080":

    - PUERTO=8080
  volúmenes:
    - mywebdata:/app/data

volúmenes:
  mywebdata:
```

Ahora que tenemos este nuevo servicio, realizamos los siguientes pasos:

1. Si los contenedores están en ejecución, ejecute `docker compose down`.
2. Ejecute `Docker Compose Up`. Esto descargará la imagen del contenedor NGINX si no está disponible localmente. También creará nuestra red llamada `roadtok8s-py_default`, que ambos servicios usarán por defecto.
3. Ejecute `docker compose exec web /bin/bash`. Esto abrirá el archivo `shell bash` para el servicio `web`.
4. Dentro del servicio `web`, ejecute el comando `curl http://my-nginx-service` o `curl http://my-nginx-service:80`.

Después de ejecutar el comando `curl`, veremos el HTML que resulta de la página estándar NGINX, como se ve en la figura 6.7.

```
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully in
stalled and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@68a2944a8c33:/app#
```



Figura 6.7 Respuesta del servicio NGINX en Docker Compose

En nuestra ventana de terminal de Docker Compose Up , veremos que my-nginx-service indicará que se produjo la solicitud, como se ve en la figura 6.8.

```

#1: start worker process 31
roadtok8s-py-my-nginx-service-1 | 2023/08/15 18:30:41 [notice] 1
#1: start worker process 32
roadtok8s-py-my-nginx-service-1 | 2023/08/15 18:30:41 [notice] 1
#1: start worker process 33
roadtok8s-py-web-1 | [2023-08-15 18:30:41 +0000] [7
] [INFO] Starting gunicorn 21.2.0
roadtok8s-py-web-1 | [2023-08-15 18:30:41 +0000] [7
] [INFO] Listening at: http://0.0.0.0:8080 (7)
roadtok8s-py-web-1 | [2023-08-15 18:30:41 +0000] [7
] [INFO] Using worker: uvicorn.workers.UvicornWorker
roadtok8s-py-web-1 | [2023-08-15 18:30:41 +0000] [8
] [INFO] Booting worker with pid: 8
roadtok8s-py-web-1 | [2023-08-15 18:30:41 +0000] [8
] [INFO] Started server process [8]
roadtok8s-py-web-1 | [2023-08-15 18:30:41 +0000] [8
] [INFO] Waiting for application startup.
roadtok8s-py-web-1 | [2023-08-15 18:30:41 +0000] [8
] [INFO] Application startup complete.
roadtok8s-py-my-nginx-service-1 | 172.30.0.3 - - [15/Aug/2023:18
:31:03 +0000] "GET / HTTP/1.1" 200 615 "-" "curl/7.88.1" "-"

```

Figura 6.8 Respuesta del servicio NGINX en Docker Compose

Como podemos ver, el servicio web pudo comunicarse con el servicio my-nginx-service utilizando el nombre del servicio como nombre de host. También podemos observar que nuestro archivo compose.yaml nunca expone el servicio my-nginx-service al host (nuestro equipo local) a través de un puerto. Esto significa que solo se puede acceder al servicio my-nginx-service desde la red de Docker Compose. Una forma rápida de practicar este concepto es hacer lo siguiente:

- Ejecute docker compose down para desactivar los servicios y las redes.
- Cambie los nombres de los servicios en compose.yaml por otros.
- Ejecute Docker Compose Up para volver a activar los servicios y las redes.
- Eliminar todos y cada uno de los puertos declarados.

- Intente ejecutar curl desde un shell bash (docker compose exec my-service / bin/bash) y vea si puede acceder al otro servicio.

Usar el contenedor NGINX nos facilitó las cosas, ya que el puerto predeterminado es el 80. Algunas aplicaciones en contenedores, como nuestras aplicaciones de Python y Node.js, no usan el puerto 80 por defecto. Es importante que, al comunicarse dentro de la red, sepa cuál es el puerto correcto.

Este tipo de comunicación es excelente porque permite que nuestros servicios de Docker Compose usen puertos comunes sin entrar en conflicto con los puertos del equipo host. Si bien es una función útil, puede generar confusión durante el aprendizaje.

Por lo tanto, en caso de duda, agregue los argumentos de los puertos a su archivo compose.yaml para exponer los puertos a su equipo host. Ahora combinaremos nuestros conocimientos sobre volúmenes y redes para crear un contenedor de base de datos que nuestra aplicación Python pueda usar.

6.4.2 Bases de datos y Docker Compose

Cuando utilizamos aplicaciones de bases de datos basadas en contenedores, debemos tener en cuenta algunas cosas:

- Al montar volúmenes, nuestros datos persistirán. Si no montamos un volumen, los datos no persistirán.
- Las bases de datos tienen puertos estándar que debemos conocer. Si tiene demasiados proyectos en su equipo host intentando usar el mismo puerto, se producirán muchos errores innecesarios. docker compose down conservará los datos en los volúmenes montados
- administrados por Docker Compose, mientras que docker compose down -v destruirá los datos en esos mismos volúmenes. Esto significa

Los datos de su base de datos se pueden borrar por completo simplemente utilizando el indicador -v con ciertos tipos de montaje de volumen.

- Utilizar el nombre del servicio en lugar del nombre de host es la forma en que sus otros servicios pueden conectarse.

Con esto en mente, instalaremos Redis para nuestra aplicación Python.

Redis es uno de los almacenes de datos clave-valor más útiles disponibles y es increíblemente fácil de configurar.

Antes de agregar nuestro servicio Redis al archivo `compose.yaml`, necesitamos actualizar el `Dockerfile` de nuestra aplicación Python para instalar `redis-tools` y poder usar el cliente de línea de comandos de Redis. La siguiente lista muestra cómo hacerlo.

Listado 6.10 Actualización de nuestro `Dockerfile` de Python para herramientas Redis

```
...  
# Ejecutar actualizaciones a nivel de sistema operativo  
EJECUTAR apt-get update  
    && \ apt-get install -y python3-venv python3-dev python3-pip  
  
# Instalar herramientas redis  
EJECUTAR apt-get install -y redis-tools  
...
```

Es importante tener en cuenta que la mayoría de las bases de datos de código abierto requieren controladores adicionales a nivel de sistema para las aplicaciones cliente (p. ej., en Docker) y paquetes específicos del lenguaje (p. ej., `requirements.txt` o `package.json`) para una conexión completa. Esto aplica tanto a aplicaciones contenedorizadas como a aplicaciones que no lo son, pero que sí están instaladas en Linux. En otras palabras, si desea usar PostgreSQL o MySQL, existen pasos adicionales que quedan fuera del alcance de este libro, pero el proceso es prácticamente el mismo. Con nuestro `Dockerfile` actualizado, agreguemos un nuevo servicio a nuestro `compose.yaml` llamado `my-redis-service`, como se muestra en la siguiente lista.

Listado 6.11 del servicio Redis en Docker Compose

```
servicios: mi-
  servicio-nginx: imagen: nginx
    web: compilación:

    contexto: .
    dockerfile: Dockerfile

  Puertos:
    - Entorno "8080:8080":

    - PUERTO=8080
  volúmenes:
    - mywebdata:/app/data my-redis-
service: imagen: redis
  volúmenes: -
  myredisdata:/
  data

volúmenes:
  mywebdata:
  myredisdata:
```

Ahora tenemos todas las condiciones necesarias para actualizar nuestro servicio web y comunicarnos con nuestro servicio Redis. Siga estos pasos para verlo en acción:

1. Ejecute `docker compose down --remove-orphans` para desactivar los servicios y las redes. El parámetro `--remove-orphans` es útil si alguna vez cambia los nombres de los servicios o los elimina del archivo `compose.yaml`.
2. Ejecute `docker compose up --build`. En este caso, el comando `--build` es necesario porque modificamos el `Dockerfile` de nuestro servicio web.
3. Ejecute `docker compose exec web /bin/bash` para ingresar al shell `bash` para el servicio web.

4. Dentro de la sesión bash, ejecute `redis-cli ping`. Debería ver el mensaje "No se pudo conectar a Redis en 127.0.0.1:6379: Conexión rechazada".

5. Ahora ejecuta `redis-cli -h my-redis-service ping`. Deberías ver "PONG" como respuesta.

Además de ajustar el entorno, verás una vez más que el nombre de host de los servicios de Docker Compose coincide directamente con lo que declaras en `compose.yaml`.

Ahora podemos iterar en nuestro código Python para usar Redis como almacén de datos. También podemos usar `redis-cli` para interactuar con nuestro servicio Redis. Si bien esto es útil, lo importante es que ahora sabemos cómo establecer comunicación entre contenedores dentro de Docker Compose mediante contenedores con y sin estado, con o sin volúmenes.

El último paso clave para mejorar nuestros contenedores es usar un archivo de variable de entorno externo llamado `dotenv`. Este archivo externo se puede usar tanto con Docker como con Docker Compose; analicemos ambos.

6.4.3 Variables de entorno con `dotenv` en Docker y Docker Compose

A lo largo de este libro, hemos visto que el valor `PORT` aparece en todas partes y, normalmente, como una variable de entorno.

Cambiar este valor es una forma sencilla pero efectiva de resaltar cuánto pueden afectar las variables de entorno a nuestras aplicaciones (contenedores o de otro tipo).

Al codificar cualquier tipo de variable de aplicación, debemos decidir si es relevante para el tiempo de ejecución de la aplicación. `PORT` es un buen ejemplo de una variable de entorno de ejecución que se puede modificar mientras afecta...

Cómo se accede a la aplicación, pero no su lógica de ejecución. En nuestro caso, diseñamos PORT para que este efecto ajustara el acceso, pero no afecta a la aplicación en sí.

Para una gran cantidad de variables de entorno, debemos asegurarnos de que estén abstraídas del código de la aplicación.

Por ejemplo, las contraseñas y las claves API nunca deben escribirse directamente en el código. La razón obvia es que al enviar código a un repositorio, se pueden filtrar estos datos confidenciales, lo que causa una vulnerabilidad de seguridad grave, incluso si el repositorio es privado. Para mitigar esto, podemos empezar a usar archivos dotenv (.env), que nos permiten extraer nuestros datos confidenciales en un archivo que no se enviará a un repositorio Git.

Los archivos dotenv se utilizan durante todo el proceso de desarrollo y, en algunos casos, en sistemas de producción. No suelo recomendar su uso en producción, pero hacerlo es sin duda mejor que tener datos confidenciales en el código.

Para la producción, recomiendo herramientas como HashiCorp Vault o Google Secret Manager para administrar datos confidenciales.

La configuración de ambas herramientas está fuera del alcance de este libro.

Para utilizar archivos dotenv, utilizamos la siguiente sintaxis:

```
CLAVE=valor  
CLAVE="valor"  
CLAVE='valor'
```

Puede leer más sobre esta sintaxis en la referencia oficial de Docker (<https://docs.docker.com/compose/environment-variables/env-file/>).

Queremos crear un archivo dotenv para la variable REDIS_HOST para que nuestro ping redis-cli -h my-redis-service pueda convertirse en redis-

recorte. Una variable `REDIS_HOST` es un buen ejemplo de una variable de entorno que no expondría en su código.

En la raíz de tu proyecto Python (`~/dev/roadtok8s/py`), crea un archivo llamado `.env`. Es importante que uses el prefijo punto en el nombre del archivo. Git ignorará este archivo y no se guardará en tu repositorio (los archivos `.gitignore` que usamos ya contienen `.env`). La siguiente lista muestra cómo agregar la variable `REDIS_HOST` a nuestro archivo `.env`, junto con algunos otros ejemplos que no usaremos.

Listado 6.12 Variable de entorno del host de Redis

```
REDIS_HOST=mi-servicio-redis
DEBUG=1
RK8S_API_KEY="esto-es-falso"
```

Para usar este archivo `.env` con Docker, es tan simple como `docker run --env-file path/to/.env -p 8080:8080 rk8s-py` o desde la raíz de nuestro proyecto `docker run --env-file .env -p 8080:8080 rk8s-py`.

¿Crees que ejecutar este contenedor fuera de Docker Compose permitirá la comunicación con `my-redis-service` dentro de Docker Compose? Inténtalo.

Para utilizar este archivo `dotenv` dentro de nuestro `compose.yaml`, podemos actualizar cualquier configuración de servicio, como en la siguiente lista.

Listado 6.13 Uso de dotenv con Docker Compose

```
servicios:
  ...
  web:
    ...
    archivo env:
      - .env
    ambiente:
      - PUERTO=8080
    volúmenes:
      ...
  ...
```

Con esta nueva configuración, ahora podemos ejecutar `docker compose up` y `docker compose exec web /bin/bash` para acceder al shell bash del servicio web. Dentro del shell bash, podemos ejecutar `redis-cli ping` y ver la respuesta: PONG. A corto plazo, esta configuración es más compleja. A largo plazo, es más sencilla, más flexible, más portátil y, en definitiva, más segura.

Como puedes imaginar, hay muchas otras formas de personalizar el uso de Docker y Docker Compose.

Este libro no está diseñado para todas las formas de personalizar estas herramientas. En cambio, ahora contamos con un modelo práctico para usar Docker Compose localmente para desarrollar nuestras aplicaciones en contenedores. Ahora es el momento de aplicar este conocimiento e implementar aplicaciones usando contenedores con Docker y Docker Compose.

Resumen

- La creación de contenedores listos para producción debe realizarse en una secuencia de CI/CD para automatizar el proceso y garantizar la coherencia y la calidad.

- Docker Compose es una forma eficaz de administrar varios parámetros necesarios para ejecutar un solo contenedor.
- Para hacer que un contenedor recuerde datos, montamos volúmenes en él, lo que le otorga una naturaleza con estado en lugar de la naturaleza efímera predeterminada.
- Docker Compose ayuda a reducir la complejidad de administrar varios contenedores a la vez y al mismo tiempo proporciona una serie de funciones útiles para administrar volúmenes y comunicación entre contenedores con redes integradas.
- Nunca es una buena idea codificar información confidencial; utilice siempre variables de entorno cuando sea posible.
- Los archivos Dotenv (.env) son un formato común para inyectar varias variables de entorno en el entorno de ejecución de un contenedor a través de la CLI de Docker o directamente en Docker Compose.
- Docker Compose puede administrar el ciclo de vida de un contenedor desde la compilación hasta la ejecución y la detención, y puede administrar volúmenes por usted si los declara en el archivo `compose.yaml` o `docker-compose.yaml`.

7 Implementación de aplicaciones en contenedores

Este capítulo cubre

- Instalación de Docker en una máquina virtual
- Creación e implementación de imágenes de Docker en una máquina virtual
- Uso de Docker Compose en producción
- Implementación de una canalización de CI/CD
- Discutiendo las limitaciones de Docker Compose

Al implementar nuestras aplicaciones por primera vez, tuvimos que instalar varias dependencias a nivel de sistema (p. ej., Supervisor, Python, Node.js, etc.) y dependencias específicas del entorno de ejecución (p. ej., `pip install`, `npm install`). Esto se aplicó tanto a nuestro sistema local como a nuestras máquinas virtuales de producción remotas.

Estas dependencias siempre serán necesarias por su propia definición, pero Docker nos ayuda a gestionar estas dependencias de una manera más efectiva, especialmente en producción.

En producción, la principal dependencia a nivel de sistema que necesitaremos es un entorno de ejecución de contenedor, en lugar de entornos de ejecución de aplicaciones individuales y sus dependencias a nivel de sistema. Dicho de otro modo, solo necesitamos instalar Docker Engine y Docker Compose en nuestra máquina virtual de producción; nunca necesitaremos instalar Python, Node, Java, MySQL, PostgreSQL, Redis, Ruby, etc. Esta es la magia de los contenedores: un entorno de ejecución para gobernarlos a todos.

Antes de implementar nuestras aplicaciones, exploremos el uso de Docker en producción mediante la implementación de un entorno público y simple

	pag	Aplicación	pag	pag
--	-----	------------	-----	-----

contenedorizada ypyg: NGINX. Comenzaremos aquí como prueba de concepto para implementar aplicaciones más complejas, como nuestras aplicaciones Python y Node.js.

7.1 Hola prod, soy Docker

Implementar contenedores es tan sencillo como instalar Docker y ejecutar el comando `docker run <image-name>` o `docker-compose up` (sí, el - es intencional). Comenzaremos con una implementación simple de la imagen del contenedor NGINX para limitar la configuración inicial, de forma similar a como hicimos con la implementación de NGINX sin contenedor. El primer paso es aprovisionar una nueva máquina virtual e instalar Docker.

7.1.1 Aprovisionamiento de una nueva máquina virtual

Recomiendo comenzar este proceso con una máquina virtual nueva. A estas alturas, ya debería estar muy familiarizado con la creación de una nueva máquina virtual. Si necesita un repaso, aquí tiene un resumen del capítulo 3:

1. Cree una cuenta en Akamai Connected Cloud en linode.com/rk8s (los lectores del libro también reciben un crédito promocional).
2. Inicie sesión en Akamai Connected Cloud (anteriormente Linode).
3. Haga clic en el menú desplegable Crear > Linode.
4. Utilice la siguiente configuración:
 - Distribución: Ubuntu 22.04 LTS (o la última versión LTS)
 - Región: Dallas, TX (o la región más cercana a usted)
 - Plan—CPU compartida > Linode 2 GB (o más)
 - Etiqueta—rk8s-docker
 - Etiquetas—<opcional>
 - Contraseña de root—Establezca una buena

- Claves SSH—<recomendadas> (revise el apéndice D para aprender cómo crear una y agregarla a Linode).
 - Todas las demás configuraciones—
- <opcional> 5. Haga clic en Crear Linode.
6. Después de unos minutos, verá aparecer una dirección IP en la consola Linode (figura 7.1).

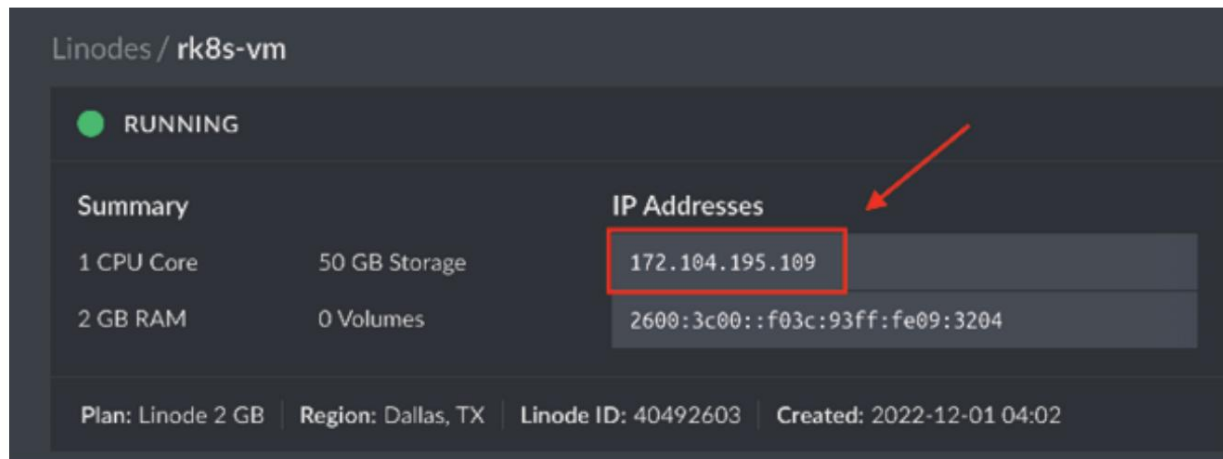


Figura 7.1 Máquina virtual Linode aprovisionada con una dirección IP

Una vez que tenga esta máquina virtual y dirección IP, verifique que pueda acceder a ella por SSH ejecutando `ssh root@<dirección-ip>`. Si puede conectarse, estará listo para instalar Docker.

7.1.2 Instalación de Docker en Ubuntu

Al instalar Docker en una máquina virtual Linux, instalamos Docker Engine, el entorno de ejecución de Docker que usaremos. Como recordatorio, instalamos Docker Desktop en nuestra máquina local. Docker Engine no es tan pesado ni tiene una interfaz gráfica de usuario como Docker Desktop.

NOTA: El término Docker tiene muchos significados, como hemos aprendido. Quizás hayas oído que Kubernetes ya no es compatible con Docker, pero no es cierto. Kubernetes

Utiliza un entorno de ejecución diferente al de Docker Engine para ejecutar contenedores. Con una configuración adicional, Kubernetes puede usar Docker Engine y otros entornos de ejecución, pero esto no es predeterminado. Cuando Kubernetes dejó de usar Docker Engine, generó confusión, como si Kubernetes ya no usara Docker, lo cual no es cierto.

Kubernetes utiliza containerd, que es un entorno de ejecución incluso más ligero que Docker Engine.

Al igual que con nuestro sistema local, existen varias maneras de instalar Docker en una máquina virtual. Mi método preferido es usar el script de instalación de Docker disponible en <https://get.docker.com>. Si desear obtener más información, consulte la documentación oficial de Docker Engine (<https://docs.docker.com/engine/>). Para instalar Docker Engine, asegúrese de iniciar una sesión SSH en su máquina virtual y luego ejecute los comandos de la siguiente lista.

Listado 7.1 Instalación de Docker Engine en Ubuntu

```
curl -fsSL https://get.docker.com -o get-docker.sh sudo sh get-docker.sh
```

Para aquellos de ustedes que son nuevos en el uso de estos comandos en Linux o Bash, así es como se desglosan:

- Curl es una forma de descargar archivos de Internet a través de la línea de comandos.
- Usar `-fsSL` con Curl es una combinación de indicadores para seguir redirecciones (`-f`), en modo silencioso (`-s`), suprimir la barra de progreso (`-S`) y seguir redirecciones de encabezado de ubicación (`-L`). Este indicador garantiza que, si se produce alguna redirección, Curl la seguirá y descargará el archivo sin ninguna salida de línea de comandos.
- El comando `sudo` garantiza que el comando anterior tenga todos los privilegios para ejecutar este script. Con muchos

En aplicaciones o instalaciones, ejecutarlo como usuario root podría no ser recomendable, ya que puede tener graves consecuencias para la seguridad. En este caso, usamos un script de Docker de confianza.

- El comando `sh` nos permite ejecutar este script como un script de shell. También se puede usar el comando `bash`.

Después de ejecutar estos comandos, si ejecuta `docker --version`, debería ver algo similar a la figura 7.2. La versión que responda no es importante, siempre que vea un número de versión y no un error de comando no encontrado.

```
root@localhost:~# docker --version
Docker version 24.0.5, build ced0996
root@localhost:~# █
```

Figura 7.2 Versión de Docker Engine

Si bien este script de instalación es excelente, no instala Docker Compose, así que hagámoslo ahora.

7.1.3 Instalación de Docker Compose en Ubuntu

¿Recuerdas cuando dije que Docker Desktop es más pesado que Docker Engine? Una de las razones es que Docker Engine no incluye Docker Compose por defecto. Para instalar Docker Compose, usaremos el gestor de paquetes `apt` integrado, como se muestra en la siguiente lista.

Listado 7.2 Instalación de Docker Compose en Ubuntu

```
sudo apt-get update sudo
apt-get install docker-compose-plugin docker compose --
version
```

NOTA: Las versiones anteriores de Docker Compose se instalaron a través de Python3 y usaron el comando `docker-compose`.

Aunque la versión anterior podría funcionar, no la recomiendo.

Ahora que tenemos Docker y Docker Compose instalados en nuestra máquina virtual, es hora de implementar nuestra primera aplicación contenedorizada directamente con Docker. Dejaremos Docker Compose para más adelante.

7.1.4 Implementación de NGINX

Implementar NGINX directamente en Ubuntu es tan fácil como ejecutar `sudo apt-get install nginx`. Con Docker Engine, implementar NGINX con Docker es tan fácil como ejecutar `docker run nginx`, pero hay una trampa. Primero, `docker run <container-image-name>` se ejecuta en primer plano por defecto. Podemos cambiar esto usando el modo desacoplado de Docker con `--detach` o `-d`, lo que permite que el contenedor se ejecute en segundo plano. Esto es similar a cómo ejecutamos nuestras aplicaciones Python y Node en segundo plano con Supervisor. Para ejecutar NGINX en el modo desacoplado en segundo plano, use `docker run -d nginx`.

Si bien ejecutar contenedores continuamente en segundo plano es una buena idea, esto plantea una nueva pregunta: ¿cómo podemos detener su ejecución? Normalmente, presionamos Control + C, pero eso no funcionará en este caso. Además, no tenemos forma de acceder a una shell Bash en este contenedor.

Docker cuenta con un gestor de procesos integrado al que se puede acceder mediante `docker ps`, que generará todos los contenedores en ejecución en el sistema. Esto funciona en todas partes, pero ahora es especialmente importante. La Figura 7.3 muestra la salida de `docker ps` tras ejecutar `docker run -d nginx`.

```

root@localhost:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
be52bea82b43   nginx     "/docker-entrypoint..." 5 seconds ago  Up 4 seconds  80/tcp       boring_jepsen

```

Figura 7.3 Administrador de procesos de Docker

Este comando nos proporciona algunos datos clave que necesitamos para cualquier contenedor determinado:

- **ID DE CONTENEDOR** : El ID de contenedor es un identificador único para este contenedor. Así es como podemos hacer referencia a este contenedor en otros comandos, como veremos en breve.
- **IMAGEN**: el nombre de la imagen y la etiqueta utilizadas para crear este contenedor.
- **PUERTOS**: Los puertos que este contenedor está utilizando actualmente.
- **NOMBRES**: Docker asignará automáticamente nombres a los contenedores a menos que usted asigne uno.

Usamos el ID DEL CONTENEDOR para detener, ver y ejecutar el shell bash:

- `docker stop <container-id>`: para detener un contenedor docker
- `logs <container-id>`: para ver los registros de un contenedor
- `docker exec -it <container-id> bash`: para ingresar a un shell bash en un contenedor

Al ejecutar cualquier imagen de contenedor Docker, se le asignará automáticamente un nombre, a menos que se le pase uno con la opción `--name <your-container-name>` (p. ej., el nombre `boring_jepsen` se generó automáticamente en este caso). Podemos usar este nombre asignado (p. ej., las columnas `NAME` de `docker ps` nos dan el nombre o los nombres de nuestros contenedores en ejecución) en lugar de `CONTAINER_ID` para detener, revisar y ejecutar el contenedor `boring_jepsen` :

- `docker stop <nombre-del-contenedor>` o `docker stop boring_jepsen`

- registros de Docker <nombre del contenedor> o registros de Docker boring_jepsen
- docker exec -it <nombre-del-contenedor> bash o docker exec -it
fiesta de aburrido_jepsen

Si eres un lector atento, notarás que hay un puerto asociado a este contenedor y, según la figura 7.3, el puerto es el 80, que es el puerto HTTP estándar. Quizás te sientas tentado a visitar `http://<dirección-ip>` en tu navegador, pero eso no funcionará. ¿Recuerdas por qué?

El puerto interno de Docker no se ha expuesto ni publicado en el puerto externo o del sistema. Hagámoslo ahora.

Antes de reparar nuestro contenedor en ejecución, debemos conocer el comando `docker ps -aq`, ya que devuelve una lista de todos los contenedores en ejecución en un formato fácil de usar con otros comandos. Esto es útil para detener o reiniciar todos los contenedores en ejecución.

La última bandera que debemos discutir es la bandera `--restart always`. Esta bandera garantizará que la aplicación se ejecute si se reinicia Docker, el contenedor falla (por alguna razón) o se reinicia la máquina virtual. Es una bandera ideal para cualquier contenedor que desee ejecutar continuamente. Antes de ejecutar la versión correcta de nuestro contenedor NGINX, detengamos todos los contenedores ejecutando `docker stop $(docker ps -aq)`.

La siguiente lista muestra un ejemplo completo de cómo ejecutar un servicio en segundo plano, reiniciar el contenedor si falla, nombrar nuestra imagen de contenedor, exponer un puerto y ejecutar el contenedor NGINX.

Listado 7.3 Detención de contenedores y publicación del puerto

```
docker run -d --restart siempre --nombre my-nginx -p 80:80 nginx
```

Si necesita detener y reiniciar manualmente este programa en ejecución
Contenedor NGINX, ejecutar `docker stop my-nginx`, `docker rm my-nginx`

para eliminar la imagen del contenedor nombrado y luego repetir el comando del listado 7.3.

Ahora puede visitar `http://<dirección-ip>` en su navegador y ver la página de bienvenida de NGINX. ¡Felicitaciones! Ha implementado su primera aplicación contenedorizada en producción.

Ahora que podemos implementar una aplicación lista para producción, analicemos cómo preparar nuestras aplicaciones usando Docker y Docker Compose antes de bloquear una versión de producción.

7.2 Contenedores de preparación

Si bien implementar continuamente nuevas versiones de las aplicaciones es recomendable, rara vez pasamos del desarrollo a la producción sin entornos de prueba. En otras palabras, habrá un lapso de tiempo entre la versión de producción de la aplicación, que es estable, y la versión de desarrollo, que no lo es (es decir, que aún no está lista para el tráfico en vivo).

El objetivo de esta sección es comprender las técnicas para preparar aplicaciones en contenedores antes de su implementación en producción. Vimos indicios de ello en el capítulo 5, cuando ejecutamos nuestras primeras imágenes de contenedor, y en el capítulo 6, cuando comenzamos a usar Docker Compose. Ahora, lo consolidaremos con varias funciones específicas de Docker.

La primera y más obvia característica es cómo etiquetamos las imágenes al crearlas. Usar etiquetas específicas es una forma rápida y sencilla de agregar versiones a nuestro código y garantizar que implementamos la versión correcta. Las etiquetas son una buena forma de indicar un cambio, pero no modifican nuestros contenedores.

El proceso de preparación de nuestra aplicación, en contenedores o no, consta de varias etapas. La primera etapa es la de desarrollo, y normalmente se realiza en su ordenador local o en un...

Entorno de desarrollo. La segunda etapa es la de preproducción ; puede considerarse como un ensayo para producción, que puede requerir la aprobación de otros miembros del equipo o una revisión personal. La preproducción de una aplicación es una buena práctica, ya que los desarrolladores o miembros del equipo intervienen si algo es técnicamente correcto (es decir, si supera todas las pruebas automatizadas), pero también si no es correcto (es decir, si no cumple con los requisitos de la empresa o del equipo). La etapa final es la de producción , que es la versión en vivo de la aplicación, disponible para el público o los usuarios finales.

Los entornos de prueba a menudo implican cambiar los entornos de ejecución internos de los contenedores (p. ej., Python 3.10 vs. Python 3.11), las configuraciones (p. ej., las variables de entorno), el código en sí (p. ej., correcciones de errores, nuevas funciones, etc.) y más. Algunas maneras de hacerlo son usar un entorno diferente.

- Archivo Docker
- Archivo de Docker Compose
- Archivo Docker Compose y un Dockerfile diferente
- Variables de entorno
- Comandos o scripts de punto de entrada

Cada una es una forma válida de modificar la ejecución de tu aplicación en diferentes etapas. Cambiar o usar diferentes archivos Dockerfile o Docker Compose es una de las formas más comunes de preparar tu aplicación para producción.

7.2.1 Dockerfiles para múltiples entornos

Docker puede usar diferentes Dockerfiles durante la fase de compilación, y es sencillo. La razón principal por la que especifiqué la opción -f

La bandera de Dockerfile en todos los ejemplos de compilación de Docker anteriores tenía como objetivo indicar la simplicidad de usar diferentes Dockerfiles.

Para algunos proyectos, es posible que termines con varios Dockerfiles para diferentes entornos o fases de lanzamiento como

- Dockerfile para desarrollo
- Dockerfile.prod para producción
- Dockerfile.stage para puesta en
- escena Dockerfile.v2 para una versión no publicada 2

Para utilizar estos Dockerfiles, también podríamos etiquetarlos específicamente para cada fase o entorno con

- `docker build -f Dockerfile -t mi-contenedor:dev .`
- `docker build -f Dockerfile.prod -t mi-contenedor:prod .`
- `docker build -f Dockerfile.stage -t mi-contenedor:stage .`
- `docker build -f Dockerfile.v2 -t mi-contenedor:v2 .`

Cuando trabajamos en un entorno de desarrollo, o en una etapa de desarrollo, podemos elegir cualquier etiqueta para cualquier Dockerfile; la flexibilidad es infinita, ya que la imagen del contenedor probablemente se mantendrá local. En cuanto queramos enviar nuestra imagen de contenedor a cualquier lugar, nuestras etiquetas deben ser válidas.

Los registros de contenedor alojarán tu imagen y requieren etiquetas específicas y válidas para alojarlas correctamente. Cuando subimos a Docker Hub en el capítulo 6, vimos que debíamos etiquetar nuestras imágenes con tres atributos específicos: nuestro nombre de usuario de Docker Hub, el nombre del repositorio y una etiqueta. Incluso los registros de Docker autoalojados requieren un etiquetado específico para funcionar correctamente.

Para la gran mayoría de los sistemas de producción, las imágenes de contenedores necesitan un registro de contenedores válido antes de poder implementarlas.

La variación de las fases o etapas de compilación dependerá de la complejidad de las aplicaciones, los requisitos del equipo o la empresa, la cantidad de plataformas para las que se necesita compilar (p. ej., ARM o x86) y la cantidad de entornos de prueba que se necesiten soportar. Para muchos equipos pequeños, es posible que solo se necesiten dos entornos: desarrollo y producción. Los equipos o aplicaciones más grandes suelen utilizar varios entornos y varias versiones mediante el control de versiones semántico (<https://semver.org/>). y muchas plataformas.

La capacidad de usar diferentes Dockerfiles es clave al empezar a mover contenedores a producción, mientras que las etiquetas indican el cambio de estado de la aplicación contenedorizada. A menudo, se tendrá un Dockerfile diferente para la fase de producción y la de desarrollo. Docker Compose también facilita la preparación de múltiples entornos de forma muy sencilla.

7.2.2 Docker Compose para múltiples entornos

Docker Compose también puede usar el indicador `-f <filepath>` para especificar qué archivo de Docker Compose usar y, por lo tanto, qué servicios ejecutar. Cada servicio de Docker Compose está vinculado a un contenedor, que a su vez puede vincularse a un Dockerfile. De nuevo, podemos aprovechar esto para especificar diferentes Dockerfiles para preparar distintos entornos. Aquí hay algunas maneras de preparar diferentes entornos con Docker Compose:

- `docker compose -f compose.yaml arriba`
- `docker compose -f compose.prod.yaml arriba`
- `docker compose -f compose.stage.yaml arriba`

- `docker compose -f docker-compose.yaml arriba`

¿Notas que falta algo? Las configuraciones de Docker Compose están diseñadas para abstraer todos los argumentos que cualquier contenedor (o servicio) pueda necesitar, incluyendo las etiquetas de imagen (p. ej., `my-repo/my-container:prod` o `my-repo/my-container:dev`).

El listado 7.4 contiene un archivo Docker Compose de un capítulo anterior. En este archivo `compose.yaml`, incluí a propósito la línea `dockerfile:` Docker para indicar que también podemos cambiar esto fácilmente.

Listado 7.4 Docker Compose con múltiples Dockerfiles

```
servicios:
  web:
    compilación:
      contexto: .
      dockerfile: Dockerfile
    Puertos:
      - Entorno
    "8080:8080":
      - PUERTO=8080
    volúmenes:
      - mywebdata:/app/data

volúmenes:
  mis datos web:
```

El elemento que omitimos en este archivo de Docker Compose es la posibilidad de especificar una imagen y una etiqueta para dicha imagen. Si no se especifica la imagen o las etiquetas, Docker Compose las configurará automáticamente según el directorio del archivo `compose.yaml`. Para actualizar esto, especificamos las etiquetas en el bloque de compilación y añadimos un bloque de imagen. Copiamos el archivo `compose.yaml` y lo actualizamos a `compose.prod.yaml` con la imagen y la etiqueta, como se muestra en el listado 7.5.

Listado 7.5 Etiquetado de imágenes de Docker Compose

```
...
web:
  imagen: codingforentrepreneurs/rk8s-py:${TAG:-latest} compilación:

  Contexto: .
  dockerfile: Etiquetas de Dockerfile: -

  "codingforentrepreneurs/rk8s-py:${TAG:-latest}"
...
```

`${TAG:-latest}` es una forma de usar la variable de entorno TAG para especificar la etiqueta de la imagen. Si no se establece la variable de entorno TAG, se usará la última etiqueta por defecto. Este es un patrón común para especificar etiquetas en archivos de Docker Compose, y ya hemos visto esta sintaxis con la variable de entorno PORT en nuestros scripts de punto de entrada. Para crear una imagen de contenedor con Docker Compose, podemos ejecutar:

- `docker-compose -f compose.prod.yaml build`: Esto etiquetará la imagen web como `codingforentrepreneurs/rk8s-py:latest`.
- `TAG=v1.0.0 docker-compose -f compose.prod.yaml build`: Esto etiquetará la imagen web como `codingforentrepreneurs/rk8s-py:v1.0.0`.
- `TAG=v1.0.1 docker-compose -f compose.prod.yaml up --build`: Este es el

voluntad
web
imagen
como

 etiqueta `codingforentrepreneurs/rk8s-py:v1.0.1`.

Antes de enviar esto a su repositorio de GitHub, asegúrese de actualizar su repositorio y reemplazar `codingforentrepreneurs` con su nombre de usuario de Docker Hub.

Para asegurarte de haber etiquetado correctamente las imágenes de tu contenedor, puedes ejecutar `docker image ls <image-name>` o `docker image ls codingforentrepreneurs/rk8s-py` en nuestro caso. Esto generará un resultado similar al de la figura 7.4.

```
% docker image ls codingforentrepreneurs/rk8s-py
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
codingforentrepreneurs/rk8s-py	latest	4dddf18b630e	2 hours ago	1.16GB
codingforentrepreneurs/rk8s-py	v1.0.0	4dddf18b630e	2 hours ago	1.16GB

Figura 7.4 Lista de imágenes de Docker

Una vez que etiquetes una imagen, puedes subirla a Docker Hub con `docker push <imagenname> --all-tags`. Antes de subir todas las etiquetas, podrías querer eliminar algunas. Para eliminar las etiquetas de imagen, puedes ejecutar `docker rmi <image-name>:<tag>` o `docker rmi <image-id>`. En mi caso, y según la figura 7.4, sería `docker rmi codingforentrepreneurs/rk8s-py:latest` o `docker rmi 4dddf18b630e`.

En este punto, debe asegurarse de que los directorios de su aplicación local (por ejemplo, `~/dev/roadtok8s/py` y `~/dev/roadtok8s/js`) tengan los siguientes archivos relacionados con Docker en la carpeta raíz:

- Dockerfile y posiblemente Dockerfile.prod
- compose.yaml y compose.prod.yaml

Si no tienes estos archivos, créalos ahora, compártelos en tu repositorio y súbelos a GitHub. Para un resumen rápido,

1. `git add Dockerfile*`: El `*` es un comodín que agregará ambos Dockerfile y Dockerfile.prod.
2. `git add compose*.yaml`: El `*` es un comodín que agregará tanto `compose.yaml` como `compose.prod.yaml`.
3. `git commit -m "Compatibilidad con Docker agregada"`
4. `git push origin main`

Ahora que nuestras aplicaciones preparadas para Docker están en GitHub, creemos un flujo de trabajo de GitHub Action para crear, enviar e implementar nuestros contenedores en un entorno de producción.

7.3 Acciones de GitHub para implementar contenedores de producción

En el capítulo 6, creamos un flujo de trabajo de GitHub Actions para compilar y enviar nuestro contenedor a Docker Hub. En esta sección, ejecutaremos una versión similar de ese flujo de trabajo, modificándolo ligeramente, para compilar, enviar e implementar nuestros contenedores en producción.

Los contenedores de producción suelen compilarse en una máquina distinta a la del host de producción. Esto supone una nueva limitación para la portabilidad de compilación única y ejecución en cualquier lugar que ofrecen los contenedores. Cada chip requiere una compilación diferente. Dicho de otro modo, si utiliza un procesador basado en ARM (por ejemplo, Apple Silicon o Raspberry Silicon), al ejecutar Docker Build , se creará, por defecto, una versión de su contenedor basada en ARM. Esto es excelente si se ejecuta en una máquina basada en ARM, pero no tanto si se ejecuta en una máquina basada en x86. Si bien existen máquinas virtuales basadas en ARM, no son tan comunes como las basadas en x86. En este libro, utilizamos máquinas virtuales basadas en x86.

Para solucionar este problema, usamos GitHub Actions para compilar nuestros contenedores. Los flujos de trabajo de GitHub Actions se ejecutan en máquinas x86 de forma predeterminada. Esto significa que los contenedores que construimos estarán basados en x86 y se ejecutarán en nuestras máquinas virtuales de producción, incluso cuando usemos otras herramientas de orquestación de contenedores como Kubernetes. Existe la opción de realizar compilaciones multiplataforma, pero esto queda fuera del contexto de este libro (consulte la documentación oficial de Docker en <https://docs.docker.com/>). para obtener más información sobre compilaciones multiplataforma).

Antes de sumergirte en esta sección, asegúrate de hacer lo siguiente:

- Cree una nueva máquina virtual en Akamai Connected Cloud (anteriormente Linode) con Ubuntu 22.04 LTS, como se indica en la sección 7.1.1. Si no tiene claves SSH, asegúrese de generarlas también (lo hemos hecho varias veces en este libro).
- Agregue la dirección IP recién creada a los secretos de GitHub Actions de cada aplicación como `AKAMAI_INSTANCE_IP_ADDRESS`.
De nuevo, hemos hecho esto varias veces a lo largo de este libro. Es importante usar una máquina virtual nueva, ya que no queremos que una máquina virtual previamente configurada interfiera con nuestra nueva configuración.

Los pasos reales que seguirá nuestro flujo de trabajo de GitHub Actions son los siguientes:

1. Revisa nuestro código para que podamos construir una imagen.
2. Inicie sesión en Docker Hub.
3. Construye nuestro contenedor con nuestro Dockerfile.
4. Etiquete nuestro contenedor con etiquetas relevantes.
5. Empujamos nuestro contenedor a Docker Hub.
6. Conéctese por SSH a nuestra máquina virtual de producción.
7. Verifique que Docker esté instalado; si no, instálelo. Si Docker no está instalado, instale también Docker Compose.
8. Copie nuestro archivo `compose.prod.yaml` a nuestra máquina virtual de producción.
9. Cree un archivo `dotenv` (`.env`) y cópielo a nuestra máquina virtual de producción.
10. Extraiga la última imagen del contenedor con `docker compose pull`.
11. Ejecute nuestro contenedor con `docker compose up -d`.
12. Limpia nuestra conexión SSH y el archivo `dotenv`.

Cada uno de estos pasos se traducirá en pasos del flujo de trabajo de GitHub Actions. Con esto en mente, comencemos con nuestro flujo de trabajo.

7.3.1 Creación y alojamiento de un contenedor de producción

En este punto, debemos tener muy claro que, desde un punto de vista técnico, un contenedor de producción es igual a uno que no es de producción. La única diferencia radica en el entorno en el que se ejecuta.

Para construir y alojar nuestro contenedor de producción, vamos a volver a implementar el flujo de trabajo de GitHub Actions del capítulo 6. Si se siente cómodo con ese flujo de trabajo, este proceso será el mismo, con algunos cambios menores. Para empezar, diseñemos nuestro flujo de trabajo para lograr lo siguiente:

- Consulte nuestro código (básicamente, cópielo).
- Inicia sesión en Docker Hub. Usa los secretos de GitHub Actions: `DOCKERHUB_USERNAME` y `DOCKERHUB_TOKEN`.
- Crea y etiqueta nuestro contenedor. Usa el secreto `DOCKERHUB_REPO` de GitHub Actions junto con la variable de entorno `GITHUB_SHA` de GitHub Actions.
- Envíe nuestra imagen de contenedor etiquetada a Docker Hub.

Como recordatorio, tu nombre de usuario de Docker Hub (`DOCKERHUB_USERNAME`) debe estar almacenado en los secretos de acción de tu repositorio de GitHub. El token de Docker Hub (`DOCKERHUB_TOKEN`) es un token que puedes crear en la configuración de tu cuenta de Docker Hub. El repositorio de Docker Hub (`DOCKERHUB_REPO`) es el nombre de su repositorio en Docker Hub; puede ser el mismo que el nombre de su repositorio de GitHub, pero no tiene que serlo.

Verifique que los valores de los secretos de Acciones de GitHub que se usan en sus flujos de trabajo (por ejemplo, en `.github/workflows/`) sean los mismos que los que se encuentran en los secretos de Acciones almacenados en el repositorio de Github y que no hayan cambiado con respecto a lo que hicimos en el capítulo 6. Si bien sus valores no deberían haber cambiado, es importante que permanezcan igual que en el capítulo 6 para garantizar que nuestro nuevo flujo de trabajo funcione como se espera.

En una de sus aplicaciones locales (por ejemplo, `~/dev/roadtok8s/py` o `~/dev/roadtok8s/js`), cree un archivo `.github/workflow/build-push-deploy.yaml` que comience con el contenido del listado 7.6.

Puedes trabajar en ambos proyectos al mismo tiempo si lo deseas, pero te recomiendo que te quedes con uno hasta que funcione siempre.

Listado 7.6 Flujo de trabajo de GitHub Actions para crear y enviar contenedores, parte 1

Nombre: Compose - Construir, enviar e implementar

en:

flujo de trabajo_despacho:

trabajos:

implementar:

se ejecuta en: ubuntu-latest

Como recordatorio, el bloque "runs-on" nos indica que usaremos ubuntu-latest para ejecutar los trabajos de este paso, que terminarán creando nuestra imagen de contenedor. Por supuesto, también usamos Ubuntu en nuestra máquina virtual de producción.

Antes de comenzar a declarar los pasos, podemos configurar las variables de entorno del trabajo con la declaración `env:`. Como se mencionó anteriormente, esto puede ayudar a que nuestro flujo de trabajo sea más conciso y fácil de mantener. La siguiente lista muestra las declaraciones `env:` necesarias para este trabajo en este flujo de trabajo.

Listado 7.7 Flujo de trabajo de GitHub Actions para crear y enviar contenedores, parte 2

se ejecuta en: ubuntu-latest

entorno:

```
DH_USER: $glossterm::[ secretos.DOCKERHUB_NOMBRE_USUARIO ]
REPO: $glossterm::[ secretos.DOCKERHUB_REPO ]
SSH_OPTS: '-o Comprobación estricta de clave de host = no'
REMOTO: 'root@$glossterm::[ secrets.AKAMAI_INSTANCE_IP_ADDRESS ]'

DIRECCIÓN_DE_APLICACIÓN_REMOTA: '/opt/app'
```

Las nuevas incorporaciones al bloque env son las siguientes:

- **SSH_OPTS:** cuando usamos ssh y scp en un flujo de trabajo automatizado, es posible que necesitemos omitir la verificación de la clave del host dada la naturaleza efímera y sin servidor de GitHub Actions. El uso del indicador -o StrictHostKeyChecking=no omitirá esta verificación.
- **REMOTO:** Este es el usuario remoto y la dirección IP de nuestra máquina virtual de producción. Lo usaremos para acceder por SSH o copiar archivos (vía SCP) a nuestra máquina virtual de producción.
- **REMOTE_APP_DIR:** este es el directorio de destino para nuestro archivo Docker Compose compose.prod.yaml y nada más.

Con estos elementos en mente, configuremos los pasos para construir y enviar nuestra imagen de contenedor a Docker Hub con los valores de la siguiente lista.

Listado 7.8 Flujo de trabajo de GitHub Actions para crear y enviar contenedores, parte 3

```

...
trabajos:
  implementar:
    ...
    entorno:
      ...

Pasos: -
Nombre: El código de pago utiliza:
  acciones/checkout@v3 - Nombre: Iniciar
sesión en Docker Hub
  usos: docker/login-action@v2 con: nombre de
  usuario:
    ${{ secrets.DOCKERHUB_USERNAME }} contraseña: $
    {{ secrets.DOCKERHUB_TOKEN }} - nombre: Construye la imagen
de Docker ejecutar: | docker build -f Dockerfile \

    -t "$DH_USER/$REPO:último" \ -t
    "$DH_USER/$REPO:${{ github.sha }}" \
    .
- nombre: Enviar la imagen de Docker a Docker Hub
  ejecutar:
    | docker push "$DH_USER/$REPO" --all-tags

```

Con este código disponible, confirma y envía el código a tu repositorio de GitHub (por ejemplo, `git add .` y `git push origin main`). Después, confirma que este flujo de trabajo se ejecuta correctamente en GitHub Actions. Aunque debería ejecutarse correctamente según nuestras acciones anteriores, debemos verificarlo antes de continuar.

Después de verificar que puede volver a crear y enviar imágenes de contenedores a Docker Hub, comencemos el proceso de configuración de nuestra máquina virtual de producción.

7.3.2 Instalación de Docker y Docker Compose en una máquina virtual con GitHub Actions

Anteriormente en este capítulo, vimos lo fácil que es instalar Docker y Docker Compose en una máquina virtual con solo unos pocos comandos. Haremos lo mismo aquí, pero con GitHub Actions.

Quizás te preguntes si deberíamos usar Ansible para configurar nuestra máquina virtual, tal como lo hicimos en el capítulo 4. Si bien es una opción válida que podría considerar adoptar en el futuro, la simplicidad de instalar Docker con el script de acceso directo significa que Ansible no es necesario por el momento. Esto es lo que haremos en lugar de usar Ansible:

- Comando `-v docker`: Esta es una comprobación sencilla para ver si Docker está instalado. Si es así, hemos terminado. Si no, necesitamos instalar Docker.
- `curl -fsSL https://get.docker.com -o get-docker.sh` y `sudo sh get-docker.sh`: estos son los mismos comandos que usamos para instalar Docker en la sección 7.1.
- `sudo apt-get update` y `sudo apt-get install docker-compose-plugin`: este es el mismo comando que usamos para instalar Docker Compose en la sección 7.2.

Podemos usar estos comandos para crear una declaración de condición que se pueda ejecutar con una conexión SSH. La siguiente lista configura nuestra clave SSH y muestra cómo se usan estos comandos.

Listado 7.9 Instalación de Docker en una máquina virtual con flujo de trabajo de GitHub Actions y SSH

```

...
trabajos:
  implementar:
    ...
    entorno:
      ...
      SSH_OPTS: '-o Comprobación estricta de clave de host = no'
      REMOTO: 'root@${ secrets.DIRECCIÓN IP DE LA INSTANCIA DE AKAMAI
}}'
pasos:
  ....
  - nombre: Implementar la clave SSH privada
    ejecutar:
      | mkdir -p ~/.ssh/ echo
      | "${ secrets.SSH_PRIVATE_KEY }" > ~/.ssh/id_rsa chmod 600 ~/.ssh/
      | id_rsa - nombre: Asegúrese de
que Docker y Docker Compose estén instalados en

Máquina virtual

    ejecutar:
      | ssh $SSH_OPTS $REMOTE << EOF
      | si ! comando -v docker &> /dev/null; entonces
      | curl -fsSL https://get.docker.com -o obtener-docker.s
h

      | sudo sh get-docker.sh rm get-
      | docker.sh # instalar Docker
      | Compose sudo apt-get update sudo apt-
      | get install docker-compose-
      | plugin fi EOF

```

Si ejecuta este flujo de trabajo, Docker y Docker Compose deberían estar instalados en su nueva máquina virtual. Verifique esto con una nueva sesión SSH en su máquina local con `ssh root@<dirección-ip> docker --version` y `ssh root@<dirección-ip>`.

Versión de Docker Compose. Al ejecutar estos comandos, se obtendrá un resultado como el de la figura 7.5.

```
% ssh root@172.232.161.201 docker --version
Docker version 24.0.5, build ced0996
% ssh root@172.232.161.201 docker compose version
Docker Compose version v2.20.2
```

Figura 7.5 Versión de Docker Compose

NOTA: Si tiene algún error de SSH en su máquina local, es posible que necesite volver a revisar cómo instalar sus claves SSH desde la sección 4.1 o consultar el apéndice C para obtener más información sobre las claves SSH.

Ahora que tenemos Docker y Docker Compose instalados en nuestra máquina virtual, ejecutemos el flujo de trabajo dos veces más.

Verá que este nuevo paso de instalación debería ejecutarse muy rápido, ya que Docker ya está instalado. También puede considerar aprovisionar una nueva máquina virtual para ver cuánto tarda en instalarse Docker y Docker Compose desde cero.

Continuemos con el siguiente paso de nuestro flujo de trabajo, utilizando Docker Compose para ejecutar nuestra aplicación de producción.

7.3.3 Uso de GitHub Actions para ejecutar Docker Compose en producción

Como ya hemos visto, Docker Compose facilita enormemente su uso. La versión de producción también es muy fácil de usar. Aquí hay algunas condiciones que ya hemos establecido para garantizar la entrega fluida de una versión de producción de nuestras aplicaciones contenedorizadas basadas en Docker Compose:

- Un Dockerfile listo para producción

- Un archivo Docker Compose listo para producción (por ejemplo, `compose.prod.yaml`)
- Una imagen de contenedor lista para producción
- Una máquina virtual host lista para producción basada en la nube
- Imágenes de contenedor alojadas en Docker Hub (públicas o privadas)
- Docker y Docker Compose están instalados en la máquina host
- Valores secretos de producción almacenados en secretos de GitHub Actions para nuestros repositorios de aplicaciones

Nuestro flujo de trabajo de GitHub Actions ya cumple varias de estas condiciones, así que comencemos usando imágenes de contenedor de Docker Hub alojadas de forma privada. Para usar imágenes privadas en producción, solo se necesita un paso más: iniciar sesión en Docker Hub mediante Docker Login. La siguiente lista muestra cómo iniciar sesión en Docker Hub con GitHub Actions y SSH.

Listado 7.10 Inicio de sesión en Docker Hub en una máquina virtual mediante SSH

```

...
trabajos:
  implementar:
    ...
    entorno:
      ...
      SSH_OPTS: '-o Comprobación estricta de clave de host = no'
      REMOTO: 'root@${ secrets.DIRECCIÓN IP DE LA INSTANCIA DE AKAMAI
}}'
  pasos:
    ...
    - nombre: Asegúrese de que Docker y Docker Compose estén instalados en
      Máquina virtual
      ...
      - nombre: Inicie sesión en Docker Hub en la
        máquina
        virtual ejecutar: | ssh $SSH_OPTS
          $REMOTE << EOF docker login -u $DH_USER -p ${ secrets.DOCKERHUB_T
OKEN }}
          EOF

```

La parte importante de este paso es asegurarse de que se realice a través de una conexión SSH y no a través de una acción de GitHub como el paso `docker/login-action@v2`.

Ahora, agreguemos nuestras variables de entorno a la ubicación `REMOTE_APP_DIR`. El archivo de variables de entorno, o archivo `dotenv` (`.env`), se creará dentro del flujo de trabajo de GitHub Actions a partir del secreto de GitHub Actions almacenado y luego se copiará a nuestra máquina virtual de producción. Una vez creado este archivo, también copiaremos `compose.prod.yaml` como `compose.yaml` a nuestra máquina virtual de producción para permitir el uso de `docker compose up -d` sin especificar un archivo de composición. La siguiente lista muestra cómo hacerlo.

Listado 7.11 Uso de SSH y SCP para agregar secretos de Dotenv a una máquina virtual

```

...
trabajos:
  implementar:
    ...
    entorno:
      ...
      SSH_OPTS: '-o Comprobación estricta de clave de host = no'
      REMOTO: 'root@${ secrets.DIRECCIÓN IP DE LA INSTANCIA DE AKAMAI
}}'

      REMOTE_APP_DIR: pasos '/opt/
app':
  ...
  - nombre: Crear archivo .env ejecutar: |
    echo
      "MY_SECRET_KEY=${ secrets.MY_SECRET_KEY }}" >>
.env
      echo "API_ENDPOINT=${ secrets.API_ENDPOINT }}" >> .e
Nevada

  - nombre: Asegurarse de que exista el directorio remoto
    ejecutar:
      | ssh $SSH_OPTS $REMOTE mkdir -p $REMOTE_APP_DIR -
  nombre: Copiar el archivo .env de GitHub Actions a la VM remota
    ejecutar:
      | scp $SSH_OPTS .env $REMOTE:$REMOTE_APP_DIR/.env

```

El archivo .env se crea directamente en el flujo de trabajo de GitHub Actions, por lo que podemos copiarlo a nuestra máquina virtual y, eventualmente, Docker Compose puede usarlo al ejecutar `docker compose up`.

NOTA: Desde una perspectiva de seguridad, podría ser una buena idea cifrar y descifrar el archivo .env durante este proceso, pero hacerlo está fuera del alcance de este libro. Proteger los secretos es un proceso continuo, por lo que debe mantenerse alerta. HashiCorp Vault y Google Secrets Manager son opciones válidas para el tiempo de ejecución.

almacenamiento secreto, pero aún así es posible que requieran mover secretos de un lugar a una o más máquinas virtuales de producción.

Tenga en cuenta que es necesario crear la carpeta de la aplicación (por ejemplo, `REMOTE_APP_DIR`) en nuestra máquina antes de copiar el archivo `.env` a esa carpeta:

- `ssh $SSH_OPTS $REMOTE mkdir -p $REMOTE_APP_DIR` crea la carpeta en la VM remota.
- `scp $SSH_OPTS .env $REMOTE:$REMOTE_APP_DIR/.env` copia el archivo `.env` a la carpeta.
- `REMOTE_APP_DIR` es la definición de carpeta que creamos en el bloque `env`: de nuestro flujo de trabajo.

Ahora que hemos creado nuestro `.env` en nuestra máquina virtual, necesitamos transferir el archivo de configuración de Docker Compose. Para asegurarnos de que tengamos la misma configuración, crearemos los archivos `compose.prod.yaml` para cada aplicación (por ejemplo, `~/dev/roadtok8s/py/compose.prod.yaml` y `~/dev/roadtok8s/js/compose.prod.yaml`).

Primero, revisaremos el archivo `compose.prod.yaml` de nuestra aplicación Python. Hay algunos atributos clave que vale la pena observar en este archivo:

- **Puertos:** Se establece en `80:8080`. El puerto `80` es el recomendado para exponer aplicaciones Docker de producción (p. ej., `http://my-ip:80` es igual que `http://my-ip` en lo que respecta a los navegadores web). **Archivo_`env`:** Se establece en `.env`, el archivo que creamos en
- el flujo de trabajo de GitHub Actions. Esta definición asume una ruta relativa, lo que significa que `.env` y nuestra configuración de Docker Compose deben estar en el mismo directorio.

- compilación: Esto ya no se incluye, ya que el archivo de composición de producción solo se usa para ejecutar imágenes. imagen: Está
- configurada como `codingforentrepreneurs/rk8s-py:${TAG:-latest}`, lo que nos permite configurar la etiqueta con la variable de entorno TAG en la máquina virtual (por ejemplo, no en `.env`). Si la variable de entorno TAG no está configurada, se usará la última versión por defecto. imagen puede ser pública o privada, ya que nuestro flujo de trabajo de GitHub Actions iniciará sesión en Docker Hub antes de ejecutar `docker compose up -d` en nuestra máquina virtual.
- reiniciar: siempre: esto se configura para garantizar que nuestro contenedor se ejecute continuamente y se reiniciará si falla por algún motivo.

Cree `compose.prod.yaml` en la raíz de su aplicación Python (por ejemplo, `~/dev/roadtok8s/py/compose.prod.yaml`) con el contenido de la siguiente lista.

Listado 7.12 Docker Compose para producción Python

```
servicios:
  web:
    Reiniciar: siempre. Imagen:
    codingforentrepreneurs/rk8s-py:${TAG:-latest}. Puertos: - "80:8080".

    ambiente:
      - PUERTO=8080
    archivo_env:
      - .env
```

Repita este proceso para nuestra aplicación JavaScript Node.js en `~/dev/roadtok8s/js/compose.prod.yaml`. El Listado 7.13 muestra el archivo `compose.prod.yaml` para nuestra aplicación Express.js. Observe que se ha actualizado a nuestra aplicación Python con algunos cambios menores.

Listado 7.13 Docker Compose para producción Node.js

Servicios: web:

reiniciar: siempre Imagen:

codingforentrepreneurs/rk8s-js:\${TAG:-latest} Puertos: - "80:3000"

ambiente:

- PUERTO=3000

archivo_env:

- .env

La principal diferencia es el valor del PUERTO predeterminado en el que se ejecuta el contenedor; fuera de eso, los archivos son casi idénticos.

Ahora que tenemos nuestros archivos de configuración de producción de Docker Compose, actualicemos nuestro flujo de trabajo de GitHub Actions para copiar este archivo, extraer imágenes y ejecutar nuestros contenedores. Los puntos clave a tener en cuenta son los siguientes:

- Cambiar el nombre a compose.yaml significa que no necesitamos especificar el archivo que usaremos.
- compose.yaml debe existir junto a .env en nuestra máquina virtual (por ejemplo, /opt/app/.env y opt/app/compose.yaml).
- Al ejecutar docker compose pull en nuestro REMOTE_APP_DIR se extraerán las últimas imágenes de Docker Hub en función de nuestro archivo compose.yaml.
- Ejecutar docker compose up -d en nuestro REMOTE_APP_DIR ejecutará nuestro(s) contenedor(es) en segundo plano en los puertos correctos.

La siguiente lista muestra cómo hacer esto para el archivo compose.prod.yaml de cualquier aplicación .

Listado 7.14 Copia de configuración, extracción de imágenes y ejecución de Docker Compose

```
...
trabajos:
  implementar:
    ...
    entorno:
      ...
      SSH_OPTS: '-o Comprobación estricta de clave de host = no'
      REMOTO: 'root@${ secrets.DIRECCIÓN IP DE LA INSTANCIA DE AKAMAI
}}'

      REMOTE_APP_DIR: pasos '/opt/
app':
  ...
  - nombre: Copiar compose.prod.yaml a la máquina virtual
    ejecutar:
      | scp $SSH_OPTS compose.prod.yaml $REMOTE:$REMOTE_APP_D IR/compose.yaml -
nombre: Extraer imágenes
actualizadas ejecutar: | ssh $SSH_OPTS
$REMOTE
<< EOF cd $REMOTE_APP_DIR docker compose
pull EOF - nombre: Ejecutar
Docker Compose ejecutar: | ssh

$SSH_OPTS $REMOTE << EOF cd

$REMOTE_APP_DIR # ejecutar contenedores docker
compose up -d EOF
```

Antes de confirmar este archivo, suele ser recomendable limpiar nuestra conexión SSH y eliminar el archivo .env del flujo de trabajo de GitHub Actions. Si bien esto puede no ser estrictamente necesario, ya que estos flujos de trabajo se destruyen una vez completados, considero recomendable asegurar el flujo de trabajo.

Limita cualquier posible filtración de información secreta o problemas de seguridad. La siguiente lista muestra cómo hacerlo.

Listado 7.15 Limpieza de SSH y el archivo dotenv en GitHub Actions

```
...
trabajos:
  implementar:
    ...
    entorno:
      ...
    pasos:
      ...
      - nombre: Limpiar archivo .env
        ejecutar: rm .env
      - nombre: Limpiar la clave privada SSH
        ejecutar: rm ~/.ssh/id_rsa
```

Asegúrate de agregar MY_SECRET_KEY y API_ENDPOINT como valores de marcador de posición en tus secretos de GitHub Actions en tus repositorios de GitHub. Cada vez que necesites agregar o eliminar variables de entorno a tu aplicación, deberás actualizar el paso de definición .env del flujo de trabajo de GitHub Actions , actualizar los secretos de GitHub Actions en tu repositorio, enviar los cambios y ejecutar el flujo de trabajo.

En este punto, confirma el archivo de flujo de trabajo (git add .github/workflows/) y la configuración de Docker Compose (git add compose.prod.yaml), envíalo (git push origin main) a tu repositorio de GitHub y, a continuación, ejecuta el flujo de trabajo de GitHub Actions. Si encuentras algún error o problema, revisa los registros y los secretos de GitHub Actions.

Si no hay errores, ¡enhorabuena! ¡Ya has implementado una aplicación contenedorizada de producción con Docker Compose y GitHub Actions! Te sugiero que...

Modificaciones a su aplicación ahora y ver si el flujo de trabajo está funcionando correctamente.

Llegar a este punto requirió varios elementos de configuración; el proceso ahora es repetible y puede usarse para cualquier número de aplicaciones. Antes de continuar con el siguiente capítulo, analicemos algunos de los desafíos con Docker Compose y los métodos que implementamos aquí.

7.4 Las limitaciones del uso de Docker Compose para producción

A medida que tus proyectos crezcan o se vuelvan más complejos, descubrirás que Docker Compose tiene algunas limitaciones importantes. Aquí tienes algunas que quiero destacar:

- Escalado de instancias de contenedores: si bien técnicamente puedes agregar servicios, no puedes escalar fácilmente varias instancias del mismo contenedor en ejecución.
- Comprobaciones de estado: Docker Compose no tiene un mecanismo de comprobación de estado integrado más allá de si el contenedor está en ejecución o no.
- Acceso remoto: Docker Compose no tiene una forma integrada de acceder de forma remota al entorno de ejecución de un contenedor (por ejemplo, `docker compose run <service-name> bash` sin SSH).
- Compatibilidad con múltiples máquinas virtuales: Docker Compose no tiene una forma integrada de ejecutar contenedores en múltiples máquinas virtuales de forma predeterminada.
- Gestión de ingreso: si bien es técnicamente posible con un servicio NGINX, no existe una forma nativa de gestionar el tráfico entrante con Docker Compose (por ejemplo, dirección IP/api va al servicio API y dirección IP va al servicio web).

- Aprovisionamiento de direcciones IP públicas: Docker Compose no cuenta con una forma integrada de aprovisionar una dirección IP pública para un contenedor o servicio. Como veremos en el siguiente capítulo, Kubernetes administrado ofrece esta posibilidad. Esto significa que puede aprovisionar una dirección IP pública para un contenedor o servicio sin necesidad de aprovisionar una máquina virtual.

Si bien existen otras limitaciones que no abordaremos aquí, podrían convencerte de aprovechar Kubernetes, como veremos en el próximo capítulo. Antes de hacerlo, veamos un ejemplo de cómo escalar contenedores con Docker Compose, independientemente de si se encuentra en producción o en desarrollo.

7.4.1 Escalado de contenedores con Docker Compose

Docker Compose no tiene una forma integrada de escalar contenedores, por lo que usaremos NGINX para brindarnos un enfoque sobre cómo podemos beneficiarnos al ejecutar el mismo contenedor varias veces como servicios de Docker Compose duplicados.

NGINX cuenta con varias funciones que ya hemos visto en este libro. Una de ellas es el reenvío de solicitudes a diferentes aplicaciones según una ruta.

Como recordatorio, podemos reenviar la ruta `/about` a una aplicación

Python y la ruta `/api` a una aplicación Node.js. Esto funciona porque NGINX puede reenviar solicitudes en la máquina local a diferentes puertos de esa misma máquina (por ejemplo, `localhost:8080` y `localhost:3000`).

Es importante recordar que, en nuestro host local, no podemos ejecutar varias aplicaciones en el mismo puerto. Los puertos están reservados para un proceso (o aplicación) a la vez. Por eso usamos puertos diferentes para cada aplicación. Pero ¿qué pasa si usamos una dirección IP diferente en lugar de `localhost`?

¿Puede NGINX seguir reenviando la solicitud independientemente del puerto? La respuesta es una gran noticia: ¡sí, se puede! NGINX puede reenviar fácilmente solicitudes a diferentes máquinas virtuales mediante direcciones IP.

NGINX tiene un as bajo la manga: el balanceo de carga. Ahora entendemos que NGINX puede reenviar solicitudes a múltiples destinos (p. ej., /about y /api) mientras son atendidas por diferentes aplicaciones. A NGINX solo le importa que el destino de la solicitud se atienda correctamente o, en otras palabras, que la aplicación en `http://localhost:8080` devuelva una respuesta válida a /about. A NGINX no le importa qué aplicación esté atendiendo la solicitud. Por lo tanto, NGINX puede reenviar solicitudes a una lista de múltiples destinos; es decir, podemos proporcionar a NGINX varios destinos válidos para una ruta determinada. Esto nos permite, presumiblemente, tener la misma aplicación ejecutándose en `http://localhost:8080`, `http://localhost:8081`, `http://mi-dirección-ip:80` y `http://mi-otra-dirección-ip:80`, y NGINX rotará las solicitudes entre estos destinos. Las solicitudes rotativas se pueden definir de diferentes maneras, como Round Robin (una tras otra), Round Robin ponderado (se prefiere una más que otra), Menos conexiones (la menos ocupada), Hash de IP (basado en la dirección IP de la solicitud) y otras. NGINX usa Round Robin por defecto, pero puede configurarse para usar otros métodos (conocidos como algoritmos de balanceo de carga), aunque esto queda fuera del alcance de este libro.

El listado 7.16 muestra un ejemplo básico de un archivo de configuración NGINX para implementar el equilibrio de carga.

Listado 7.16 Conceptos básicos del equilibrio de carga de NGINX

```
upstream myapp
{
    servidor localhost:8080;
    servidor localhost:8081;
    servidor mi-dirección-ip:80;
    servidor mi-otra-dirección-ip:80;
}

servidor
{
    escuchar
    80; nombre_del_servidor localhost;

    ubicación / {
        contraseña_proxy http://miaplicación;
    }

    ubicación /api
    {
        proxy_pass http://localhost:3000;
    }
}
```

La definición de <your-name> ascendente es cómo podemos especificar los distintos destinos que queremos utilizar.

Ahora podemos modificar esta configuración para que funcione con los servicios de Docker Compose. Aunque no vamos a implementar esto en nuestras aplicaciones actuales, los siguientes listados pueden servir como ejemplos que puedes probar por tu cuenta. Estos son los pasos que debemos seguir para configurar NGINX y equilibrar la carga entre varios servicios de Docker Compose:

1. Defina compose.yaml con múltiples nombres de servicio.
Para este ejemplo, usaremos bruce, tony y thor como nombres de nuestros servicios.
2. Con los nombres de servicio como nuestros nombres de host (es decir, reemplazando la dirección IP y el host local), configuraremos nuestra definición ascendente en un archivo de configuración NGINX.

3. Actualice la configuración de `compose.yaml` con un servicio NGINX que use el nuevo archivo de configuración NGINX.

El listado 7.17 muestra un archivo Docker Compose minimizado con solo los servicios de la aplicación listados, sin la definición de puertos . Cabe recordar que Docker Compose permite la comunicación entre servicios; por lo tanto, no se necesitan puertos para exponer estos servicios al host.

Listado 7.17 Docker Compose con múltiples servicios del mismo contenedor

```
servicios:
  bruce:
    imagen: codingforentrepreneurs/rk8s-py reiniciar: siempre tony:
    imagen:

    codingforentrepreneurs/rk8s-py reiniciar: siempre thor: imagen:
      codingforentrepreneurs/
rk8s-py
    reiniciar: siempre scott: imagen: codingforentrepreneurs/rk8s-js
    reiniciar: siempre
```

Con estos servicios declarados, veamos un archivo de configuración NGINX (por ejemplo, `custom-nginx.conf`) que podríamos usar para implementar estos diferentes servicios, en el listado 7.18.

Listado 7.18 Configuración ascendente de NGINX

```
upstream myapp
{
    servidor bruce:8080;
    servidor tony:8080;
    servidor thor:8080;
    servidor scott:3000;
}

servidor
{
    escuchar
    80; nombre_del_servidor localhost;

    ubicación / {
        contraseña_proxy http://miaplicación;
    }
}
```

Ahora vemos bruce:8080, tony:8080, thor:8080 y scott:3000 como nuestros destinos upstream en lugar de localhost o una dirección IP. Docker Compose dirigirá automáticamente estas solicitudes al servicio correcto, ya que los nombres de servicio coinciden con los nombres de host. Los puertos 8080 y 3000 son los puertos predeterminados definidos en las imágenes de contenedor para los diferentes servicios referenciados.

Finalmente, para actualizar nuestra configuración de Docker Compose, necesitamos usar nuestro archivo de configuración NGINX personalizado (por ejemplo, custom-nginx.conf). El Listado 7.19 muestra cómo hacerlo.

Listado 7.19 Docker Compose con equilibrio de carga de servicio en NGINX

```
servicios:
  nginx:
    imagen: nginx
    puertos:
      - 8000:80
    volúmenes:
      - ./custom-nginx.conf:/etc/nginx/conf.d/default.c
encendido
Bruce:
  ...
Tony:
  ...
Thor:
  ...
Scott:
  ...
```

Con esta configuración, podría ejecutar Docker Compose Up, navegar a localhost:8000 y debería ver el balanceo de carga de NGINX en acción. Si recarga la página suficientes veces, debería ver las diferentes instancias de contenedor en uso. Modifique este ejemplo como considere oportuno para comprender mejor cómo funciona.

Si bien es muy útil usar NGINX para equilibrar la carga de servicios en Docker Compose, aún presenta varios desafíos que escapan a lo que Docker Compose puede hacer de forma realista. En el siguiente capítulo, analizaremos cómo Kubernetes puede ayudarnos a resolver muchos de los desafíos relacionados con el escalado y la gestión de múltiples contenedores.

Resumen

- Instalar Docker en máquinas virtuales generalmente es más fácil que en la mayoría de lenguajes y marcos de programación.
- Docker Compose es una forma eficaz de ejecutar, administrar y actualizar contenedores en desarrollo y producción.
- Los volúmenes administrados de Docker Compose pueden causar problemas importantes, incluida la eliminación de datos, si no se configuran correctamente.
- GitHub Actions proporciona una forma eficaz de crear contenedores listos para producción que funcionan en máquinas virtuales basadas en x86 de forma predeterminada.
- Docker Compose puede administrar un puñado de contenedores y servicios, pero comienza a fallar a medida que escalamos.
- El equilibrio de carga con NGINX es una forma efectiva de escalar aplicaciones, en contenedores o no, sin necesidad de utilizar métodos de orquestación más robustos.
- El uso de documentos, como `compose.yaml` o un flujo de trabajo de GitHub Actions, para administrar configuraciones es una forma eficaz de garantizar la coherencia y la repetibilidad, al tiempo que proporciona una única fuente de verdad para la configuración de una aplicación.

8 Kubernetes administrado

Despliegue

Este capítulo cubre

- Aprovisionamiento de un clúster de Kubernetes administrado
- Instalación de la CLI de Kubernetes (kubectl)
- Conexión a un clúster con kubectl
- Ejecutar un contenedor con una implementación
- Exponer una implementación con un servicio
- Comunicación de contenedor a contenedor con DNS
- Contenedores de equilibrio de carga con servicios de ingreso

Hemos llegado a nuestro destino principal: el mundo de Kubernetes. En pocas palabras, Kubernetes ejecuta y gestiona contenedores en un clúster de máquinas virtuales. Kubernetes puede iniciar, detener y reiniciar contenedores; ejecutar de 0 a N instancias de contenedor; cambiar o revertir versiones; facilitar la comunicación entre contenedores a escala; gestionar volúmenes y almacenamiento; inyectar variables de entorno; y mucho más. K8s, abreviatura de Kubernetes (se pronuncia "kay eights"), también cuenta con un enorme ecosistema de terceros que amplía aún más sus capacidades. La gran cantidad de funciones y herramientas de terceros puede hacer que aprender Kubernetes parezca abrumador. Este capítulo se centra en algunas funciones clave de Kubernetes que permitirán dos objetivos principales: implementar nuestras aplicaciones y sentar las bases prácticas para aprender temas más avanzados de Kubernetes.

A partir del capítulo 5, aprendimos que los contenedores abren un nuevo mundo de portabilidad, a la vez que reducen la sobrecarga en la configuración del sistema de producción. Esta reducción se debe a que, con las aplicaciones contenedorizadas, nuestras máquinas virtuales generalmente solo necesitan un entorno de ejecución de contenedor (p. ej., Docker Engine) para ejecutar nuestras aplicaciones con muy poca configuración adicional. Dicho de otro modo, el uso de contenedores agiliza el proceso de implementación al dedicar mucho menos tiempo a configurar las máquinas virtuales para diversos entornos de ejecución (p. ej., Python, Node.js o Java), a la vez que reduce el tiempo necesario para la configuración adicional específica de la aplicación en la máquina virtual. Si bien las aplicaciones contenedorizadas aún requieren configuraciones específicas para su ejecución, este tipo de configuración puede ocurrir mucho antes de que la aplicación llegue a una máquina virtual o entorno de producción.

Si tienes un contenedor y un entorno de ejecución de contenedores, tu aplicación se puede implementar prácticamente en cualquier lugar. Si bien esta afirmación tiene algunas salvedades, como la necesidad de crear imágenes independientes para las plataformas x86 y ARM, las aplicaciones en contenedores pueden moverse entre ubicaciones de alojamiento casi con la misma facilidad con la que las empresas de alojamiento pueden cobrarte.

¿Necesitas un sistema de producción local? ¡Genial! Puedes usar contenedores.

¿Necesitas migrar esos contenedores para lograr una mayor escalabilidad usando la nube y sin invertir en nuevo hardware? ¡Genial! Puedes usar los mismos contenedores.

¿Necesitas distribuir la aplicación a otras partes del mundo con un balanceador de carga? ¡Genial! Puedes usar los mismos contenedores.

¿Necesitas migrar la aplicación a otro proveedor de nube? ¡Genial! Puedes usar los mismos contenedores.

La lista continúa mientras se plantea el punto: los contenedores son la forma más portátil de implementación de aplicaciones.

Si bien los contenedores resuelven un desafío enorme en la implementación de aplicaciones, la tecnología no fue la primera en hacerlo. Popularizado por Heroku y otras plataformas como servicio

En las empresas (PaaS), los buildpacks permiten a los desarrolladores de aplicaciones enviar su código a través de Git, y luego la herramienta configura automáticamente el servidor o la máquina virtual. Con los buildpacks, los desarrolladores crean un archivo de configuración simple, llamado Procfile, que contiene una sola línea para ejecutar la aplicación (p. ej., web: gunicorn main:app). Además del Procfile, los buildpacks usan el código para inferir cómo configurar el servidor. Esto suele significar que si se encuentra requirements.txt en el código, se usará Python. Si es package.json, se usará Node.js. Tenga en cuenta que omití a propósito la versión del entorno de ejecución de la aplicación, ya que no todos los buildpacks son compatibles con todas las versiones de entornos de ejecución de aplicaciones. Por eso necesitamos una mejor solución que facilite la portabilidad y la implementación de nuestras aplicaciones.

Si bien los buildpacks son potentes, la configuración del entorno de ejecución a nivel de sistema operativo se infiere en gran medida en lugar de declararse, como ocurre con Dockerfiles o herramientas de infraestructura como código (IaC) como Ansible, SaltStack o Puppet. Además, los buildpacks no siempre son compatibles con entornos de ejecución modernos (p. ej., Python 3.11 o Node.js 18.17 LTS, etc.), algo que, como sabemos, los contenedores hacen muy bien. Dicho esto, los buildpacks allanaron el camino para que los contenedores se convirtieran en la forma más portátil de implementación de aplicaciones. Afortunadamente, también podemos usar buildpacks para crear imágenes de contenedores mediante una herramienta de código abierto llamada pack (<https://buildpacks.io/>). El uso de buildpacks queda fuera del alcance de este libro, pero es importante tener en cuenta que el reto de la portabilidad existe desde hace mucho tiempo.

Los paquetes de compilación y los contenedores marcaron el comienzo de una nueva era en el desarrollo de software que permitió que el campo de DevOps floreciera. Con estas herramientas y servicios de alojamiento en la nube, los desarrolladores han podido crear e implementar más aplicaciones listas para producción que nunca, a menudo como desarrolladores individuales o en equipos muy pequeños. Además,

Cada vez más empresas han adoptado todo tipo de software (en la nube o de otro tipo), y seguimos observando una creciente necesidad de gestionar e implementar más aplicaciones, tanto públicas como privadas. Aquí es donde entra Kubernetes.

Una vez que hemos creado imágenes de contenedores y acceso a un clúster de Kubernetes, implementar una aplicación es tan simple como escribir un archivo de configuración YAML conocido como manifiesto de Kubernetes.

8.1 Introducción a Kubernetes

Kubernetes está diseñado para funcionar en un clúster de computadoras, ya sean máquinas virtuales en la nube, un rack de servidores locales, un grupo de Raspberry Pis o incluso computadoras de escritorio reutilizadas que estaban inactivas en algún lugar de tu armario. Todas estas opciones pueden ejecutar una versión de Kubernetes que, a su vez, ejecuta todos tus contenedores. Entonces, ¿cuál elegir? ¿Autoservicio o Kubernetes administrado?

8.1.1 ¿Kubernetes autoservicio o administrado?

Si bien Kubernetes es de código abierto y podríamos administrarlo nosotros mismos, prefiero usar Kubernetes administrado basado en la nube por las siguientes razones:

- Direcciones IP estáticas públicas: Kubernetes administrado puede aprovisionar nuevas direcciones IP estáticas públicas en segundos (más allá de sus máquinas virtuales). Si bien esto es posible con K8 de autoservicio, no es tan fácil, rápido ni sencillo.

- Volúmenes y almacenamiento: En menos de 10 líneas de YAML, podemos reservar un volumen para un clúster de K8s administrado. Podemos reservar tantos volúmenes como nuestro proveedor de nube permita en nuestra cuenta. De nuevo, esto es posible en K8s de autoservicio; simplemente no es tan fácil, rápido ni sencillo.
- Basado en la nube: los hosts de hardware, los costos de energía, la refrigeración, la configuración y todo lo que conlleva la nube son inexistentes gracias a Kubernetes administrado en la nube. Los costos son predecibles, el hardware es confiable, las interrupciones son breves y no son nuestra responsabilidad, etc.
- Simplicidad: usar Akamai Connected Cloud para aprovisionar un clúster de Kubernetes es casi tan simple como aprovisionar varias máquinas virtuales.
- Velocidad: Akamai Connected Cloud puede crear sus clústeres K8s en solo unos minutos.
- Plano de control: El plano de control de K8s se encarga de gestionar el clúster y las máquinas virtuales que ejecutan los contenedores. En otras palabras, es el cerebro de la operación. Al utilizar una solución gestionada en la nube, el proveedor de servicios gestiona el plano de control de Kubernetes con un coste adicional mínimo o nulo.
- IaC habilitado: usar una herramienta de Infraestructura como Código (IaC) como Terraform permite activar o desactivar un clúster de Kubernetes administrado en cuestión de minutos. Si bien las herramientas de IaC también pueden ayudar a configurar un clúster autoadministrado, no es tan sencillo.

Teniendo esto en cuenta, es importante comprender que Kubernetes está diseñado para ser lo más portátil posible, lo que facilita considerablemente la migración de un clúster de Kubernetes a otro. Las partes más complejas son los servicios administrados, como volúmenes, balanceadores de carga y direcciones IP públicas relacionadas. Estos servicios suelen ser específicos de la nube y requieren una configuración adicional para una migración exitosa.

La portabilidad reside en lo que abordaremos en gran parte de este capítulo: los manifiestos de Kubernetes. Con esto en mente, aprovisionaremos un clúster de Kubernetes con Akamai Connected Cloud.

8.1.2 Aprovisionamiento de un clúster de Kubernetes

Estos son los pasos para aprovisionar un clúster de Kubernetes usando Nube conectada de Akamai:

1. Inicie sesión en Akamai Connected Cloud.
2. Haga clic en la pestaña Kubernetes.
3. Haga clic en el botón Crear clúster.
4. Cree una cuenta en Akamai Connected Cloud en linode.com/rk8s (los lectores del libro también reciben un crédito promocional).
5. Inicie sesión en Akamai Connected Cloud (anteriormente Linode).
6. Haga clic en el menú desplegable Crear > y seleccione Kubernetes.
7. Utilice la siguiente configuración:
 - Etiqueta del clúster: rk8s-cluster
 - Región: Seattle, WA (o la región más cercana a usted)
 - Versión: 1.26 o la que sea la última versión Plano de
 - control de alta disponibilidad (alta disponibilidad): no (o sí si desea pagar aproximadamente \$60 más por mes)
8. También haga lo siguiente:
 - Agregar grupos de
 - nodos Seleccionar CPU
 - compartida Seleccionar Linode 2 GB (o más grande)
 - Asegúrese de que el recuento sea 3 o mayor (el valor predeterminado es 3).

Antes de hacer clic en Crear clúster, verifique que su pantalla se parezca a la figura 8.1.

Kubernetes / Create Cluster

Cluster Label:

Region:

Kubernetes Version:

I/A Control Plane: Recommended for production workloads, a high-availability I/A control plane is replicated on multiple master nodes to 99.99% uptime. [Learn more about the I/A control plane.](#)

☒ Yes, enable I/A control plane. (\$33.00/month)

☐ No

Add Node Pools: Add groups of Linodes to your cluster. You can have a maximum of 100 Linodes per node pool.

☒ Dedicated CPU ☐ Shared CPU ☐ High Memory ☐ Premium CPU

Shared CPU Linodes are great for medium duty workloads and an equivalent of performance, resources, and price.

Plan	Monthly	Hourly	RAM	CPU	Storage	
Linode 2 GB	\$12	\$0.018	2 GB	1	50 GB	- 0 + Add

Cluster Summary

Linode 2 GB Plan

1 CPU, 2 GB Storage

— 2 +

\$33.00/mo

\$36.00/mo

Figura 8.1 Opciones de clúster de Kubernetes preaprovisionadas

Linode Kubernetes Engine (LKE) aprovisionará automáticamente las instancias de máquinas virtuales según nuestra configuración previa. Estas instancias no necesitan nuestras claves SSH, ya que no iniciaremos sesión directamente en ellas. Las instancias de máquinas virtuales aparecerán en el panel de Linode como cualquier otra instancia. LKE gestionará todas las máquinas virtuales relacionadas y el plano de control de Kubernetes.

Haga clic en "Crear clúster" y, tras unos instantes, veremos la vista detallada de LKE para este clúster (ubicada en <http://cloud.linode.com/kubernetes>). El resultado se muestra en la figura 8.2.

The screenshot shows the Linode Kubernetes Engine dashboard for a cluster named 'rk8s-cluster'. The top section displays cluster details: Version 1.26, 3 CPU Cores, 6 GB RAM, 150 GB Storage, and a monthly cost of \$36.00. It also provides the Kubernetes API Endpoint and a link to the kubeconfig file. The 'Node Pools' section shows a single pool named 'Linode 2 GB' with three nodes in the 'Provisioning' status. Each node has a unique ID and a public IP address. The dashboard includes buttons for 'Upgrade To HA', 'Delete Cluster', 'Add a tag', 'Recycle All Nodes', and 'Add A Node Pool'.

Linode	Status	IP Address	Recycle
lke127692-189305-64efac166e59	Provisioning	172.232.160.218	Recycle
lke127692-189305-64efac169fcb	Provisioning	172.232.160.254	Recycle
lke127692-189305-64efac16ceb2	Provisioning	172.232.160.239	Recycle

Figura 8.2 Clúster de aprovisionamiento de Kubernetes Engine de Linode

Algunos aspectos clave a tener en cuenta:

- El clúster se llama rk8s-cluster (o como lo hayas llamado).
- Se ha creado un archivo kubeconfig (rk8s-cluster-kubeconfig.yaml). Este archivo contiene los permisos necesarios para acceder a su cuenta de servicio raíz en su clúster de Kubernetes. Puede descargar, ver o restablecer los valores de este archivo. Restablecer el archivo kubeconfig es similar a restablecer una contraseña.
- Los grupos de nodos son una característica avanzada de Kubernetes que nos permite configurar nodos adicionales (máquinas virtuales) en nuestro clúster después del aprovisionamiento inicial. Modificar grupos de nodos está fuera del alcance de este libro, pero es importante tener en cuenta que podemos agregar nodos adicionales a nuestro clúster en cualquier momento.
- Como era de esperar, vemos direcciones IP públicas en cada máquina virtual (nodo) del grupo de nodos.
- Se puede acceder a los puntos finales de la API de Kubernetes y a la GUI a través de esta URL (más información sobre esto próximamente).

8.1.3 Conceptos y componentes básicos

Antes de profundizar en el uso de Kubernetes, hay varios conceptos fundamentales que me gustaría que comprendiera a grandes rasgos. Estos conceptos no son exhaustivos para Kubernetes, pero serán el enfoque de este capítulo. Los comprenderá mejor a medida que comience a usar Kubernetes y a aprovisionar los distintos recursos de su clúster. La Figura 8.3 muestra una visualización general de estos conceptos.

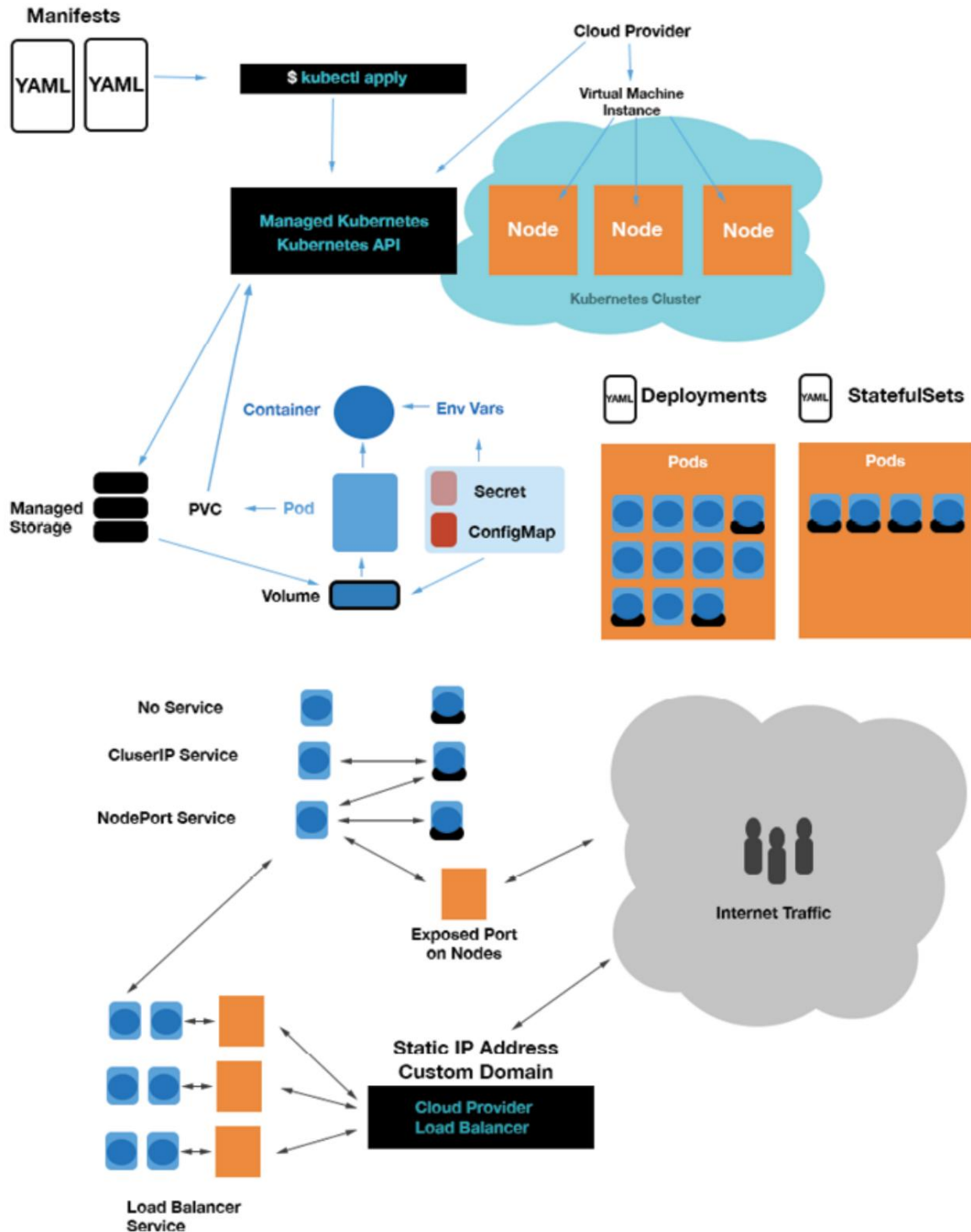


Figura 8.3 Conceptos de Kubernetes

Aquí están los términos:

- **Clúster:** Un grupo de máquinas virtuales que ejecutan Kubernetes y nuestros contenedores. Si bien los clústeres pueden ejecutarse en otros tipos de dispositivos informáticos, en este libro nos centraremos en las máquinas virtuales.
- **Nodo:** una máquina virtual que es miembro de nuestro clúster de Kubernetes.
- **Contenedor:** Un paquete de aplicaciones portátil que se ejecuta de forma aislada mediante un entorno de ejecución de contenedores. Kubernetes utiliza un entorno de ejecución de contenedores ligero llamado containerd. Cabe recordar que Docker Engine es un entorno de ejecución de contenedores que utiliza Docker Compose.
- **Pod:** es la unidad mínima de ejecución de un contenedor, incluyendo la configuración que este necesita. Los pods también pueden definir varios contenedores que comparten recursos (p. ej., CPU, memoria, etc.). Si bien podemos crear pods directamente, es más común usar una implementación para crearlos.
- **Implementación:** Las implementaciones definen las condiciones para ejecutar contenedores mediante pods. En una implementación, podemos especificar la imagen del contenedor, la cantidad de réplicas de pods que queremos ejecutar, etc. Las implementaciones son la forma más común de ejecutar contenedores en Kubernetes.
- **Servicio:** Los Servicios exponen nuestros Pods (y, por lo tanto, nuestros contenedores) a nuestra red de Kubernetes (interna) o a internet (externa). Usaremos los Servicios junto con las Implementaciones para exponer nuestros contenedores a internet con balanceo de carga en la nube.
- **Espacios de nombres:** Los espacios de nombres son una forma de organizar nuestros recursos de Kubernetes. Se puede asignar un espacio de nombres a cualquier recurso de Kubernetes (por ejemplo, una implementación, un servicio, un pod, etc.), lo que nos permite filtrar nuestros recursos por espacio de nombres. Puedes usar espacios de nombres para mantener los proyectos separados, por ejemplo, para el entorno de pruebas de la aplicación.

(por ejemplo, producción, preproducción, desarrollo, investigación y desarrollo, etc.), para diferentes organizaciones o para equipos internos.

- **Secretos y ConfigMaps:** Estos son los mecanismos integrados en Kubernetes para almacenar pares clave-valor (similares a los archivos dotenv que conocemos y apreciamos). Los secretos son simplemente valores codificados en base64, mientras que los ConfigMaps son pares clave-valor. Ambos se utilizan para inyectar variables de entorno en los contenedores. Suelo considerarlos juntos porque los secretos de Kubernetes, aunque prácticos, no son muy seguros. Abordaremos ambos en este capítulo.
- **Conjuntos con estado:** Las implementaciones, al igual que los contenedores, están diseñadas para ser efímeras. Los conjuntos con estado están diseñados para ser más permanentes y se utilizan a menudo para bases de datos (p. ej., PostgreSQL, Redis, MySQL, etc.) y otras aplicaciones con estado.
- **Volúmenes:** cuando necesitamos que los datos persistan más allá del ciclo de vida de un contenedor, podemos adjuntar volúmenes persistentes administrados directamente desde nuestro proveedor de nube.
- **Cuenta de servicio :** Kubernetes no tiene usuarios en el sentido tradicional; en cambio, tiene cuentas de servicio.
Las cuentas de servicio se utilizan para autenticarse con la API de Kubernetes y administrar o acceder a diversos recursos de Kubernetes. Al aprovisionar un nuevo clúster administrado, normalmente se obtiene una cuenta de servicio raíz con permisos ilimitados para el clúster de Kubernetes. Esta cuenta de servicio raíz también incluye claves de acceso confidenciales mediante un archivo kubeconfig generado automáticamente. Si bien la creación de cuentas de servicio queda fuera del alcance de este libro, puede ser importante a medida que aumenta el número de colaboradores en el equipo o proyecto de Kubernetes.
- **Kubeconfig:** con clústeres administrados, la cuenta de servicio raíz se aprovisiona con un archivo kubeconfig (por ejemplo, rk8s-

cluster-kubeconfig.yaml) que contiene las claves de acceso confidenciales a su clúster de Kubernetes. Trate este archivo como cualquier otro secreto (p. ej., compártalo solo con personas de confianza, restablezca las claves si cree que se han visto comprometidas, no lo registre en Git, etc.). El archivo kubeconfig se usa para autenticarse con la API de Kubernetes y acceder a una cuenta de servicio específica. En este capítulo, crearemos un nuevo archivo kubeconfig. kubectl: la interfaz de línea de comandos de Kubernetes se llama kubectl y se pronuncia de diversas maneras: "cube CTL", "cube control" y "cube

- "cuddle". Junto con una cuenta de servicio a través de un archivo kubeconfig, usaremos kubectl para administrar nuestro clúster de Kubernetes, implementar contenedores, administrar servicios, crear secretos, reiniciar implementaciones y más. kubectl es una herramienta poderosa y la usaremos mucho en este capítulo.

Con esto en mente, aprendamos algunas formas de conectarnos e interactuar con nuestro nuevo clúster de Kubernetes.

8.2 Conexión a Kubernetes

Kubernetes ofrece dos formas principales de conectarse a un clúster: la interfaz gráfica de usuario (GUI) del panel de control de Kubernetes y la CLI de Kubernetes (kubectl). Empecemos por la GUI.

8.2.1 La interfaz gráfica de usuario del panel de Kubernetes

El panel de Kubernetes es una manera sencilla de revisar lo que sucede en su clúster de Kubernetes, así como de administrar varios recursos como pods, implementaciones, servicios y más.

Personalmente, tiendo a usar kubectl mucho más que K8s.

Tablero de instrumentos, pero sigue siendo una herramienta útil para una rápida

Experimentación. Para acceder al Dashboard, haga lo siguiente:
siguiente:

1. Inicie sesión en Akamai Connected Cloud.
2. Navegue a Kubernetes.
3. Seleccione su clúster (rk8s-cluster).
4. Descargue el archivo kubeconfig archivo (rk8s-cluster-kubeconfig.yaml).
5. Haga clic en el botón Panel de Kubernetes.

Estos pasos abrirán una nueva pestaña en su navegador y le solicitarán

Para iniciar sesión con su archivo kubeconfig, seleccione

Kubeconfig (no token), elija su archivo de descarga rk8s-cluster-kubeconfig.yaml y haga clic en Iniciar sesión.

El resultado debería parecerse a la figura 8.4.

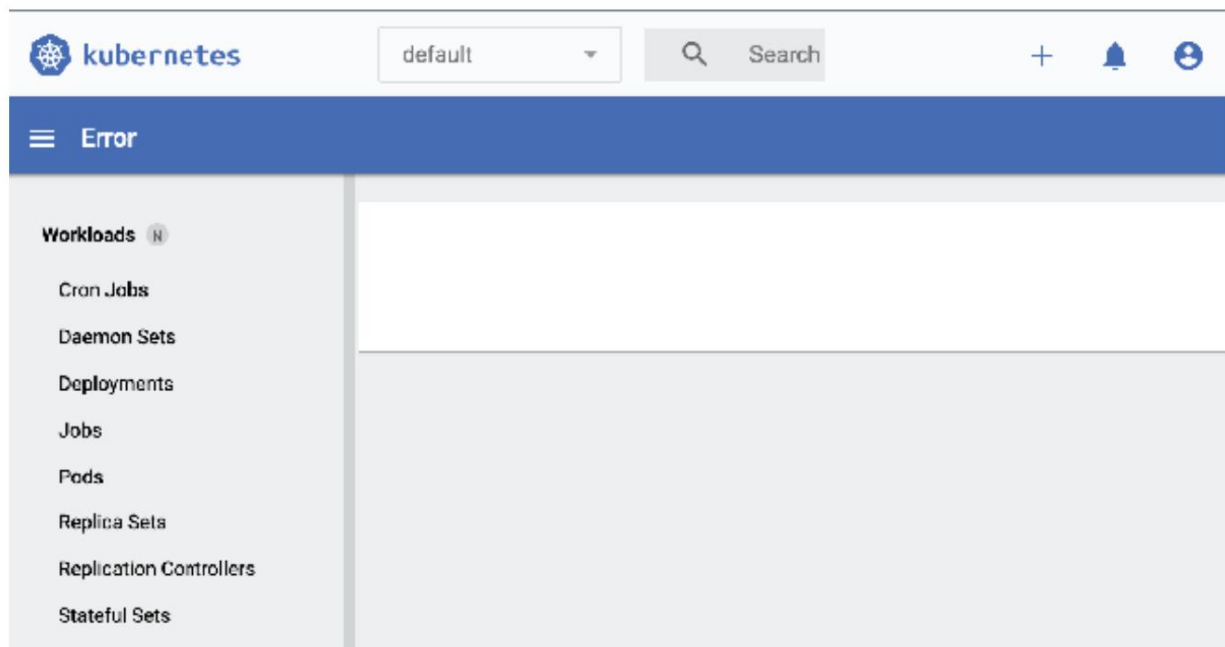


Figura 8.4 Panel de Kubernetes

Este panel está dolorosamente vacío y no tiene ningún asistente.

Formularios para ayudarnos a configurar nuestro clúster o recursos de K8s. Este

Es así por diseño. Para configurar Kubernetes, debemos usar archivos YAML específicos, conocidos como manifiestos de Kubernetes. Si pulsa el signo más (+) en la esquina superior derecha, puede pegar los manifiestos para crear recursos. Pronto aprenderemos más sobre los manifiestos. Incluso sin recursos aprovisionados, podemos ver los nodos disponibles en su clúster, como puede ver en la pestaña Nodos , que se parecerá a la figura 8.5.

Name	Labels	Ready	CPU requests (cores)
ike127692-189305-64efac166e59	beta.kubernetes.io/arch: amd64 beta.kubernetes.io/instance-type: g6-st-stand-1 beta.kubernetes.io/os: linux Show all	True	250.00m (25.00%)
ike127692-189305-64efac169fcb	beta.kubernetes.io/arch: amd64 beta.kubernetes.io/instance-type: g6-st-stand-1 beta.kubernetes.io/os: linux Show all	True	250.00m (25.00%)
ike127692-189305-64efac16ceb2	beta.kubernetes.io/arch: amd64 beta.kubernetes.io/instance-type: g6-st-stand-1 beta.kubernetes.io/os: linux Show all	True	450.00m (45.00%)

Figura 8.5 Nodos del panel de Kubernetes

Aunque algunos usuarios podrían preferir usar el panel para visualizar mejor lo que sucede en su clúster, yo prefiero kubectl porque es una herramienta que puedo usar en mi terminal y automatizar con scripts personalizados y en mis pipelines de CI/CD. Ahora que hemos revisado el panel, instalemos kubectl en nuestra máquina local.

8.2.2 Instalación de kubectl

Antes de instalar kubectl, es importante tener en cuenta que no tendremos una versión local de Kubernetes instalada en nuestra máquina. kubectl intenta automáticamente conectarse a una versión localhost de Kubernetes, como verá en breve.

A diferencia de Docker Desktop, kubectl se instala principalmente en la línea de comandos (p. ej., Terminal o PowerShell) y no como una aplicación GUI. Como muchas herramientas, existen varias maneras de instalar kubectl, según el sistema operativo. Si necesita comprender otras maneras de instalar kubectl en diferentes sistemas operativos, consulte la documentación oficial de instalación de kubectl (<https://kubernetes.io/docs/tasks/tools/>) .

Una vez instalado kubectl , puede verificarlo con el comando `kubectl version --client --output=yaml`. El resultado debería ser similar al de la figura 8.6.

```
% kubectl version --output yaml
clientVersion:
  buildDate: "2023-01-18T15:51:24Z"
  compiler: gc
  gitCommit: 8f94681cd294aa8cfd3407b8191f6c70214973a4
  gitTreeState: clean
  gitVersion: v1.26.1
  goVersion: go1.19.5
  major: "1"
  minor: "26"
  platform: darwin/arm64
kustomizeVersion: v4.5.7

The connection to the server localhost:8080 was refused
```

Figura 8.6 versión de kubectl

La Figura 8.6 muestra que tengo kubectl instalado y la conexión al servidor localhost:8080 falló. Este error es

Esto es de esperar, ya que no tenemos kubectl configurado y probablemente no haya una versión local de Kubernetes ejecutándose (con toda seguridad, no en el puerto 8080). Les muestro este resultado al principio para que sepan qué esperar al instalar kubectl en su equipo.

Para instalar kubectl, hay tres opciones principales que usaremos según su sistema operativo:

- En macOS y usando el administrador de paquetes Homebrew.
- En Windows y utilizando el administrador de paquetes Chocolatey.
- En Linux, como en una máquina virtual o en el subsistema de Windows para Linux (WSL), descargaremos la versión binaria de kubectl y la instalaremos manualmente.

Los métodos que proporciono en las siguientes secciones son los que prefiero porque tienden a ser los más fáciles de poner en funcionamiento.

INSTALAR KUBECTL CON HOMEBREW EN MACOS

Homebrew es un gestor de paquetes de código abierto para macOS y algunas distribuciones de Linux. Facilita enormemente la instalación de una gran cantidad de software de terceros, incluyendo kubectl.

Instalar kubectl con Homebrew es tan simple como lo siguiente:

1. Ejecute `brew update` en la Terminal para actualizar Homebrew. Si el comando `brew` falla, instale Homebrew desde <https://brew.sh/>.

2. Ejecute `brew install kubectl`.
3. Verifique kubectl con la versión de kubectl `--client --output=yaml`.

Si tiene problemas con Homebrew, considere usar la instalación del binario de kubectl , como se muestra en la sección "Instalar el binario de Kubectl en Linux". Si usa Apple Silicon, asegúrese de usar la versión ARM (p. ej., export PLATFORM="arm64"). Si no usa Apple Silicon, use la versión x86 (p. ej., export PLATFORM="amd64").

INSTALAR KUBECTL CON CHOCOLATEY EN WINDOWS

Chocolatey es un gestor de paquetes de código abierto para Windows que facilita la instalación de una gran cantidad de software de terceros. Podemos usar Chocolatey para instalar kubectl y muchas otras herramientas. A continuación, se explica cómo instalar kubectl con Chocolatey:

1. Ejecute choco upgrade chocolatey en PowerShell para actualizar Chocolatey. Si el comando choco falla, instale Chocolatey desde <https://chocolatey.org/install>.
2. Ejecute choco install kubernetes-cli.
3. Verifique kubectl con la versión de kubectl --client --output=yaml.

Si tiene problemas con Chocolatey, considere usar el subsistema de Windows para Linux (WSL) y siga las instrucciones de la siguiente sección.

INSTALAR EL BINARIO KUBECTL EN LINUX

Instalar el binario de kubectl en Linux requiere algunos pasos adicionales que usar gestores de paquetes. Los pasos del listado 8.1 provienen directamente de la documentación oficial de kubectl (<https://kubernetes.io/docs/tasks/tools/install-kubectl-linux/>), pero están modificados para facilitar el cambio de plataforma (p. ej., amd64 o arm64) para diferentes arquitecturas de sistema.

Listado 8.1 Instalación de kubectl con apt en Linux

```
# si es x86, usa amd64 export
PLATFORM="amd64"

# si es ARM, use arm64 y # descomente
la siguiente línea # export PLATFORM="arm64"

# obtener la última versión de lanzamiento export
VER=$(curl -L -s https://dl.k8s.io/release/stable.txt)

# descargar la última versión curl -LO "https://
dl.k8s.io/release/$VER/bin/linux/$PLATFORM/ kubectl"

# verificar la versión curl -LO "https://
dl.k8s.io/$VER/bin/linux/$PLATFORM/kubectl. sha256" echo "$(cat kubectl.sha256) kubectl" | sha256sum --
check

# instalar kubectl sudo install
-o root -g root -m 0755 kubectl /usr/local/bin/k uubectl

# verificar la instalación de la versión
de kubectl --client --output=yaml
```

8.2.3 Configuración de kubectl

Ahora que tenemos un clúster de Kubernetes aprovisionado, un archivo kubeconfig descargado y kubectl instalado, podemos configurar kubectl para que se conecte a nuestro clúster. Para ello, usaremos el archivo kubeconfig que descargamos de Akamai Connected Cloud.

Para configurar kubectl, podemos:

- Utilice ~/.kube/config como ruta para nuestro archivo kubeconfig. Esto significa que nombraríamos el archivo config (sin .yaml) y lo almacenaríamos en el directorio .kube para nuestro usuario.
- Use la variable de entorno KUBECONFIG con la ruta absoluta a nuestro archivo kubeconfig. Esto significa que podemos nombrar el archivo como queramos y almacenarlo donde queramos.

Utilizaremos la variable KUBECONFIG para el desarrollo local para facilitar el uso de múltiples archivos kubeconfig en múltiples proyectos de Kubernetes.

Usar KUBECONFIG es tan sencillo como lo siguiente:

- Terminal (macOS/Linux/WSL)—
KUBECONFIG=/ruta/a/kubeconfig.yaml kubectl obtener nodos
- PowerShell (Windows)—setx "C:\ruta\kubeconfig.yaml" KUBECONFIG
kubectl obtener nodos

Escribir la variable de entorno completa cada vez puede ser tedioso, por lo que podemos configurar la variable de entorno para cada sesión de línea de comandos con

- Terminal (macOS/Linux/WSL)—
KUBECONFIG=/ruta/a/kubeconfig.yaml -exportar
- PowerShell (Windows)—setx "C:\ruta\kubeconfig.yaml" KUBECONFIG

y luego simplemente kubectl obtiene los nodos.

Nuevamente, tendremos que recordar configurar la variable de entorno KUBECONFIG cada vez que abramos una nueva sesión de Terminal o PowerShell. Si bien podríamos configurar la variable de entorno KUBECONFIG globalmente, esto anula el propósito de usar varios archivos kubeconfig para diferentes proyectos.

Si eres como yo, querrás configurar tu entorno de desarrollo local para usar múltiples clústeres de inmediato. Si bien esto requiere más configuración, te prepara para el éxito a medida que aprendes Kubernetes y lo adaptas a tus proyectos.

Una de las razones por las que uso Visual Studio Code (VSCode) como editor de texto es lo sencillo que es configurar la terminal de VSCode para usar variables de entorno predefinidas. Cabe recordar que las terminales de los editores de texto (incluidas las basadas en PowerShell) no son iguales a la Terminal o PowerShell del sistema operativo. Aunque otros editores de texto pueden tener características similares, te mostraré cómo configurar VSCode para usar varios archivos kubeconfig para diferentes proyectos.

En VSCode, puedes actualizar tu archivo `.code-workspace` para incluir la configuración `terminal.integrated.env` para cada plataforma con pares clave-valor específicos. Por ejemplo, puedes usar `terminal.integrated.env.osx` para establecer `PORT` en 8080 o `KUBECONFIG` en `some/path/rk8s-cluster-kubeconfig.yaml`. Con estos valores definidos y tu espacio de trabajo de VSCode abierto, puedes abrir una nueva sesión de terminal de VSCode. Estos valores se inyectarán automáticamente en tu línea de comandos. Puedes verificarlo ejecutando `echo $YOUR_VARIABLE` como `echo $KUBECONFIG`. El Listado 8.2 muestra ejemplos multiplataforma en un archivo `.code-workspace` que establece varias variables de entorno (es decir, `KUBECONFIG`, `PY_PORT`, `JS_PORT`) para cada plataforma.

Listado 8.2 Configuración de KUBECONFIG en VSCode

```
{
  ...
  "ajustes": {
    ...
    "terminal.integrated.env.osx": { "KUBECONFIG":
      "${workspaceFolder}/.kube/kubeconfig.
g.yaml",
      "PY_PORT": "8080",
      "JS_PORT": "3000"
    },
    "terminal.integrated.env.windows": {
      "KUBECONFIG": "${workspaceFolder}\\\.kube\\kubecon
figura.yaml",
      "PY_PORT": "8080",
      "JS_PORT": "3000"
    },
    "terminal.integrated.env.linux": { "KUBECONFIG":
      "${workspaceFolder}/.kube/kubeconfig.
g.yaml",
      "PY_PORT": "8080",
      "JS_PORT": "3000"
    },
  },
}
```

La variable `${workspaceFolder}` es especial para VSCode y se reemplazará por la ruta absoluta al espacio de trabajo. Al usar la línea `${workspaceFolder}/.kube/kubeconfig.yaml`, se obtiene la ruta relativa a la raíz del espacio de trabajo (normalmente la misma carpeta que contiene el archivo `.code-workspace`), lo cual también aplica a la versión de Windows (`terminal.integrated.env.windows`). Si prefiere rutas absolutas, puede hacer lo siguiente:

- En macOS \$ o Linux—
`{workspaceFolder}/.kube/kubeconfig.yaml /ruta/a/mi/` a
`kube/config.yaml`

- En Windows: cambie a "\${workspaceFolder}\\kube\\kubeconfig.yaml"

"C:\\ruta\\a\\mi\\kube\\config.yaml". Observe las barras invertidas dobles.

Independientemente de cómo configure la variable de entorno KUBECONFIG , debería poder verificar que kubectl se conecta con kubectl get nodes y ver los nodos de su clúster. El resultado debería ser similar al de la figura 8.7.

```
% kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
lke127692-189305-64efac166e59	Ready	<none>	21h	v1.26.3
lke127692-189305-64efac169fcb	Ready	<none>	21h	v1.26.3
lke127692-189305-64efac16ceb2	Ready	<none>	21h	v1.26.3

Figura 8.7 kubectl Obtener nodos

Ahora que tenemos kubectl configurado, implementemos nuestro primer contenedor.

8.3 Implementar contenedores en Kubernetes

Con Kubernetes y kubectl configurados, podemos iniciar el proceso de creación de recursos en Kubernetes. Nuestro objetivo al implementar contenedores es tener varios archivos de manifiesto que describan los recursos que intentamos configurar. Estos recursos abarcan desde el propio contenedor hasta otras características que facilitan su ejecución.

Antes de empezar a trabajar con los manifiestos, crearemos una nueva carpeta para todos nuestros manifiestos de Kubernetes. En mi caso, crearé la carpeta ~/dev/roadtok8s-kube , que contendrá los siguientes archivos:

- `.kube/kubeconfig.yaml`: el archivo que describe la configuración de Akamai Connected Cloud. La ruta es `kube/.kube/kubeconfig.yaml`. `roadtok8s-kube.code-workspace`: ejecuta el archivo de mi `~/dev/roadtok8s-` espacio de trabajo de VSCode
- que contiene la variable de entorno `KUBECONFIG` para la ruta a mi archivo `kubeconfig`. `my-first-pod.yaml`: por ahora, este archivo estará vacío, pero lo ampliaremos en breve.
-

Con nuestro directorio `roadtok8s-kube` listo, creemos su primer manifiesto de Pod para ejecutar su primera imagen de contenedor.

8.3.1 Tu primer Pod y manifiesto

Kubernetes ofrece muchos tipos de recursos diferentes. Cada tipo de recurso se declara en el manifiesto con el formato `kind: <resource>` o, en nuestro caso, `kind: Pod`. En esta sección, usaremos el recurso Pod para ejecutar una sola instancia de la imagen del contenedor NGINX. Los pods son la unidad configurable más pequeña de Kubernetes, por lo que son ideales para nuestro primer manifiesto. Estas son algunas características clave que todos los manifiestos de Kubernetes deben incluir:

- `apiVersion`: la versión de la API de Kubernetes que se utilizará. Normalmente, se establece en `v1` para la mayoría de
- los recursos. `kind`: el tipo de recurso que se creará. Hay muchas opciones de recursos, pero en esta sección nos centraremos en Pod y Service y ampliaremos la lista a otras opciones más
- adelante en este capítulo. `metadata.name`: los metadatos de cualquier recurso nos ayudan a describirlo (p. ej., nombres, etiquetas, etc.). El campo de nombre dentro del bloque de metadatos es obligatorio para los recursos.

- **Especificación:** Este es el bloque donde definimos nuestras especificaciones para este recurso en particular. Este capítulo incluirá varios ejemplos de especificaciones para diferentes recursos. Este bloque es la parte más importante de cualquier manifiesto y, con diferencia, la que ofrece más opciones potenciales.

Con esto en mente, vamos a crear tu primer archivo de manifiesto que ejecute un único pod basado en NGINX. Si aún no lo has hecho, crea el archivo `my-first-pod.yaml` y añade el contenido de la siguiente lista, que contiene un manifiesto mínimo para un pod que ejecuta la imagen pública del contenedor NGINX administrada por Docker.

Listado 8.3 Manifiesto YAML de K8s para un pod NGINX

```
apiVersion: v1 kind:
Pod metadata:
name: first-pod-
  nginx spec: containers: - name:
nginx
  image: nginx
```

Ahora que tenemos este archivo, podemos hacer algunas cosas con él:

- `kubectl apply -f my-first-pod.yaml`: Este comando creará el pod en nuestro clúster de Kubernetes. Si el pod ya existe, lo actualizará con cualquier cambio en el manifiesto.
- `kubectl delete -f my-first-pod.yaml`: este comando eliminará el Pod de nuestro clúster de Kubernetes si existe.

Si está utilizando el panel de Kubernetes, puede usar este archivo para crear un nuevo pod directamente en el panel.

Hacerlo queda fuera del alcance de este libro, pero te animo a que lo explores si estás interesado.

Después de ejecutar el comando `kubectl apply`, podemos verificar que se creó el recurso ejecutando `kubectl get pods`, lo que debería generar como se muestra en la figura 8.8.

```
% kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
first-pod-nginx     1/1     Running   0           4s
```

Figura 8.8 `kubectl get pods`

El estado de cualquier Pod, conocido como fase de un Pod, puede ser uno de los siguientes:

- **Pendiente:** El pod se está creando y aún no se está ejecutando. Esto suele significar que el contenedor se está extrayendo de un registro de contenedores o que aún se está iniciando.
- **En ejecución:** el Pod se está ejecutando y, por lo tanto, el o los contenedores del Pod también se están ejecutando.
- **Correcto:** para los pods que no se ejecutan de manera continua, este estado indica que el pod se completó correctamente.
- **Fallido:** no es sorprendente que este estado indique que el Pod ha fallado.
- **Desconocido:** por alguna razón, no se pudo obtener el estado del Pod.

La imagen del contenedor NGINX debería ejecutarse correctamente después de un par de minutos. Si no se ejecuta, puede investigar el pod usando `kubectl describe pod <pod-name>` o `kubectl logs <pod-name>`. La imagen del contenedor NGINX es muy madura y, por lo tanto, es poco probable que se produzcan errores debido al propio contenedor (esto puede o no ocurrir en los contenedores que creamos para nuestros proyectos de Python y Node.js).

Ahora que el contenedor se está ejecutando gracias al recurso Pod, ¿cómo visitamos realmente el NGINX con acceso a Internet?

¿Página predeterminada de este Pod? Ingresa tu primer servicio.
recurso.

8.3.2 Su primer Servicio

Al igual que los contenedores no publicados en máquinas virtuales, los pods no pueden comunicarse con el exterior sin una configuración adicional. La comunicación entre pods también es imposible sin configuración adicional.

Antes de abordar la comunicación interna del clúster, hagamos algo más interesante: exponer nuestro Pod a internet mediante el recurso Servicio de Kubernetes. Ver la página "¡Bienvenido a nginx!" es una gran ventaja en un clúster de Kubernetes recién creado. Para ello, es necesario crear un nuevo manifiesto con el recurso Servicio (tipo: Servicio) para exponer nuestro Pod NGINX a internet.

Antes de crear el recurso de Servicio, revisemos algunos de los tipos fundamentales de Servicios de Kubernetes:

- **NodePort (tráfico externo):** Nuestro clúster cuenta con tres nodos (máquinas virtuales) y cada nodo tiene una dirección IP pública estática proporcionada por el ISP y Akamai Connected Cloud. El recurso NodePort Service le permite exponer un pod a un valor de puerto específico, como 30007, en cada nodo disponible de su clúster K8s.
- **Balanceador de carga (tráfico externo):** Este es uno de mis tipos de servicio favoritos porque aprovisiona automáticamente un balanceador de carga en la nube con una dirección IP pública estática para un recurso de servicio K8s. En el caso de Akamai Connected Cloud, Linode Kubernetes Engine (es decir, el servicio de Kubernetes administrado que utilizamos) nos proporcionará una dirección IP pública estática.

a través del servicio de equilibrio de carga basado en la nube llamado Linode NodeBalancers.

- ClusterIP (tráfico interno exclusivo del clúster): Este tipo de servicio permite que otros contenedores de nuestro clúster de Kubernetes se comuniquen con nuestros pods (o implementaciones), lo que facilita la comunicación entre contenedores dentro del clúster. En concreto, las ClusterIP se utilizan para la comunicación entre pods o entre implementaciones dentro del clúster.

Los ClusterIP son ideales para pods o implementaciones que no necesitan estar expuestos a Internet, como bases de datos, API internas y otros.

Comenzaremos con NodePort para nuestro Pod porque es el más fácil de probar rápidamente sin agregar nada más a nuestro clúster o a nuestro proveedor de nube.

Antes de crear un recurso de servicio, debe actualizar el manifiesto de su pod para que un servicio pueda encontrarlo y conectarse a él fácilmente. Para ello, añada una etiqueta al manifiesto de su pod (your-first-pod.yaml) dentro del bloque de metadatos, como en el listado 8.4.

Listado 8.4 Manifiesto YAML de K8 para un pod NGINX con una etiqueta

```
apiVersion: v1 tipo:
Pod
metadatos:
  nombre: first-pod-nginx etiqueta:
    aplicación: my-nginx
...
```

Si bien normalmente agregamos etiquetas de metadatos cuando creamos por primera vez recursos como Pods, quería agregar énfasis adicional a la importancia ahora: si su Pod o Implementación no tiene una etiqueta, el recurso de servicio no funcionará correctamente.

Con esto en mente, actualice su pod ejecutando `kubectl apply` en el manifiesto actualizado con `kubectl apply -f my-first-pod.yaml`. Ahora ejecute `kubectl get pods` y su pod debería seguir ejecutándose. Ejecute `kubectl get pods my-nginx -o yaml` para agregar aún más detalles sobre su pod (incluida la verificación de que su etiqueta esté presente en la salida).

Dentro de cualquier manifiesto de Kubernetes, independientemente de su tipo (Pod, Servicio, Implementación, etc.), podemos aplicar una etiqueta de metadatos al recurso. Las etiquetas de metadatos son pares clave-valor que se pueden usar para filtrar cualquier recurso de Kubernetes. En el caso del recurso Pod actualizado y la etiqueta `app: my-nginx`, puede ver el filtrado en acción con el comando `kubectl get pods -l app=my-nginx`. Al ejecutar este comando, se obtendrá una lista de Pods que coinciden con este selector o no se obtendrá ningún Pod.

El selector, o filtro, para restringir la lista de resultados de cualquier recurso de Kubernetes es el indicador `-l` o `--selector`, como en `kubectl get <resource> -l <key>=<value>` o `kubectl get <resource> --selector <key>=<value>`.

A continuación, se muestran algunos ejemplos del filtro selector en acción:

- `kubectl get pods -l mi-etiqueta=mi-valor`
- `kubectl obtener pods -l app=mi-nginx`
- `kubectl obtener pods --l app=nginx`
- `kubectl obtener servicios -l app=mi-nginx`
- `kubectl obtener implementaciones -l sugar=spice`
- `kubectl obtener implementaciones -l maverick=pilot`

Como indican estos ejemplos, la etiqueta de metadatos puede ser cualquier cosa; esto aplica tanto a la clave como a su valor relacionado. En otras palabras, el valor de la clave de la etiqueta de metadatos de un recurso (por ejemplo, `app`: en nuestro manifiesto de pod) no tiene un significado especial para Kubernetes; es simplemente un nombre convencional. Mientras que

Algunos metadatos pueden tener un significado especial, pero este no. Simplemente recuerda que el par clave-valor que uses (p. ej., amigo: holly, hola: mundo, cacahuete: mantequilla) se conocerá como el valor del selector del recurso .

Ahora que hemos visto los selectores en acción basados en pares clave-valor de metadatos, definamos algunas declaraciones de clave que necesitamos para nuestros Servicios dentro del bloque spec :

- `type: <service-type>`—Seleccione el tipo de servicio de Kubernetes que desea usar: `NodePort`, `LoadBalancer` o `ClusterIP`. `selector: <labeled-metadata-key-value-pair>`—El selector es la parte más importante de nuestro servicio. El servicio equilibrará automáticamente la carga de las solicitudes según los pares clave-valor coincidentes definidos aquí. Si el par clave-valor utilizado no produce resultados filtrados (p. ej., `kubectl get pods -l incorrect:data`), el servicio es inútil. `ports`—Si recuerda nuestro argumento de publicación de Docker (`-p` o `--publish`), especificamos el puerto que
- queríamos exponer y el puerto al que queríamos mapear (p. ej., `-p <expose>:<mapping>`). En el caso de `kind=NodePort`, define tres valores de puerto: `port`, `targetPort` y `nodePort`. `port` es para que la comunicación interna basada en clúster pueda acceder a este servicio. `targetPort` es el puerto del contenedor que se ejecuta según lo definido por el pod o la implementación (sí, igual que el puerto mapeado en Docker). `nodePort` es el puerto que queremos exponer en cada nodo de nuestro clúster (por ejemplo, `http://<node-static-public-ip>:<nodePort>`).

Ahora hagamos una comprobación rápida antes de definir nuestro Servicio:

- El puerto que usaremos en nuestros nodos es 30007 (por ejemplo, `nodePort: 30007`).
- El puerto que usaremos para la comunicación interna del clúster es el 5050 (p. ej., `puerto: 5050`). Usaremos la comunicación interna del clúster más adelante en este capítulo.

- El puerto que espera el contenedor es el puerto que usaremos para nuestro targetPort (por ejemplo, targetPort: 80).
- Nuestro Pod tiene una etiqueta con app: my-nginx, por lo que nuestro Servicio puede usar el selector app: my-nginx para encontrar nuestro Pod.

Con esto en mente, cree un archivo llamado my-first-service.yaml en su directorio roadtok8s-kube con el contenido de la siguiente lista.

Listado 8.5 Manifiesto YAML de K8s para un servicio NGINX

```
apiVersion: v1
tipo: Servicio
metadatos:
  nombre: first-service-nginx
especificación:
tipo:
  NodePort
puertos:
  -
    puerto: 5050
    targetPort: 80
    nodePort: 30007
selector:
  aplicación: my-nginx
```

Aprovisione este servicio con `kubectl apply -f my-first-service.yaml`.

Una vez hecho esto, puede verificar que el servicio esté disponible y listado con `kubectl get service` o la versión abreviada `kubectl get svc`. El resultado debería ser similar al de la figura 8.9.

```
% kubectl get service
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
first-service-nginx NodePort    10.128.113.185 <none>         5050:30007/TCP   13h
kubernetes          ClusterIP   10.128.0.1     <none>         443/TCP          43h

% kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
first-service-nginx NodePort    10.128.113.185 <none>         5050:30007/TCP   13h
kubernetes          ClusterIP   10.128.0.1     <none>         443/TCP          43h
```

Figura 8.9 Servicio de obtención de kubectl

Los servicios enumerados deben coincidir con nuestro first-service-nginx y el servicio predeterminado de Kubernetes. El servicio predeterminado de Kubernetes es la forma en que nuestro clúster se comunica con la API administrada de Kubernetes; esta API, conocida como plano de control, es la que gestiona los recursos. El plano de control es el que programa y ejecuta los recursos en el clúster. Con Kubernetes administrado, el plano de control lo administra el proveedor de servicios, pero sigue siendo esencialmente el nexo que lo mantiene todo unido. La herramienta kubectl se comunica directamente con la API de Kubernetes mediante este servicio. Para nuestro servicio, es importante tener en cuenta lo siguiente:

- Tipo: Verifique que vea NodePort para nuestro servicio.
- IP del clúster: este es el valor de la dirección IP para la comunicación interna de Pod (o clúster de implementación a implementación) a Pod.
- IP externa: No se muestra ninguna porque usamos NodePort y no LoadBalancer. Si fuera NodeBalancer, se nos asignaría una dirección IP pública estática.
- Puertos: 5050:30007. Esto significa que la comunicación interna del clúster puede acceder a este servicio en el puerto 5050, y el tráfico externo puede acceder a este servicio en el puerto 30007 para cada nodo.

Ahora, verifiquemos que NodePort funciona correctamente obteniendo cada dirección IP estática pública de cada nodo. Para ello, poder

- Inicie sesión en Akamai Connected Cloud Console y revíselo.
- Ejecute el comando `kubectl get nodes -o wide` (que enumerará las direcciones IP internas y externas).

Una vez que encuentre la dirección IP pública estática de un nodo, visite `http://<your-node-public-ip>:30007` y vea NGINX

Hola mundo en todo su esplendor. ¡Felicidades! Acabas de implementar tu primer contenedor conectado a internet en Kubernetes.

Trabajo terminado, ¿verdad? Todavía no. Ahora mismo tienes un contenedor en funcionamiento gracias a un pod; necesitamos hacer lo que Kubernetes creó para: escalar nuestros contenedores.

8.3.3 De los pods a las implementaciones

El recurso de Implementación es la forma más común de implementar contenedores en Kubernetes, ya que nos permite escalar nuestros Pods y, por lo tanto, nuestros contenedores. Las implementaciones nos permiten tener varias instancias de la misma aplicación de contenedor ejecutándose en nuestro clúster (sí, en cada nodo y máquina virtual). Una vez que escalamos nuestras implementaciones, nuestro recurso de Servicio equilibrará automáticamente el tráfico a cada Pod disponible y listo para el tráfico.

Las implementaciones son básicamente grupos de pods y, dado que son grupos de pods, un manifiesto de implementación es básicamente un Manifiesto del pod con algunas declaraciones adicionales en el bloque spec :

- Réplicas: Este es el número de pods que queremos ejecutar con esta implementación. Si no se especifica, el valor predeterminado es 1.
- Selector: Con las implementaciones, predefinimos el selector de etiquetas para que coincida con nuestros pods. Este selector también se usará en la plantilla de pod (véase la siguiente explicación). Plantilla: Este es
- exactamente el manifiesto del pod sin las declaraciones de tipo ni de APIVersion . Puede copiar y pegar los metadatos y los valores de especificación . Dentro de los valores de los metadatos , asegúrese de que el par clave-valor de la etiqueta coincida con el selector en el bloque selector: de la implementación (aún no con el servicio).

- `apiVersion`: Aunque no se encuentra en el bloque `spec`:, usamos `apps/v1` como `apiVersion` para las implementaciones. Esta es la única `apiVersion` que usaremos para las implementaciones en este libro.

Con esto en mente, creemos un nuevo archivo llamado `my-first-deployment.yaml` en su directorio `roadtok8s-kube` con el contenido de la siguiente lista.

Listado 8.6 Manifiesto YAML de K8s para una implementación de NGINX

```
apiVersion: apps/v1 kind:
Metadatos de
implementación:
  nombre: nginx-deployment spec:
réplicas:
  15 selector:

    matchLabels:
      aplicación: my-nginx
plantilla:
  metadatos:
    etiquetas:
      aplicación: my-nginx

  especificación:
    contenedores: -
      nombre: nginx imagen: nginx:latest
```

Con este manifiesto, aplique los cambios con `kubectl apply -f my-first-deployment.yaml`. Una vez hecho esto, puede verificar la implementación con `kubectl get deployments` o la versión abreviada `kubectl get deploy`. El resultado debería ser similar al de la figura 8.10.

```
% kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
rk8s-deployment    15/15   15           15          14s
% kubectl get deploy
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
rk8s-deployment    15/15   15           15          16s
```

Figura 8.10 Implementación de kubectl get

En este ejemplo, incluí 15 réplicas para ilustrar lo sencillo que es ampliar el número de pods en ejecución gracias al recurso de implementación y, una vez más, destacar el funcionamiento del filtro selector. Si ejecuta `kubectl get pods --selector app=my-nginx`, debería ver al menos 15 pods en ejecución (y posiblemente 16 con nuestro manifiesto de recursos de pod anterior).

El recuento de réplicas es el estado deseado para los recursos de implementación de Kubernetes. Esto significa que Kubernetes implementará Pods continuamente hasta que 15 de ellos estén en funcionamiento y en buen estado, según lo definido en nuestro manifiesto. Si algún Pod falla, Kubernetes implementará automáticamente nuevos Pods hasta que se alcance el número de réplicas. El proceso de implementación es por niveles de forma predeterminada, por lo que si se actualiza un manifiesto de implementación, solo se actualizarán unos pocos Pods a la vez. Esta es una excelente función, ya que nos permite implementar nuevas versiones de nuestros contenedores sin tiempo de inactividad y ayuda a garantizar que los contenedores con fallos tengan un impacto mínimo o nulo en nuestro Servicio.

El recurso Pod no tiene un recuento de réplicas, por lo que su comportamiento es diferente al del recurso Deployment. Esto significa que si un manifiesto de Pod (p. ej., `kind: Pod`) tiene un contenedor con errores, usted, como desarrollador, deberá intervenir manualmente.

Con el manifiesto de implementación aprovisionado, notarás que no necesitamos cambiar nuestro manifiesto de servicio en absoluto. ¿Recuerdas por qué? Porque usamos...

Se utiliza el mismo selector para la implementación que para el pod. Esto significa que nuestro servicio equilibrará automáticamente el tráfico entre los 15 pods del recurso de implementación (p. ej., my-first-deployment.yaml) y el único pod del recurso de pod (my-first-pod.yaml), lo que suma un total de 16 pods disponibles para nuestro recurso de servicio. Para limpiar, elimine los tres recursos con los siguientes comandos:

```
kubectl delete -f mi-primer-pod.yaml kubectl  
delete -f mi-primer-despliegue.yaml kubectl delete -f mi-  
primer-servicio.yaml
```

Una vez agotados todos estos recursos, veamos cómo podemos realizar algunas modificaciones en nuestro contenedor NGINX utilizando un nuevo recurso llamado ConfigMaps.

8.3.4 Personalizar NGINX con ConfigMaps

Los ConfigMaps son recursos que podemos asociar a una implementación (o directamente a un recurso de pod) como volumen o variables de entorno. Son una excelente manera de modificar la configuración predeterminada de un contenedor sin tener que reconstruir su imagen.

En esta sección, usaremos un recurso ConfigMap para modificar la página web predeterminada de NGINX. Para ello, cree una carpeta llamada custom-nginx en el directorio roadtok8s-kube . Dentro de esta carpeta, cree el archivo 1-configmap.yaml con el contenido de la siguiente lista. El uso de 1- será importante, como verá en breve.

Listado 8.7 Manifiesto YAML de K8 para un ConfigMap de NGINX

```
apiVersion: v1 tipo:
ConfigMap metadatos:
nombre: nginx-

  configmap datos: index.html: | <html>

    <head>
      <title>Hola mundo</title> </head> <body>
    <h1>Hola
    mundo</
      h1> </body> </html>
```

El recurso ConfigMap usa `data:` en lugar de `spec:` para definir los pares clave-valor que elegimos. En otras palabras, los bloques `data:` son increíblemente flexibles. Lo que se ve aquí es que usamos la clave `index.html` con un valor de varias líneas, indicado por la barra vertical `|`, que tiene HTML muy rudimentario.

Para anular el valor predeterminado de `index.html` que viene con la imagen del contenedor NGINX, se usará el valor de la clave ConfigMap de `index.html:` en `data:` en nuestra implementación. Para ello, montaremos ConfigMap como un volumen en nuestro manifiesto de implementación.

Al montar un volumen en una implementación, se trata de un volumen de solo lectura, ya que el contenedor no puede modificar el ConfigMap. Si necesitamos montar volúmenes de lectura y escritura, las implementaciones o pods pueden usar volúmenes persistentes, de los que hablaremos más adelante en este capítulo.

Declarar volúmenes en un bloque de plantilla de una implementación (o en un manifiesto de pod) es muy parecido a usar volúmenes con Docker

Redactar: Debemos declarar el volumen y montarlo. Podemos tener varios volúmenes y varios montajes para cada volumen. En nuestro caso, tendremos un volumen y un montaje.

Cree un archivo llamado 2-deployment.yaml dentro de la carpeta custom-nginx del directorio roadtok8s-kube con el contenido. Esta implementación es prácticamente idéntica a la anterior, con dos nuevas adiciones para declarar el volumen (el bloque volumes :) y montarlo (el bloque volumeMounts dentro de la definición de contenedores). La siguiente lista muestra esto en acción.

Listado 8.8 Manifiesto YAML de K8s para una implementación de NGINX con un ConfigMap

```

apiVersion: apps/v1 kind:
Implementación metadatos:
nombre:
  custom-nginx-deploy spec: réplicas: 15
selector:
  matchLabels:
  aplicación:
    custom-nginx
    plantilla: metadatos: etiquetas:
  aplicación:
    custom-nginx

  especificación:
    contenedores: -
      nombre: nginx imagen:
      nginx:latest
      volumeMounts: - nombre: my-configmap-
        volume mountPath: /usr/share/nginx/html readOnly:
        true volúmenes: -
      nombre: my-
      configmap-volume configMap: nombre: nginx-
      configmap

```

Ahora que tenemos una implementación, creemos un nuevo servicio para ella. Cree un archivo llamado 3-service.yaml dentro de la carpeta custom-nginx del directorio roadtok8s-kube con el contenido del listado 8.9.

Listado 8.9 Manifiesto YAML de K8 para un servicio NGINX con un ConfigMap

```

apiVersion: v1 kind:
Service metadata:
name: custom-
  nginx-svc spec: type: NodePort ports:
- port:
  8080 targetPort: 80
  nodePort:
  30008 selector: app:
    custom-nginx

```

Observe que el puerto y el puerto del nodo han cambiado con respecto a nuestro servicio anterior; esto es solo para garantizar que no haya conflictos debido a estos valores. Ahora nuestra carpeta custom-nginx contiene los siguientes archivos:

- 1-configmap.yaml
- 2-implementación.yaml
- 3-servicio.yaml

El orden de visualización en nuestro ordenador debe coincidir con el anterior. En este caso, el orden de aprovisionamiento es necesario porque la implementación depende del ConfigMap y el servicio depende de la implementación. Si el orden se realiza incorrectamente, es posible que algunos o todos estos servicios no se aprovisionen correctamente o que sea necesario aprovisionarlos varias veces para que funcionen correctamente. Queremos que funcionen a la primera; por lo tanto, el orden es importante. Disponemos de las siguientes opciones para aprovisionar estos archivos:

- `kubectl aplicar -f personalizado-nginx/1-configmap.yaml`

- `kubectl aplicar -f custom-nginx/2-deployment.yaml`
- `kubectl aplicar -f personalizado-nginx/3-service.yaml`

O simplemente:

- `kubectl aplicar -f nginx personalizado/`

Este último aprovisionará todos los archivos de la carpeta en el orden en que se muestran en tu ordenador. Por eso, les puse los prefijos 1-, 2- y 3- para asegurar que el orden sea correcto. ¡Qué función tan útil, ¿verdad?!

Ahora, si abre el nodePort (por ejemplo, 30008) en cualquiera de las direcciones IP públicas de su nodo, debería ver el resultado que se muestra en la figura 8.11.

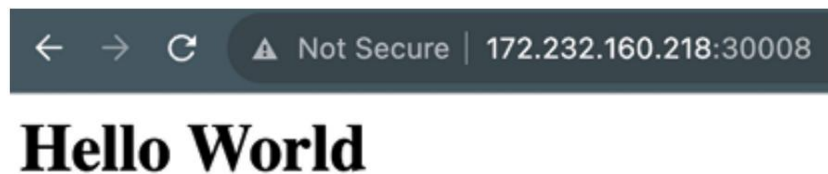


Figura 8.11 Salida del contenedor NGINX basado en ConfigMap

¡Éxito! Debería ver la nueva salida HTML en lugar de la página de bienvenida estándar de NGINX. Si tuvo problemas, verifique lo siguiente:

- El valor de nodePort que utilizó no está en uso en otro servicio.
- El nombre de ConfigMap coincide con el nombre del bloque configMap referenciado en el bloque de volumen de la implementación.
- El nombre del volumen de implementación coincide con el nombre del bloque volumeMounts referenciado en el bloque de contenedores de implementación.

- Tiene el par clave-valor `index.html` correcto en el recurso `ConfigMap`.
- No se ejecutan demasiadas réplicas en ninguna implementación. Elimine las demás implementaciones y limite el número a unas 15 para este ejemplo.

Podemos usar `ConfigMaps` para más que solo montar volúmenes.

También podemos usar `ConfigMaps` para configurar variables de entorno, como veremos en breve. Recuerde que los `ConfigMaps` no se abstraen de ninguna manera, por lo que debe tratarlos como cualquier código: omitir cualquier dato sensible.

Antes de continuar, limpiemos nuestra implementación de NGINX basada en `ConfigMap` con el comando `kubectl delete -f custom-nginx/`. De nuevo, podemos usar un acceso directo para llamar a todos los manifiestos de una carpeta, en este caso para eliminar la implementación.

Ahora que hemos visto el caso de uso montado en volumen para `ConfigMaps`, veamos cómo podemos usar `ConfigMaps` para establecer variables de entorno.

8.3.5 Variables de entorno con `ConfigMaps` y secretos

Las variables de entorno son muy comunes al implementar contenedores en cualquier entorno. Kubernetes no es la excepción. Podemos definir las de cualquiera de las siguientes maneras:

- Codificado directamente en el manifiesto de recursos (por ejemplo, en un manifiesto de `pod` o implementación)
- Usando un `ConfigMap`
- Usando un `Secreto`

La primera opción es la menos flexible, la menos reutilizable y probablemente la menos segura. Los ConfigMaps y Secrets se pueden asociar a un número ilimitado de recursos, lo que los convierte, con diferencia, en los más reutilizables. Teniendo esto en cuenta, cambiar la configuración puede ser tan sencillo como modificar el ConfigMap o el Secret en lugar de modificar cada recurso individualmente.

Además, algunas variables de entorno pueden exponerse públicamente (incluso si solo se trata de compañeros de equipo), mientras que otras no. El uso de un ConfigMap o un Secret ayuda a minimizar esta visibilidad. La siguiente lista muestra un ejemplo de configuración de variables de entorno directamente en un manifiesto de implementación.

Listado 8.10 Variables de entorno codificadas de forma rígida en un manifiesto de implementación

```
tipo: Implementación
```

```
...
```

```
especificación:
```

```
...
```

```
plantilla:
```

```
...
```

```
especificación:
```

```
contenedores: -
```

```
  nombre: nginx imagen: nginx:latest
```

```
  entorno:
```

```
    - nombre: MY_ENV_VAR
```

```
      valor: "Esto es simple" - nombre:
```

```
        PY_PORT valor: "8080"
```

```
    - nombre: JS_PORT
```

```
      valor: "3000"
```

Como podemos ver, este tipo de configuración de variables de entorno es fácil, pero no son exactamente reutilizables. Es más, si accidentalmente envías este manifiesto a un repositorio público con una variable de entorno sensible, podrías encontrarte en un mundo de...

problema. Muevamos las variables de entorno PY_PORT y JS_PORT a un recurso ConfigMap, como se ve en la siguiente lista.

Listado 8.11 ConfigMap para variables de entorno

```
apiVersion: v1 tipo:
ConfigMap metadatos:
nombre:
  proyecto-puertos-cm datos:

  PY_PORT: "8080"
  JS_PORT: "3000"
  ESPECIAL: "entrega"
```

Ahora que tenemos estas variables ConfigMap, podemos actualizar nuestro bloque de entorno de implementación como en la siguiente lista.

Listado 8.12 Variables de entorno basadas en ConfigMap en un manifiesto de implementación

```
tipo: Implementación
```

```
...
```

```
especulación:
```

```
...
```

```
plantilla:
```

```
...
```

```
especificación:
```

```
contenedores: -
```

```
  nombre: nginx imagen: nginx:latest
```

```
  entorno:
```

```
    - nombre: MY_ENV_VAR
```

```
      valor: "Esto es simple" - nombre:
```

```
  PY_PORT
```

```
  valorDe:
```

```
    configMapKeyRef: nombre:
```

```
      proyecto-puertos-cm clave: PY_PORT
```

```
    - nombre: JS_PORT
```

```
  valueFrom:
```

```
    configMapKeyRef:
```

```
      nombre: proyecto-puertos-
```

```
      cm clave: JS_PORT envFrom: -
```

```
      configMapRef:
```

```
  nombre:
```

```
  proyecto-puertos-cm
```

Lo que vemos aquí es que hay dos formas en las que podemos usarlo
ConfigMaps para establecer variables de entorno:

- `valueFrom.configMapKeyRef.name` y
`valueFrom.configMapKeyRef.key`: estos capturarán pares clave-valor
específicos dentro de un recurso ConfigMap especificado.
- `envFrom.configMapRef.name`: esto cargará todos los pares clave-valor
declarados en el bloque `data`: en el recurso referenciado.

ConfigMap (por ejemplo, PY_PORT, JS_PORT y SPECIAL de projects-port-cm en el listado 8.11).

Si bien los ConfigMaps no son seguros, su recurso primo, los Secrets, sí lo son. Los Secrets están diseñados para almacenar datos confidenciales como contraseñas y claves API, pero son simplemente valores codificados en Base64. En otras palabras, son solo ligeramente más seguros que los ConfigMaps. Dicho esto, veamos cómo podemos usar los Secrets para establecer variables de entorno. Para crear nuestro recurso Secret, cree un archivo llamado dont-commit-this-secrets.yaml en el directorio roadtok8s-kube con el contenido que se muestra a continuación.

Listado 8.13 Secreto para variables de entorno

```
apiVersion: v1 tipo:
Metadatos secretos:

  nombre: algo secreto
tipo: Cadena
opacaDatos:
  VIDEO SECRETO DE YT: "dQw4w9WgXcQ"
```

Al definir secretos, podemos usar dos configuraciones diferentes: data o stringData. El bloque de datos requiere el envío de valores codificados en Base64, mientras que el bloque stringData solo requiere texto sin formato. En este caso, usamos stringData para simplificar las cosas. Ahora, verifiquemos estos secretos con

- `kubectl -f apply dont-commit-this-secrets.yaml`—Crea el secreto
- `kubectl get secrets`—Enumera todos los secretos
- `kubectl get secrets something-secret -o yaml`—Enumera el secreto específico

Después de realizar estos pasos, debería ver el mismo resultado que en la figura 8.12.

```
% kubectl get secrets
NAME                                TYPE                                DATA  AGE
default-token-ssrrg                 kubernetes.io/service-account-token 3      2d3h
somewhat-secret                     Opaque                              1      12s
% kubectl get secrets somewhat-secret -o yaml
apiVersion: v1
data:
  SECRET_YT_VIDEO: ZFF3NHc5V2dYY1E=
kind: Secret
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Secret","metadata":{"annotations":{},"name":
ingData":{"SECRET_YT_VIDEO":"dQw4w9WgXcQ"},"type":"Opaque"}}
  creationTimestamp: "2023-09-02T00:50:09Z"
  name: somewhat-secret
  namespace: default
  resourceVersion: "169650"
  uid: dcce1bed-90db-4d76-863a-f9161e09cf2e
type: Opaque
```

Figura 8.12 Salida de kubectl get secret

Como puede ver, la clave SECRET_YT_VIDEO ahora tiene el valor ZFF3NHc5V2dYY1E=. Esta es una versión codificada en base64 del texto sin formato que enviamos. Haga una búsqueda rápida en línea y encontrará una manera fácil de decodificar estos datos.

Por esta razón, recomiendo usar herramientas de terceros para proteger mejor el valor real de sus secretos. Algunas opciones son HashiCorp Vault (que cuenta con versiones administradas y de código abierto), AWS Secrets Manager (un servicio administrado) y Google Secrets Manager (también un servicio administrado).

La ventaja de los recursos Secret y ConfigMap es que podemos reutilizarlos en cualquier implementación o pod de nuestro clúster. Si bien esto representa otro riesgo potencial para la seguridad, es una excelente manera de eliminar mucha información redundante que nuestros recursos de Kubernetes podrían necesitar.

Ahora que tenemos las bases de algunos recursos básicos de Kubernetes, veamos cómo implementar contenedores con estado como

bases de datos a Kubernetes.

8.4 Volúmenes y contenedores con estado

Al usar Implementaciones de Kubernetes, observará que cada nombre de Pod contiene el nombre de la Implementación junto con valores aleatorios adicionales (por ejemplo, custom-nginx-95d4d5bcd-75zn6 o custom-nginx-95d4d5bcd-4j66r). Estos valores aleatorios se deben a que los Pods mediante Implementaciones están diseñados para ser intercambiables y efímeros. En otras palabras, no se garantiza el orden en que se crean o destruyen los Pods. Aquí tiene una prueba rápida para ver este esquema de nombres de Pods en acción:

1. Aprovechone un recurso de implementación con `kubectl apply -f my-first-deployment.yaml`.
2. Ejecute `kubectl get pods` y debería ver un Pod con un nombre como custom-nginx-95d4d5bcd-75zn6.
3. Ejecute `kubectl delete pod custom-nginx-95d4d5bcd-75zn6` y debería ver un nuevo Pod creado con un nuevo nombre como custom-nginx-95d4d5bcd-4j66r.
4. Repetir. Las implementaciones siempre crearán nuevos pods con nombres nuevos y su objetivo será alcanzar el número de réplicas.

El esquema de nombres de pods del recurso de implementación implica que usar pods específicos por nombre es un poco complicado. Por ejemplo, si quisiéramos usar el shell bash de un pod en ejecución dentro de una implementación, tenemos dos opciones:

- `kubectl exec -it <pod-name> — bin/bash`—Esto, por supuesto, requiere que conozcamos el <pod-name>, lo cual es casi imposible de predecir, por lo que podemos usar `kubectl get pods -l`

app=some-label para delimitar el grupo de pods de cualquier implementación. Este filtro nos permite encontrar un <pod-name> válido. Sin embargo, recuerda que este <pod-name> recién encontrado seguramente cambiará , especialmente si realizamos cambios en la implementación o si el pod necesita actualizarse.

- `kubectl exec -it deployments/<nombre-de-implementación> — bin/bash`— Este comando es el que recomiendo si no necesita diagnosticar problemas con un pod específico (ya que a veces podría necesitarlo). Usar `deployments/<nombre-de-implementación>`, donde <nombre-de-implementación> es el valor de metadata.name para una implementación, buscará un pod dentro del grupo de pods administrados por la implementación.

El formato de `kubectl <comando> implementaciones/<nombre-de-implementación>` funciona muy bien para lo siguiente:

- Registros: `kubectl logs <pod-name>` o `kubectl logs deployments/<deployment-name>`. Utilice este comando para revisar los registros relacionados con un pod o una implementación.
- Describe—`kubectl describe pod <pod-name>` o `kubectl describe deployments/<nombre-de-implementación>`. Use este comando describe para los detalles de un pod o un pod administrado por una implementación. comprender mejor

Este extraño esquema de nombres no solo era una forma disimulada de introducir algunos comandos nuevos, sino que también nos ayudaba a plantearnos la pregunta: ¿Pods tener nombres lógicos? Algo como custom-nginx-01 o custom-nginx-02 sería útil. Si bien podríamos crear un manifiesto de Pod con estos nombres autoincrementales (p. ej., `kind: Pod`), sería un desperdicio y una tontería cuando ya contamos con un recurso de Kubernetes que lo hace: el recurso `StatefulSet`.

¿Los `StatefulSets` son simplemente implementaciones que nombran los pods de forma diferente? Sí y no. Aunque los `StatefulSets` sí nombran los pods en

orden (por ejemplo, custom-nginx-01 o custom-nginx-02), también tienen otro beneficio importante: plantillas de reclamo de volumen.

Antes de aprovisionar nuestro primer StatefulSet, veamos las solicitudes de volumen dentro de una implementación. Las solicitudes de volumen son solicitudes al proveedor de la nube para acceder a un volumen de almacenamiento administrado que se puede conectar a un pod.

8.4.1 Volúmenes e implementaciones

Los volúmenes persistentes, o volúmenes de almacenamiento en bloque, se pueden agregar a un clúster a través de un recurso PersistentVolumeClaim (PVC). Con un clúster de Kubernetes gestionado en la nube, al definir este recurso de manifiesto, se hará ping a su proveedor de nube (en nuestro caso, Akamai Connected Cloud) y se solicitará un nuevo volumen. Este volumen se conectará a un nodo del clúster y estará disponible para su uso.

Crea un nuevo directorio llamado deploy-pvc, en el cual haremos lo siguiente:

- Cree un recurso PersistentVolumeClaim.
- Crear un recurso de implementación.

Si se ejecuta correctamente, el recurso PersistentVolumeClaim nos proporcionará un nuevo volumen que podremos conectar a nuestro recurso de implementación. Este volumen se conectará a un nodo de nuestro clúster y estará disponible para su uso. Estas son las configuraciones principales que debemos definir en nuestro recurso PersistentVolumeClaim:

- Modos de acceso: Kubernetes ofrece una amplia gama de modos de acceso, como ReadWriteOnce, ReadOnlyMany y ReadWriteMany. Aun así, no todos los proveedores de nube admiten todos los modos de acceso. Por ejemplo, Akamai Connected Cloud solo admite ReadWriteOnce, que...

significa que el volumen no se puede compartir entre varios pods.

- `Resources.requests.storage`: Este es el tamaño del volumen que queremos solicitar. Este valor está en gigabytes: 10 Gi corresponden a 10 gigabytes de almacenamiento.
- `storageClassName`: Aunque existen numerosas opciones, usaremos un valor proporcionado por la nube. En nuestro caso, podemos usar `linode-block-storage` para las unidades que deben eliminarse junto con el recurso PVC , o `linode-block-storage-retain` para las unidades que permanecerán en su cuenta incluso si se elimina el recurso PVC .

Cree un archivo llamado `1-pvc.yaml` en el directorio `deploy-pvc` con el contenido de la siguiente lista.

Listado 8.14 Recurso PersistentVolumeClaim

```
apiVersion: v1 tipo:
PersistentVolumeClaim metadatos: nombre:
first-pvc
  especificación:

accessModes:
  - Leer y escribir una vez
recursos:
  solicitudes:
    almacenamiento:
      10Gi storageClassName: linode-block-storage-retain
```

Con este recurso PVC, podemos conectar este volumen a un recurso de implementación. Cree un archivo llamado `2-deployment.yaml` en el directorio `deploy-pvc` con el contenido que se muestra a continuación.

Listado 8.15 Recurso de implementación con una PVC

```

apiVersion: apps/v1 kind:
Implementación metadatos:
nombre: nginx-
  deployment spec: réplicas: 1

...
plantilla:
  ...
  especulación:
    ...
    contenedores: -
      nombre: nginx
      ...
      volumeMounts: -
        mountPath: "/usr/share/nginx/html" nombre: first-pv-storage
      volúmenes: - nombre: first-pv-storage

  persistentVolumeClaim: claimName: first-
    pvc

```

Aplique estos recursos con `kubectl apply -f deploy-pvc/`.

Después de una breve espera, debería suceder lo siguiente:

- Solo se debe ejecutar un único pod de implementación de nginx (por ejemplo, réplicas: 1).
- Se debe aprovisionar un nuevo volumen. Para verificarlo, ejecute `kubectl describe pvc first-pvc` y busque `Volume: <value>`; debería ser similar a `pvc-b50fb2b7b4744cce`. Esta es la etiqueta exacta del volumen en su cuenta de Akamai Connected Cloud.

Ahora que hemos visto nuestro primer volumen generado, analicemos las limitaciones del modo de acceso `ReadWriteOnce`: el límite de réplicas. De forma predeterminada, este modo de acceso en particular significa

El volumen solo se puede conectar a un pod a la vez. Es decir, no se puede compartir entre varios pods. Si escalamos nuestra implementación a dos o más réplicas, los nuevos pods no se aprovisionarán porque el volumen ya está conectado al primero. Esto se debe al modo de acceso `ReadWriteOnce` y no a Kubernetes.

Como ya sabrá, compartir el mismo volumen persistente con varios ordenadores, aplicaciones o pods puede provocar corrupción de datos (es decir, que distintos procesos escriban en el mismo archivo simultáneamente). Además, existen mejores opciones de almacenamiento (p. ej., bases de datos, colas de mensajes, almacenamiento de objetos compatible con S3, etc.) para compartir datos entre varios pods. Creo que por esta razón, muchos proveedores de nube, incluido Akamai Connected Cloud, optan por admitir únicamente el modo de acceso `ReadWriteOnce` al momento de escribir este libro.

Antes de continuar, destruya estos recursos con `kubectl delete -f deploy-pvc/`; esto eliminará tanto el PVC, el volumen en su cuenta de Akamai y la implementación.

Aunque aún necesite varios contenedores ejecutándose con acceso a volúmenes de almacenamiento, puede hacerlo usando otro recurso de Kubernetes: el `StatefulSet`. Veamos cómo podemos asociar un volumen a cada pod en un `StatefulSet`.

8.4.2 Conjuntos con estado

A lo largo de este libro, hemos visto el uso de ejemplos de "Hola Mundo" en nuestras aplicaciones web. Si bien estas aplicaciones han sido útiles para explicar muchos conceptos, carecen de un aspecto práctico clave. Si bien las aplicaciones web pueden ejecutarse independientemente de los almacenes de datos (p. ej., bases de datos, mensajes...)

colas, etc.), como hemos hecho muchas veces en este libro, las aplicaciones web realmente valiosas rara vez lo hacen. Los almacenes de datos son una combinación de una aplicación en ejecución y un volumen de almacenamiento que la aplicación utiliza para leer y escribir datos. Esto significa que la aplicación del almacén de datos no tiene estado y puede ser un contenedor, mientras que los datos sí tienen estado y deben estar en un volumen de almacenamiento.

Los StatefulSets proporcionan una forma sencilla de adjuntar un volumen de almacenamiento a cada réplica de Pod declarada por el manifiesto, lo que garantiza que los Pods tengan estado y sean candidatos ideales para ejecutar almacenes de datos.

Los StatefulSets desbloquean la capacidad de ejecutar pods (contenedores) con estado al proporcionar algunas características clave:

- Plantilla de volumen: cada nuevo Pod en un StatefulSet tendrá un nuevo volumen adjunto.
- Esquema de nombres de Pod : cada nuevo Pod en un StatefulSet tendrá un nombre nuevo que sea predecible y ordenado.
- Orden de pods: Cada nuevo pod en un StatefulSet se creará en orden. Esto significa que custom-nginx-01 se creará antes que custom-nginx-02, y así sucesivamente.
- Eliminación de pods: Cada pod nuevo en un StatefulSet se eliminará en orden inverso. Esto significa que custom-nginx-03 se eliminará antes que custom-nginx-02 , y así sucesivamente.

Estas características hacen que StatefulSets sea una excelente manera de implementar contenedores con estado como bases de datos, colas de mensajes y otras aplicaciones que requieren un volumen persistente.

Tenga en cuenta que implementar varias instancias de un contenedor no las conecta automáticamente entre sí. En otras palabras, implementar una base de datos Postgres con un StatefulSet no crea automáticamente un clúster Postgres. La creación de bases de datos en clúster es un tema para otro libro.

Para crear un StatefulSet, necesitamos definir algunas configuraciones clave:

- **initContainers:**—Cuando montamos un nuevo volumen en un contenedor, viene con un directorio llamado `lost+found`. Este directorio debe eliminarse para que el contenedor de Postgres funcione correctamente. El bloque `initContainers:` permite ejecutar un contenedor antes de iniciar el contenedor principal. En nuestro caso, ejecutaremos una imagen mínima del contenedor que monta nuestro volumen, elimina el directorio `lost+found` con el comando `sh -c "rm -rf lost+found"` y, a continuación, sale. Si bien la carpeta `lost+found` puede usarse para la recuperación de archivos, no tenemos archivos que recuperar en este momento. Este es un patrón común para inicializar volúmenes y es una excelente manera de garantizar que nuestro contenedor de Postgres funcione correctamente sin borrar accidentalmente los datos que realmente necesitamos. **réplicas:** el número de pods que se ejecutarán; el valor predeterminado es 1. **selector:** al igual que con las
- implementaciones, nuestro StatefulSet necesita saber qué pods
- administrar mediante el selector.
- **Plantilla:** La configuración que usamos aquí es idéntica a la de un manifiesto de Pod independiente sin las declaraciones de tipo y `apiVersion`. Se denomina plantilla porque contiene los valores predeterminados del grupo de Pods que administrará el StatefulSet. **VolumeClaimTemplates:** Al crear nuevos Pods, esta configuración
- incluye la plantilla de reclamación de volumen que se usará para cada Pod. El formato es idéntico al del recurso PVC que usamos en la sección anterior, sin las declaraciones de tipo y `apiVersion`.

Con esto en mente, creemos una nueva carpeta llamada `postgres-db` con los siguientes archivos:

- 1-secrets.yaml: Asegúrese de añadirlo a .gitignore , ya que contiene datos confidenciales. Este archivo se usará para configurar el nombre, el usuario y la contraseña predeterminados de la base de datos de Postgres. 2-
- statefulset.yaml: Este es el recurso StatefulSet que creará nuestros pods y PVC de Postgres para cada pod.

Cree un archivo llamado 1-secrets.yaml en el directorio postgres-db con el contenido de la siguiente lista.

Listado 8.16 Secretos para Postgres

```

Versión de API: v1
tipo: Secreto
metadatos:
  nombre: postgres-db-secret
tipo: Cadena
opacaDatos:
  NOMBRE DE LA BD: "postgres-db"
  DB_USER: "postgres"
  CONTRASEÑA DE LA BASE DE DATOS: "dQw4w9WgXcQ"
```

A continuación, cree un archivo llamado 2-statefulset.yaml en el directorio postgres-db y comience con el contenido de la siguiente lista.

Listado 8.17 StatefulSet para Postgres, parte 1

```
apiVersion: apps/v1 tipo:
```

```
StatefulSet metadatos:
```

```
  nombre: postgres
```

```
especificación:
```

```
  réplicas: 1
```

```
    selector:
```

```
      matchLabels:
```

```
    aplicación:
```

```
      postgres
```

```
    plantilla:
```

```
      metadatos: etiquetas: aplicación: postgres
```

En este punto, esto es idéntico a lo que podríamos ver en un recurso de implementación, excepto que usamos kind: Service. Antes de agregar la configuración de la plantilla de pod, agreguemos el bloque volumeClaimTemplates:. Los bloques spec.template y spec.volumeClaimTemplates deben estar al mismo nivel, como se muestra en la siguiente lista.

Listado 8.18 StatefulSet para Postgres, parte 2

apiVersion: apps/v1 tipo:

StatefulSet metadatos:

nombre: postgres

especificación:

réplicas: 1

selector:

matchLabels:

aplicación:

postgres plantilla: metadatos:

...

especificación:

...

volumeClaimTemplates: - metadatos:

nombre: db-vol

especificación: nombreDeClaseDeAlmacenamiento: linode-block-storage-retain

modosDeAcceso: ["Lectura/EscrituraUna Vez"]

recursos:

solicitudes:

almacenamiento: 20Gi

Ahora, comencemos a definir nuestra plantilla Pod dentro del StatefulSet. Primero, necesitamos que la etiqueta de metadatos coincida con el selector y, a continuación, definir el bloque initContainers , como se muestra en la siguiente lista.

Listado 8.19 StatefulSet para Postgres, parte 3

```
...
especulación:
  ...
  plantilla:
    metadatos:
      etiquetas:
        aplicación: postgres

    especificación: initContainers:
      - nombre: delete-lost-found imagen:
        alpine:latest comando: ["sh", "-c",
          "rm -rf /mnt/lost+found"] volumeMounts: - nombre: db-vol

      mountPath: /mnt
    volumeClaimTemplates: - metadatos:

      nombre: db-vol

    especulación:
      ...
```

Ahora podemos definir los contenedores: bloque.

Listado 8.20 StatefulSet para Postgres, parte 4

...

especulación:

...

plantilla:

metadatos:

contenedores: -

nombre: postgres-container imagen:

postgres:12.16 imagePullPolicy:

IfNotPresent puertos: - containerPort: 5432

entorno:

- nombre: POSTGRES_DB

valorDe:

secretKeyRef:

nombre: postgres-db-secret clave:

DB_NAME -

nombre: POSTGRES_USER

valorDe:

referenciaClaveSecreta:

nombre: postgres-db-secret clave:

DB_USER - nombre:

POSTGRES_PASSWORD valorDe:

secretKeyRef:

nombre: postgres-db-secret clave:

DB_PASSWORD

volumeMounts: -

nombre: db-vol

mountPath: /var/lib/postgresql volumeClaimTemplates:

- metadatos:

nombre: db-vol

especulación:

...

Este StatefulSet es un ejemplo muy práctico que combina gran parte de lo aprendido hasta ahora. A continuación, un breve resumen de lo que hemos definido:

- Una sola réplica (por ejemplo, réplicas: 1).
- Un selector que coincide con la etiqueta app: postgres.
- Una plantilla que coincide con la etiqueta app: postgres y tiene un contenedor con la imagen postgres:12.16.
- Las variables de entorno POSTGRES_DB, POSTGRES_USER y POSTGRES_PASSWORD se establecen en los valores DB_ relacionados en el recurso postgres-secrets .
- Un contenedor de inicio que monta el volumen y elimina el directorio lost+found .
- Una plantilla de reclamación de volumen que solicita un volumen de 20 GB con el modo de acceso ReadWriteOnce .

Antes de verificar este StatefulSet, crearemos un servicio para permitir el acceso a los pods de aprovisionamiento del StatefulSet mediante el selector definido. En esta ocasión, usaremos ClusterIP para no exponer nuestra base de datos a internet. Cree un archivo llamado 3-service.yaml en el directorio postgres-db con el contenido que se muestra a continuación.

Listado 8.21 Servicio ClusterIP para Postgres

```
apiVersion: v1 kind:
Servicio metadatos:
nombre:
  postgres-svc spec: tipo:

ClusterIP puertos: - puerto:
  5432
  targetPort: 5432
  selector:

  aplicación: postgres
```

Con estos manifiestos definidos, ejecute `kubectl apply -f postgres-db/`. Tras un breve periodo, la base de datos debería estar aprovisionada y ejecutándose en el pod `postgres-0`.

Para verificar el funcionamiento de la base de datos PostgreSQL, podemos usar la función de reenvío de puertos de `kubectl`. Esta función permite que nuestro equipo local, como `localhost`, se conecte a un recurso del clúster. Aquí hay algunas maneras:

- Reenvíe un puerto de Pod—`kubectl port-forward <pod-name> <local-port>:<k8s-container-port>` como `kubectl port-forward postgres-0 3312:5432` y luego en su computadora local, use `localhost:3312` junto con su cliente PostgreSQL favorito.
- Redireccionar un StatefulSet (`kubectl port-forward statefulsets/postgres 3312:5432`). Esto se hará de forma predeterminada en el primer pod del StatefulSet.
- Puerto hacia adelante a Servicio—`kubectl` reenvío de puertos servicio/postgres 3312:5432.

Si tiene el cliente PostgreSQL `psql` instalado en su máquina local, puede ejecutar `PG_PASSWORD=<password-set> psql -h localhost -U postgres -d <db-set> -p <port-set>` como `PG_PASSWORD="dQw4w9WgXcQ" psql -h localhost -U postgres -d postgres-db -p 3312`.

Si no tiene instalado el cliente `psql`, simplemente ejecute `kubectl get pods` (para obtener una lista de los pods en ejecución) y `kubectl get svc` (para obtener una lista de los servicios). Ambos comandos deberían verificar el funcionamiento de los pods. Si no se están ejecutando, use `kubectl describe pod <pod-name>` para comprobar qué sucede.

Aunque usar una versión local del cliente `psql` funciona bien, no nos muestra cómo establecer la comunicación entre contenedores desde nuestro clúster. Hagámoslo ahora.

8.4.3 Comunicación de contenedor a contenedor dentro de Kubernetes

Aunque creo que actualizar nuestras aplicaciones Python o Node.js con una base de datos sería una buena idea, requeriría demasiado código y perderíamos de vista los aspectos importantes de Kubernetes. En su lugar, usaremos algunas de las excelentes funciones de Kubernetes para conectarnos a nuestra base de datos desde el clúster. Para ello, crearemos un pod temporal y único, instalaremos el cliente PostgreSQL y luego nos conectaremos a nuestra base de datos.

Crear un contenedor cada vez que necesitas probar algo no siempre es práctico. En nuestro caso, queremos probar el comando `psql` desde nuestro clúster, pero no tenemos un contenedor con el comando `psql` instalado. Aunque esta situación podría no ocurrirte a menudo, es recomendable omitir la creación del contenedor y, en su lugar, usar un pod eliminable donde podamos instalar cosas.

Para ello, usaremos el aprovisionamiento manual de un pod directamente en la línea de comandos con `kubectl`. A largo plazo, siempre recomiendo usar manifiestos (YAML) para aprovisionar recursos, pero saber cómo crear un pod único con `kubectl` puede ayudarte a probar diversas configuraciones (es decir, con ConfigMaps o Secrets), imágenes de contenedor y comunicación entre contenedores en tu clúster. En muchos sentidos, crear pods únicos con `kubectl` es muy similar a crearlos con la interfaz de línea de comandos de Docker.

Para ejecutar un contenedor eliminable solo con Docker, ejecutamos el comando: `docker run --rm <container-image-name> --name <runtime-name> -p 8080:8080` donde `<container-image-name>` es el nombre de la imagen del contenedor, `<runtime-name>` es el nombre del contenedor y `-p 8080:8080` es la asignación del puerto.

La parte eliminable es el indicador `--rm`, que le indica a Docker que elimine el contenedor cuando salga.

Para ejecutar un Pod que se puede eliminar en cualquier momento con `kubecttl`, usaremos lo siguiente:

- Una imagen de contenedor `debian:bullseye-slim` porque tiene aproximadamente 30 MB y, por lo tanto, se aprovisiona rápidamente.
- Debian cuenta con el gestor de paquetes `apt`, al igual que Ubuntu; solo que ocupa menos espacio. Ubuntu es un sistema Debian, pero con muchas más herramientas integradas y, por lo tanto, ocupa más espacio. Dado que hemos usado Ubuntu a lo largo de este libro, estos comandos deberían resultarles familiares. `sleep infinity` es un comando de punto de entrada
- que podemos usar para mantener el contenedor del Pod en ejecución hasta que lo eliminemos. Si no incluyéramos `sleep infinity`, el Pod se cerraría inmediatamente después de iniciarse, ya que no hay ningún comando ni punto de entrada que ejecutar.

Con esto en mente, ejecutemos los siguientes comandos:

1. `kubecttl run my-psql --image=debian:bullseye-slim -- sleep infinity`. Después de ejecutar este comando, puede verificar el pod con `kubecttl get pods my-psql -o yaml`. Al usar `-o yaml`, se mostrará el manifiesto actual de este pod, incluyendo el estado en el clúster de K8s.
2. Acceda al shell del pod con `kubecttl exec -it my-psql -- /bin/sh`. Esto nos permitirá ejecutar comandos dentro del contenedor del pod y comunicarnos con otros pods del clúster.
3. Instale el cliente `psql` con `apt update && apt install -y postgresql-client`. No necesita el comando `sudo` debido a la configuración de la imagen del contenedor `debian:bullseye-slim`.

En este punto, tienes un pod creado manualmente que ejecuta un contenedor con psql instalado. Conectemos este cliente psql a nuestro pod de base de datos PostgreSQL mediante postgres-svc.

Para ello tenemos dos opciones:

- La IP del clúster para nuestro servicio postgres-svc
- El servicio DNS integrado de Kubernetes para postgres-svc
Servicio

Antes de ver el servicio DNS integrado, obtengamos el valor de la IP del clúster para este servicio. Es tan sencillo como ejecutar `kubectl get service <service-name>` como `kubectl get service postgres-svc`. El resultado debería ser similar al de la figura 8.13.

```
% kubectl get service postgres-svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
postgres-svc	ClusterIP	10.128.189.63	<none>	5432/TCP	159m

Figura 8.13 Salida del servicio `kubectl get`

En la columna Cluster-IP de la figura 8.13, el valor es 10.128.189.63.

Este valor corresponde a la dirección IP interna de este servicio en particular.

Apuesto a que tiene un valor diferente, pero incluso si no lo tuviera, es posible que sí.

El valor de la IP del clúster es el valor del host que podemos usar para conectarnos a este servicio desde nuestro clúster. Cabe recordar que los recursos del servicio de Kubernetes no se fijan en el tipo de conexión; la IP del clúster intentará reenviar las solicitudes al pod correcto, tal como lo haya configurado.

Si bien la IP del clúster es un enfoque válido, presenta un defecto importante: la dirección IP dada es impredecible. Una IP impredecible implica que la comunicación entre nuestros pods se convierte en un juego de comprobación constante de los valores de las direcciones IP internas aprovisionadas dinámicamente. Lo mejor es que podemos usar la función integrada de Kubernetes.

Servicio DNS para conectarse a nuestra base de datos utilizando valores muy predecibles. El formato es el siguiente:

```
<nombre-recurso>.<espacio-nombres>.<tipo-recurso>.cluster.local.
```

Para nuestro ejemplo, se desglosa de la siguiente manera:

- **<resource-name>**: El nombre de nuestro recurso es postgres-svc. El tipo de recurso, en este caso, es Servicio, pero esto no aplica aquí. **<namespace>**:
- El espacio de nombres predeterminado es literalmente "predeterminado". En breve veremos cómo cambiar este valor. **<resource-type>**:
- Dado que usamos un servicio, el valor **<resource-type>** es svc, de forma similar a como `kubectl get svc` mostrará todos los servicios. `cluster.local`: Este es el sufijo DNS predeterminado para todos
- los recursos en Kubernetes. Si bien esto se puede modificar, es un tema más avanzado.

A continuación se muestran algunos ejemplos de puntos finales de DNS para varios Servicios en varios espacios de nombres (más información sobre espacios de nombres en breve):

- `postgres-svc.default.svc.cluster.local` para el servicio `postgres-svc` en el espacio de nombres predeterminado . `redis-svc.default.svc.cluster.local` sería el punto final DNS para un servicio con el nombre `redis-svc` en el espacio de nombres predeterminado .
- `mysql.doubleunit.svc.cluster.local` sería el punto final DNS para un servicio con el nombre `mysql` en el espacio de nombres `doubleunit` . `mariadb.imf.svc.cluster.local` sería el punto final DNS para un servicio con el nombre `mariadb` en el espacio de nombres `imf` .
-

Te dejo decidir qué te resulta más fácil, el punto final DNS o la IP del clúster. Ahora, probemos ambos comandos, empezando por el punto final DNS correcto. Asegúrate de acceder al shell bash con `kubectl exec -it my-psql — /bin/sh`. Luego, ejecuta lo siguiente:

- `psql -h postgres-svc.default.svc.cluster.local -p 5432 -U postgres -d postgres-db`: Se le solicitará una contraseña. El nombre de usuario (-U), el nombre de la base de datos (-d) y la contraseña (se solicita) se configuraron en el recurso `postgres-secrets`. El puerto (-p) es el puerto predeterminado para PostgreSQL, así como el puerto que configuramos en el servicio `postgres-svc`.

1. Escriba el comando SQL: `CREATE DATABASE hello_world`; 2. Salga del cliente `psql` con `\q`.
3. Ahora, usando la IP del clúster, `psql -h 10.128.189.63 -p 5432 -U postgres -d postgres-db`. El único cambio es el nombre de host (-h).
4. Enumere las tablas con `\dt` y debería ver el Base de datos `hello_world` listada.
5. Salga del cliente `psql` con `\q`.
6. Salga del shell bash con `exit`.

Cada uno de estos pasos está encapsulado en la figura 8.14.

```

% kubectl exec -it my-psql -- /bin/sh
# psql -h postgres-svc.default.svc.cluster.local -p 5432 -U postgres -d postgres-db
Password for user postgres:
psql (13.11 (Debian 13.11-0+deb11u1), server 15.4 (Debian 15.4-1.pgdg120+1))
WARNING: psql major version 13, server major version 15.
        Some psql features might not work.
Type "help" for help.

postgres-db=# CREATE DATABASE hello_world
postgres-db=# \q
# psql -h 10.128.12.138 -p 5432 -U postgres -d postgres-db
Password for user postgres:
psql (13.11 (Debian 13.11-0+deb11u1), server 15.4 (Debian 15.4-1.pgdg120+1))
WARNING: psql major version 13, server major version 15.
        Some psql features might not work.
Type "help" for help.

postgres-db=# \l

```

List of databases					
Name	Owner	Encoding	Collate	Ctype	Access privileges
postgres	postgres	UTF8	en_US.utf8	en_US.utf8	
postgres-db	postgres	UTF8	en_US.utf8	en_US.utf8	
template0	postgres	UTF8	en_US.utf8	en_US.utf8	=c/postgres +
					postgres=CTc/postgres
template1	postgres	UTF8	en_US.utf8	en_US.utf8	=c/postgres +
					postgres=CTc/postgres

```

(4 rows)

postgres-db=# \q
# exit

```

Figura 8.14 Comunicación entre servicios y psql

Si bien esto es lo más básico que se puede hacer con los comandos SQL con psql y una base de datos PostgreSQL, es un excelente ejemplo de cómo conectarse a una base de datos desde dentro de un clúster de Kubernetes y, lo que es más importante, realizar la comunicación de contenedor a contenedor.

8.4.4 Espacios de nombres para administrar el clúster recursos

En esta sección, veremos rápidamente cómo usar espacios de nombres para agrupar recursos de una manera diferente.

Los espacios de nombres se pueden aplicar a cualquier recurso de Kubernetes y

Son una excelente manera de agrupar recursos por equipo, proyecto o incluso entorno.

Si no se usa ni se declara un espacio de nombres, todos los recursos se aprovisionarán en el espacio de nombres de Kubernetes, añadido automáticamente y llamado "predeterminado". Puede verificar la existencia del espacio de nombres predeterminado ejecutando "kubectl get namespaces" para revisar todos los espacios de nombres disponibles. En otras apply -f", se usó algo como . ., palabras, al ejecutar "kubectl "kubectl apply -f . . -n default". El indicador "-n" se usa para especificar un espacio de nombres (a menos que esté declarado en el manifiesto, como veremos más adelante).

Dado que acabamos de ver cómo los espacios de nombres pueden afectar a los endpoints DNS administrados por Kubernetes desde un clúster, crearemos otro almacén de datos de ejemplo con Redis. Redis es otra aplicación con estado que requiere un StatefulSet, al igual que PostgreSQL. En esta sección, usamos el recurso de espacio de nombres para destacar cómo podemos aislar y distinguir mejor los pods y servicios dentro de nuestro clúster. Claro que existen algunas diferencias técnicas entre Redis y PostgreSQL que quedan fuera del alcance de este libro, pero creo que es importante ver cómo los espacios de nombres pueden ayudar a dividir aún más estos dos programas.

Cree una nueva carpeta llamada redis-db con los siguientes archivos:

- 1-namespace.yaml: el recurso de espacio de nombres que creará nuestro espacio de nombres redis-db . 2-
- statefulset.yaml: el recurso StatefulSet que creará nuestros pods y PVC de Redis para cada pod.

Si bien Redis se puede proteger mediante una contraseña, nos centraremos más en el aspecto del espacio de nombres de este ejemplo.

Cree un archivo llamado 1-namespace.yaml en el directorio redis-db.

Listado 8.22 Recurso de espacio de nombres para Redis

```
apiVersion: v1 kind:
Espacio de nombres
metadatos:
  nombre: rk8s-redis
```

Antes de crear cualquier otra cosa, aprovisiona el espacio de nombres con `kubectl apply -f redis-db/1-namespace.yaml`. Una vez completado, ejecute `kubectl get pods -n rk8s-redis` y luego `kubectl get pods -n default`. Si su Postgres StatefulSet aún se está ejecutando, debería ver Pods en el segundo comando `get pods`, pero no en el primero. Esto se debe a que el indicador `-n` se usa para filtrar recursos por espacio de nombres.

Si bien esto parece similar a nuestro filtro selector, es diferente. Si bien los espacios de nombres se pueden usar para filtrar resultados con un selector, también podemos crear cuentas de usuario específicas para limitar el acceso a espacios de nombres y recursos específicos dentro de ellos. También podemos configurar políticas para evitar la comunicación interna entre espacios de nombres. Este tipo de configuración queda fuera del alcance de este libro, pero es útil conocerla.

¿Cómo añadimos recursos a un espacio de nombres? De dos maneras:

- Agregue el valor `metadata.namespace` al manifiesto del recurso. Esta es la forma más común de agregar un recurso a un espacio de nombres.
- Use el parámetro `-n` con `kubectl apply -f <manifiesto-de-recursos> -n <espacio-de-nombres>`, como `kubectl apply -f postgres-db/ -n rk8s-redis`. El parámetro `-n` con `kubectl apply` no funcionará si el manifiesto ya tiene definido un `metadata.namespace`.

Con esto en mente, cree el archivo `2-statefulset.yaml` en la carpeta `redis-db` para nuestro Redis StatefulSet.

Listado 8.23 StatefulSet para Redis

```

apiVersion: apps/v1 kind:
StatefulSet metadata: name:
redis
  namespace: rk8s-
  redis spec: replicas: 1 selector:

matchLabels: app:
  redis template:
    metadata: labels:

      aplicación: redis

  especificación:
    contenedores: -
      nombre: redis imagen: redis:latest
      puertos:
        - containerPort: 6379 volumeMounts:
          - nombre: redis-data
            mountPath: /data
            volumeClaimTemplates: -
      metadata: nombre: redis-data

  especificación: nombre de clase de almacenamiento: linode-block-
    storage modos de acceso: [ "Lectura y escritura única" ]
  recursos:
    solicitudes:
      almacenamiento: 10Gi

```

Como podemos ver, Redis no requiere una declaración InitContainers como PostgreSQL. Ahora, creemos un servicio para este StatefulSet. Cree un archivo llamado 3-service.yaml en el directorio redis-db .

Listado 8.24 Servicio ClusterIP para Redis

```
apiVersion: v1 kind:
Service metadata:
name: redis-
  svc namespace: rk8s-
  redis spec: type: ClusterIP ports: -
port:
  6379 targetPort: 6379
selector:
app: redis
```

Aplique los cambios con `kubectl apply -f redis-db/`. Tras un breve periodo, la base de datos debería estar aprovisionada y ejecutándose en el pod `redis-0`. Ahora pregúntese: ¿cuál es el punto final DNS de este servicio? Si respondió `redis-svc.rk8s-redis.svc.cluster.local`, está en lo cierto. Verifiquémoslo siguiendo un patrón similar al que usamos para verificar la base de datos PostgreSQL, StatefulSet y el servicio:

1. Ingrese al shell del Pod temporal con `kubectl exec -it my-psql—/bin/sh`.
2. Instale la biblioteca cliente de Redis con `apt update && apt instalar -y redis-tools`.
3. Verifique que el cliente Redis exista con el comando `-v redis-cli`, que debería devolver `/usr/bin/redis-cli`.
4. Utilice el comando `redis-cli ping` con el indicador `host -h` como: `redis-cli -h redis-svc.rk8s-redis.svc.cluster.local ping`. Esto debería devolver PONG, lo que significa que la conexión fue exitosa.

Ahora tenemos una base de datos Redis en funcionamiento a la que podemos conectarnos desde nuestro clúster. Además, podemos...

comunicarse a través de DNS interno de Pod a Pod a través de un recurso de servicio independientemente del espacio de nombres.

Si ya no usa el pod temporal, puede eliminarlo ahora con `kubectl delete pod my-psql`. También puede considerar eliminar sus recursos de PostgreSQL y Redis con `kubectl delete -f postgres-db/` y `kubectl delete -f redis-db/`, respectivamente. Al igual que con muchos recursos administrados en la nube, los volúmenes tendrán un costo adicional; aunque este monto no será alto, es recomendable saberlo y eliminar los recursos que no vaya a utilizar.

Ahora que entendemos completamente cómo implementar contenedores, tanto con estado como sin estado, pongamos nuestras aplicaciones Python y Node.js a trabajar en Kubernetes junto con un nombre de dominio personalizado.

8.5 Implementar aplicaciones en producción con Kubernetes

En esta sección, implementaremos nuestras aplicaciones sin estado y en contenedores en las que hemos estado trabajando a lo largo del libro. Dado que las aplicaciones no tienen estado, usaré un recurso de implementación de Kubernetes por defecto. Para implementar nuestras aplicaciones en producción, haremos lo siguiente:

- Cree ConfigMaps para las variables de entorno con cada aplicación.
- Definir manifiestos de implementación para implementar los contenedores en ejecución.
- Implementar un servicio de equilibrio de carga para enrutar el tráfico externo a nuestras aplicaciones.

Enrutar el tráfico externo, o basado en internet, a nuestras aplicaciones es fundamental para cualquier aplicación de producción. Para enrutar el tráfico entrante, también conocido como tráfico de entrada, contamos con dos opciones integradas en el recurso del Servicio Kubernetes: NodePort o LoadBalancer. Como ya se explicó, usar un Servicio NodePort usará un puerto en cada nodo. Si la configuración se deja en blanco, este puerto se asignará automáticamente a un valor de puerto HTTP no estándar para dar cabida a otros Servicios que sí lo usan. Si desea tener un dominio personalizado, necesita un puerto HTTP estándar de 80 o 443 para el tráfico de internet, ya que los navegadores web usan el puerto 80 por defecto para el tráfico HTTP y el 443 para el HTTPS. Usaremos el puerto 80, ya que el 443 es para tráfico seguro, pero requiere configuración y certificados adicionales que no se abordan en este libro. El uso de un puerto HTTP estándar (80 o 443) significa que podemos usar un dominio personalizado (por ejemplo, roadtok8s.com) sin especificar un número de puerto (por ejemplo, roadtok8s.com:30000) como lo haría con un servicio NodePort.

Dado que queremos usar el puerto 80, podríamos usar NodePort configurado en ese puerto, pero eso significaría que solo un servicio puede estar en el puerto 80, ya que solo un servicio puede ejecutarse en un solo puerto. Esto no es un problema si solo tienes un servicio, pero si tienes dos o más, necesitas usar un servicio LoadBalancer. Veamos un ejemplo.

8.5.1 Servicios de balanceador de carga

Todos los recursos del servicio Kubernetes realizan balanceo de carga, independientemente del tipo definido. Esto significa que, si se recibe una solicitud, Kubernetes la enrutará al pod correcto y considerará la cantidad de tráfico o carga que ese pod maneja actualmente. El recurso de servicio ClusterIP realiza el balanceo de carga interno entre los pods. El NodePort

Expone el puerto de un nodo para equilibrar la carga de las solicitudes a los pods de todo el clúster (independientemente del nodo del que provenga la solicitud). Si queremos equilibrar la carga entre las máquinas virtuales (incluido su estado actual) y entre los pods del clúster, podemos usar el tipo de servicio LoadBalancer.

Con un clúster de Kubernetes administrado, el uso del tipo de servicio LoadBalancer solicitará un balanceador de carga administrado al proveedor de la nube. En nuestro caso, este tipo implica solicitar un NodeBalancer directamente a Akamai Connected Cloud. Linode NodeBalancer es un servicio de balanceador de carga administrado que proporciona una dirección IP pública estática junto con solicitudes de balanceo de carga a los nodos de nuestro clúster. Los balanceadores de carga administrados como Linode NodeBalancer suelen tener costos adicionales: no suele ser un servicio gratuito.

En entornos de balanceadores de carga administrados, como NodeBalance en la nube conectada de Akamai, se suele proporcionar una dirección IP pública estática. Esta IP estática puede servir como destino para nuestro dominio personalizado configurando un registro de dirección (registro A). Un registro A es una configuración DNS dentro de nuestro servicio de alojamiento de dominios (p. ej., Route53, Linode Domains, Name.com, GoDaddy.com, etc.) que asigna un nombre de dominio a una dirección IP. Al actualizar este registro, podemos dirigir dominios personalizados como `py.book.roadtok8s.com` o `js.book.roadtok8s.com` a la IP pública estática de nuestro balanceador de carga. Esto facilita la asociación de dominios personalizados con servicios en un clúster de Kubernetes.

Antes de crear nuestros Servicios, cree la carpeta `apps` en el directorio `roadtok8s-kube` para ambas aplicaciones (Python y Node.js) y agregue lo siguiente como archivos de marcador de posición vacíos:

- 1-configmap.yaml 2-
- deployment-py.yaml 3-
- deployment-js.yaml 4-
- service-py.yaml 5-
- service-js.yaml

Antes de analizar el ConfigMap, pensemos en el nombre de dominio interno que queremos para el servicio de cada aplicación. Esto es lo que creo que sería útil:

- Python: py-book.default.svc.cluster.local
- Node.js: js-book.default.svc.cluster.local

La razón por la que los defino ahora es porque serán nuestra guía para el ConfigMap y el Servicio de cada aplicación. Con esto en mente, crearemos un ConfigMap para cada aplicación. Cree un archivo llamado 1-configmap.yaml en el directorio de aplicaciones.

Listado 8.25 ConfigMap para aplicaciones

```
apiVersion: v1 tipo:
ConfigMap metadatos:
nombre: rk8s-
  apps datos:

PY_ENDPOINT: "py-book.default.svc.cluster.local"
PY_PORT: "8080"
JS_ENDPOINT: "js-book.default.svc.cluster.local"
JS_PORT: "3000"
```

Cada uno de estos valores se utilizará en los recursos de implementación para cada aplicación, lo que significa que necesitamos este ConfigMap en nuestro clúster antes de que se creen los recursos de implementación (de ahí el prefijo 1-).

Si bien PY_ENDPOINT y JS_ENDPOINT no se utilizarán en este libro, existen para mostrar más ejemplos de puntos finales de DNS en caso de que decida modificar sus aplicaciones web para comunicarse entre sí en el clúster.

Con esto en mente, crearemos los recursos de implementación para cada aplicación. Cree un archivo llamado 2-deployment-py.yaml en el directorio de aplicaciones.

Listado 8.26 Implementación de la aplicación Python

```

apiVersion: apps/v1 kind:
Implementación metadatos:
nombre: py-
  deploy especificación:
réplicas:
  3 selector:
  matchLabels:
    aplicación: py-
    deploy plantilla:
metadatos:
  etiquetas:
    aplicación:
      py-deploy

  especificación:
    contenedores: - nombre: rk8s-py
      imagen: codingforentrepreneurs/rk8s-py:latest puertos: - nombre: puerto
      http
      containerPort: 8080

    entorno:
      - nombre: PUERTO
        valorDe:
          configMapKeyRef:
            nombre: rk8s-apps clave:
              PY_PORT envFrom:
            -
          configMapRef: nombre:
            rk8s-apps

```

La única novedad es que hemos nombrado el puerto del contenedor como puerto http. Este nombre facilita su reutilización al aprovisionar el servicio. Si bien este paso es opcional, resulta muy práctico cuando se realizan múltiples implementaciones con varios puertos de contenedor, como en nuestro caso.

Para crear la versión Node.js de esta implementación, copie este archivo, cámbiele el nombre a 3-deployment-js.yaml y actualice los siguientes valores:

- Cambie el valor de containerPort para que coincida con el puerto de la aplicación Node.js. En mi caso, lo cambiaré a 3000.
- Reemplace todas las instancias de py- con js- para que el nombre y el selector sean js-deploy y PY_ con JS_ , de modo que usemos JS_PORT para la variable de entorno PORT.
- Establezca la imagen del contenedor en codingforentrepreneurs/rk8s-js:latest o la imagen que tenga en Docker Hub.
- Establezca el valor del PUERTO en 3000.

En este punto, puede ejecutar `kubectl apply -f apps/` y verificar que los pods se estén ejecutando para ambas imágenes de contenedor. Si hay errores, considere usar nginx como imagen de contenedor para verificar que el contenedor que está usando no sea el problema. Como recordatorio, puede usar los siguientes comandos para verificar que los pods se estén ejecutando:

- `kubectl obtener pods -l app=py-deploy` o `kubectl obtener pods -l app=js-deploy`
- `kubectl describe implementaciones/py-deploy` o `kubectl describe implementaciones/ js-deploy`
- Implementaciones de registros de `kubectl/py-deploy` o implementaciones de registros de `kubectl/js-deploy`

Con las implementaciones implementadas, crearemos los recursos de servicio para cada aplicación. Cree un archivo llamado 4-service-py.yaml en el directorio de aplicaciones .

Listado 8.27 Servicio para la aplicación Python

```
apiVersion: v1 kind:
Service metadata:
name: py-book
  spec: type:

LoadBalancer ports: - port: 80
  targetPort:
  http-port selector:
    app: py-deploy
```

Queremos establecer el puerto en 80 porque este es el valor del puerto HTTP predeterminado, lo que significa que nuestra aplicación estará disponible en `http://<loadbalancer-static-ip>:80` y `http://<loadbalancer-static-ip>`;

Una vez más, copie este archivo, cámbiele el nombre a `5-service-js.yaml` y actualice los siguientes valores:

- Cambie el nombre del servicio de `py-book` a `js-book` (¿recuerda los nombres DNS?).
- Cambie el selector de `py-deploy` a `js-deploy`.

Ahora ejecute `kubectl apply -f apps/` y verifique que los Pods y los Servicios se crearon con `kubectl get pods` y `kubectl get svc` (o `kubectl get services`), lo que debería resultar muy parecido a la figura 8.15.

```

% kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
js-deploy-86b77bf6d7-h5n2s        1/1     Running   0           3m18s
js-deploy-86b77bf6d7-qr4cm        1/1     Running   0           3m22s
js-deploy-86b77bf6d7-wzht6        1/1     Running   0           3m19s
my-psql                            1/1     Running   0           23h
py-deploy-648989c4cb-7k7jr        1/1     Running   0           3m22s
py-deploy-648989c4cb-cd4fx        1/1     Running   0           3m18s
py-deploy-648989c4cb-p6dw4        1/1     Running   0           3m20s

% kubectl get svc
NAME            TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
js-book         LoadBalancer 10.128.253.219   172.232.176.126 80:30890/TCP     16s
kubernetes      ClusterIP     10.128.0.1      <none>           443/TCP          5d19h
py-book         LoadBalancer 10.128.62.33    172.232.176.29  80:32000/TCP     3m26s

```

Figura 8.15 Salida de kubectl get pods y servicios

Ambos servicios deben tener direcciones IP externas que dirigirán el tráfico a sus implementaciones a través del servicio LoadBalancer. Si no ve una dirección IP externa, espere unos minutos, ya que el LoadBalancer a veces tarda en aprovisionar una nueva dirección IP pública estática. Si sigue sin ver una dirección IP externa, utilice `kubectl describe service py-book` o `kubectl describe service js-book` para comprobar la situación.

Ahora que tiene una dirección IP pública estática, puede apuntar su nombre de dominio a ella. Para ello, debe usar un registro A cuyo valor sea la dirección IP del servicio LoadBalancer y donde el subdominio puede ser el que usted elija.

8.5.2 Implementación en Kubernetes con GitHub Actions

Ahora es el momento de automatizar las implementaciones de Kubernetes con flujos de trabajo de CI/CD en GitHub Actions. Como antes, el flujo de trabajo que diseñará aquí se puede modificar para que funcione con otras plataformas de CI/CD con pequeños ajustes.

Razón: todas las herramientas son de código abierto. La clave para automatizar una implementación de Kubernetes es...

- Una cuenta de servicio válida con los permisos correctos
- Un archivo kubeconfig válido para la cuenta de servicio
- Una versión válida de kubectl
- Un clúster de Kubernetes en ejecución

La creación y gestión de cuentas de servicio es un tema de seguridad más avanzado que queda fuera del alcance de este libro. Si te interesa, puedes leer mi entrada de blog (<https://www.codingforentrepreneurs.com/blog/kubernetes-rbac-service-account-github-actions/>). ya que está relacionada con GitHub Actions y Kubernetes con Linode Kubernetes Engine de Akamai Connected Cloud.

Si bien este proceso puede realizarse en los mismos repositorios de GitHub que nuestras aplicaciones Python y Node.js, recomiendo separar la infraestructura (Kubernetes) de la aplicación. Nuestras aplicaciones pueden seguir compilando los contenedores, pero no necesitan ser responsables de configurar el clúster de Kubernetes.

Nuestro flujo de trabajo hará lo siguiente:

- Agregue nuestro archivo kubeconfig a la ubicación predeterminada de kubectl : ~/.kube/config.
- Echo ejecutando pods e implementaciones en nuestro espacio de nombres predeterminado con kubectl get pods y kubectl get deployments.
- Aplique cualquier cambio de manifiesto en la carpeta apps/ con kubectl apply -f apps/.
- Aplicar un reinicio de implementación a las implementaciones de py-deploy y js-deploy con kubectl rollout restart deployment py-deploy y kubectl rollout restart deployment js-deploy.

El comando `kubectl rollout` es una excelente manera de actualizar implementaciones sin tener que eliminarlas y volver a crearlas sin tiempos de ejecución.

Como habrás notado en los manifiestos de implementación (p. ej., `apps/2-deployment-py.yaml`), usamos la etiqueta `:latest` para cada imagen. Si ejecutamos `kubectl apply -f apps/` de nuevo con el mismo manifiesto, Kubernetes no sabrá que debe actualizar la implementación, ya que el manifiesto no ha cambiado. Si el manifiesto cambia, Kubernetes sabrá que debe iniciar una nueva implementación.

Establecer explícitamente la etiqueta del contenedor de la implementación (por ejemplo, `:latest`) en un número de versión o ID de confirmación se considera la mejor práctica para las implementaciones de producción por los siguientes motivos:

- Registro histórico : auditoría y reversión más sencillas directamente en el historial de su repositorio Git.
- Depuración: encuentre rápidamente problemas con cualquier contenedor determinado.
- Errores de implementación: si la implementación falla por completo en una versión automatizada, será mucho más fácil identificar y revertir las versiones defectuosas.
- Inmutabilidad: si se hace correctamente, cada etiqueta será una versión única e inmutable.
- Pruebas de implementaciones: simplifica la prueba de múltiples versiones de la misma imagen de contenedor.

Aunque existen otras razones, estas son las consideraciones más comunes para usar una etiqueta de imagen de contenedor específica, especialmente en producción. Implementar etiquetas de contenedor explícitas queda fuera del alcance de este libro, pero lo recomiendo para implementaciones en producción.

Para fines de desarrollo y aprendizaje, creo que usar `:latest` suele ser una excelente opción, ya que permite actualizar rápidamente los contenedores en ejecución sin tener que cambiar el manifiesto ni el flujo de trabajo de GitHub Actions para la creación de contenedores. Por eso, ejecutamos `kubectl rollout restart deployment <deployment-name>` para forzar un despliegue con el mismo manifiesto. El comando `kubectl rollout restart` también es práctico si alguna vez necesita reiniciar un despliegue después de realizar actualizaciones relacionadas con la configuración (por ejemplo, los cambios en los secretos o los mapas de configuración no activan automáticamente los despliegues).

Aunque puedes configurar manualmente la imagen de un contenedor con el comando `kubectl set image deployment/<deployment-name> <container-name>=<image-name>:<image-tag>`, no recomiendo hacerlo en un flujo de trabajo de GitHub Actions, ya que si tu manifiesto tuviera `:latest` y usaras `kubectl set` para usar `:v1`, tu manifiesto no estaría sincronizado. La razón principal es que el archivo YAML del manifiesto del repositorio no coincidiría con el valor de la imagen del despliegue real, lo que podría provocar una o varias implementaciones no deseadas al ejecutar `kubectl apply -f apps/` o algo similar.

Con esto en mente, creemos nuestro flujo de trabajo dentro del directorio `roadtok8s-kube` en `./.github/workflows/k8s-apps-deploy.yaml`.

Listado 8.28 Flujo de trabajo de GitHub Actions para implementar nuestras aplicaciones

Nombre: Actualizar las implementaciones de la aplicación K8s
en:

flujo de trabajo_despacho:

trabajos:

verify_service_account: nombre:

Verificar la cuenta de servicio de K8s se ejecuta en:

ubuntu-latest pasos: - nombre:

Verificar

el código del repositorio usos: actions/

checkout@v4 - usos: azure/setup-

kubectll@v3 - nombre: Crear/Verificar el directorio

`.kube`

ejecutar: mkdir -p ~/.kube/ - nombre:

Crear configuración de kubectl ejecutar: | cat <<

EOF >>

~/.kube/config \${{ secrets.KUBECONFIG_SA }}

EOF

- nombre: Echo pods

ejecutar:

| kubectl get pods - nombre:

Echo implementaciones ejecutar: |

kubectl

get implementaciones

- nombre: Aplicar cambios de la aplicación

ejecutar:

| kubectl apply -f apps/ - nombre:

Implementar aplicaciones

ejecutar:

| kubectl rollout restart deployment -n default py-de

táctica

kubectl rollout restart implementación -n predeterminado js-de

táctica

Asegúrate de confirmar este flujo de trabajo junto con el resto de tus manifiestos de Kubernetes. Una vez confirmado, agrega

Añade tu archivo kubeconfig a los secretos de tu repositorio con el nombre KUBECONFIG_SA y el valor correspondiente al contenido de tu archivo kubeconfig. Esto permitirá que GitHub Actions use tu archivo kubeconfig para conectarse a tu clúster de Kubernetes.

Te dejo que pruebes y ejecutes este flujo de trabajo. Si falla, asegúrate de probar cada uno de estos comandos localmente para comprobar que funcionan.

¡Felicitaciones! Has implementado con éxito varias aplicaciones en un clúster de Kubernetes. Si bien es un gran logro, te permitirá acceder al amplio mundo de Kubernetes, más allá del alcance de este libro. Espero que continúes tu aprendizaje y crees, implementes y publiques más aplicaciones.

En el siguiente y último capítulo, analizaremos dos alternativas a Kubernetes para implementar contenedores: Docker Swarm y HashiCorp Nomad. Si bien creo que Kubernetes es la mejor opción, conviene saber qué otras opciones existen y cómo se comparan con Kubernetes.

Resumen

- Los archivos Kubeconfig se utilizan para conectarse a un clúster de Kubernetes con la CLI de Kubernetes, kubectl.
- La variable de entorno KUBECONFIG se puede utilizar para establecer la ubicación del archivo kubeconfig para su uso con kubectl.
- Las cuentas de servicio permiten acceder a diversos recursos dentro de un clúster. La cuenta de servicio predeterminada tiene permisos ilimitados en el archivo Kubeconfig.

- Los pods son el recurso más pequeño de Kubernetes y se pueden definir y aprovisionar para una implementación de contenedores rápida y eficiente.
- Las implementaciones se utilizan para administrar y escalar pods sin estado en un clúster de nodos de Kubernetes.
- Los StatefulSets se utilizan para administrar pods con estado a escala e incluyen soporte integrado para escalar y adjuntar volúmenes con pods escalados.
- Las implementaciones y los StatefulSets están diseñados para garantizar que una cantidad específica de pods se estén ejecutando en función del recuento de réplicas y, si un pod se elimina, falla o no se ejecuta, se aprovisionará un nuevo pod para cumplir con el recuento de réplicas.
- Actualizar o cambiar una imagen de contenedor dentro de una implementación o un StatefulSet activará una implementación de Pod, que es un reemplazo de Pods sin tiempo de inactividad.
- Los ConfigMaps son recursos reutilizables que almacenan valores clave estáticos que se pueden adjuntar a otros recursos como Pods, Implementaciones y StatefulSets como valores de variables de entorno y volúmenes de solo lectura.
- Los secretos son exactamente iguales a los ConfigMaps, pero ocultan los datos de los valores de los pares clave-valor mediante codificación Base64. Están diseñados para ser legibles cuando se adjuntan a pods, implementaciones y conjuntos con estado como valores de variables de entorno y volúmenes de solo lectura.
- El uso de servicios de terceros como HashiCorp Vault, AWS Secrets Manager o Google Secrets Manager es una buena alternativa a Kubernetes Secrets para obstruir valores más allá de la codificación Base64.
- Los selectores se pueden utilizar para filtrar pods y otros recursos por pares clave-valor.
- Los espacios de nombres se pueden utilizar para agrupar recursos y para filtrarlos.

- Los servicios administran la comunicación interna y externa con pods, implementaciones y conjuntos con estado.
- Los recursos de servicio de tipo ClusterIP solo permiten tráfico interno y no son accesibles desde fuera del clúster.
- Los recursos del servicio NodePort exponen un puerto en cada nodo para enrutar el tráfico externo a los pods.
- El recurso del servicio LoadBalancer utiliza un servicio de balanceador de carga administrado en la nube para enrutar el tráfico externo a los nodos y pods de Kubernetes.
- Los tipos de servicio ClusterIP, NodePort y LoadBalancer pueden dirigir y equilibrar la carga del tráfico interno a través de un punto final DNS interno o una dirección IP aprovisionada dinámicamente denominada IP de clúster.
- La comunicación de contenedor a contenedor es posible mediante el uso de recursos del Servicio.
- Los registros A, o registros de dirección, se utilizan para configurar un dominio personalizado en una dirección IP estática.
- GitHub Actions puede automatizar y aprovisionar recursos de Kubernetes con archivos de manifiesto mediante el uso de un archivo kubeconfig válido con una cuenta de servicio con los permisos correctos y kubectl.

9 herramientas de orquestación alternativas

Este capítulo cubre

- Ejecución de múltiples contenedores con Docker y Docker Enjambre
- Modificar archivos de Docker Compose para ejecutarlos en Docker Enjambre
- Ejecución de múltiples contenedores con HashiCorp Nomad
- Uso de la interfaz de usuario integrada de Nomad para administrar trabajos de contenedores

Un objetivo clave de este libro es ayudarle a adoptar el uso de contenedores como estrategia de implementación para cualquier aplicación. Hemos visto que Kubernetes es una forma eficaz de ejecutar y gestionar contenedores en producción. Si bien considero que Kubernetes es valioso para proyectos de todos los tamaños, puede ser útil conocer algunas alternativas destacadas: Docker Swarm y HashiCorp Nomad.

Docker Swarm es básicamente Docker Compose distribuido en un clúster de máquinas virtuales. Veremos cómo funciona la implementación técnica, pero se trata básicamente de un archivo de Docker Compose ligeramente modificado que se conecta a las máquinas de trabajo que ejecutan Docker. Docker Swarm es una buena opción para quienes ya están familiarizados con Docker Compose y desean escalar sus aplicaciones en múltiples máquinas virtuales.

Después de analizar Docker Swarm, implementaremos una herramienta de orquestación de contenedores llamada HashiCorp Nomad. Nomad utiliza

El lenguaje de configuración propietario de HashiCorp, llamado HCL, en lugar de YAML. Esta diferencia en la sintaxis de configuración podría ser un factor decisivo para algunos, pero HCL se utiliza en muchas herramientas de HashiCorp, como Terraform y Vault, por lo que podría ser útil aprenderlo. Nomad funciona de forma similar a Docker Swarm, pero presenta algunas diferencias clave que veremos en este capítulo.

La ejecución de GitOps (por ejemplo, flujos de trabajo de CI/CD con GitHub Actions) con estas herramientas está fuera del alcance de este libro. Sin embargo, esto no es muy diferente a lo que hemos hecho a lo largo de este libro. Empezamos creando un clúster de máquinas virtuales y usando Docker Swarm.

9.1 Orquestación de contenedores con Docker Swarm

Tradicionalmente, Docker Compose se ejecuta en un solo host, como una máquina virtual basada en la nube o su computadora local. Cada servicio declarado en una configuración de Docker Compose normalmente está vinculado a una única instancia en ejecución de una imagen de contenedor (por ejemplo, `codingforentrepreneurs/rk8s-py`).

Docker Swarm utiliza prácticamente la misma configuración que Docker Compose (en realidad, es una configuración heredada de Docker Compose) para ejecutar contenedores en varias máquinas virtuales. Docker Swarm no está diseñado para desarrollo local, sino para implementaciones en producción.

Afortunadamente, puedes usar Docker Swarm y Docker Compose indistintamente, como veremos, lo cual es importante para verificar que tu configuración funciona antes de implementarla en un clúster de Docker Swarm.

La terminología de Docker Swarm es la siguiente:

- **Nodo administrador** : La máquina o máquinas virtuales que administran los nodos de trabajo. El nodo administrador básicamente les indica a los nodos de trabajo qué ejecutar. De forma predeterminada, un nodo administrador también es un nodo de trabajo.
- **Nodo de trabajo** : La máquina o máquinas virtuales que ejecutan las tareas (p. ej., contenedores) que el nodo administrador les indica. Un nodo de trabajo no puede ejecutarse sin un nodo administrador.
- **Tarea**: Este es el término de Docker Swarm para un contenedor en ejecución. El nodo administrador indica al nodo trabajador qué tarea (contenedor) ejecutar. Las tareas no mueven nodos, por lo que, una vez definidas, permanecen en su lugar.
- **Servicio**: al igual que con Docker Compose, un servicio es la definición de una tarea a ejecutar en los nodos de administrador o de trabajo.
- **Pila**: Una pila es un conjunto de servicios implementados en el clúster. Se define en un archivo de Docker Compose.

Para relacionar estos términos con Kubernetes, podemos pensarlos de la siguiente manera:

- **Docker Swarm Manager** es como el plano de control de Kubernetes. Al igual que usamos Linode Kubernetes Engine (como plano de control de K8s) para gestionar tareas de Kubernetes como la programación de pods y la configuración de nodos, Docker Swarm Manager desempeña funciones similares, como la programación de tareas y la gestión de clústeres, dentro del entorno de Docker Swarm.
- **Docker Swarm Worker** es similar al nodo de Kubernetes. Ejecuta las tareas (o contenedores) que le indica el Docker Swarm Manager. El nodo de Kubernetes ejecuta los pods (o contenedores) que le indica el plano de control de Kubernetes.
- La **tarea Docker Swarm** es como un pod de Kubernetes. La tarea Docker Swarm es un contenedor en ejecución.

g Kubernetes Pod es un contenedor en ejecución.

- El servicio Docker Swarm es como una implementación de Kubernetes. El servicio Docker Swarm es un conjunto de tareas (o contenedores) implementadas en el clúster. La implementación de Kubernetes es un conjunto de pods (o contenedores) implementados en el clúster.
- Docker Swarm Stack es como un espacio de nombres de Kubernetes. La pila Docker Swarm es una colección de servicios (o contenedores) implementados en el clúster. El espacio de nombres de Kubernetes es una colección de pods (o contenedores) implementados en el clúster.

Ahora que tenemos algunas definiciones, vamos a aprovisionar una máquina virtual para un nodo administrador. Estos pasos le resultarán familiares, ya que ya lo hemos hecho antes con una excepción (direcciones IP privadas):

1. Cree una cuenta en Akamai Connected Cloud en linode.com/rk8s (los lectores del libro también reciben un crédito promocional).
2. Inicie sesión en Akamai Connected Cloud (anteriormente Linode; <https://linode.com/>).
3. Haga clic en el menú desplegable y seleccione Crear > Linode.
4. Utilice la siguiente configuración:
 - Distribución: Ubuntu 22.04 LTS (o la última versión LTS)
 - Región: Seattle, WA (o la región más cercana a usted)
 - Plan— CPU compartida > Linode 2 GB (o más)
 - Etiqueta: rk8s-swarm-manager-server (etiqueta importante)
 - Etiquetas: <opcional>
 - Contraseña root: Establece una buena.
 - Claves SSH : <recomendado>. Consulta el apéndice D para aprender a crear una y añadirla a Linode.

- Adjuntar una VLAN—<omitir>
- Complementos:
 - Copias de seguridad—<opcional>
 - IP privada—Seleccionada

5. Haga clic en Crear Linode.

Si olvida seleccionar la IP privada, puede agregarla más tarde haciendo lo siguiente:

1. Navegue a la página de detalles de su instancia.
2. Haga clic en la pestaña Red.
3. Haga clic en Agregar una dirección IP.
4. En IPv4, seleccione IP privada.
5. Haga clic en Asignar (si no puede hacer clic en este botón, es posible que ya tenga una dirección IP privada asociada a su instancia).

Antes de acceder a esta máquina virtual por SSH, anotemos la dirección IP privada asignada. En la página de detalles de su instancia, vaya a la pestaña Red y desplácese hasta el final de la página. La Figura 9.1 muestra la dirección IP privada de nuestra instancia.

IP Addresses		IP Transfer		IP Sharing	Add An IP Address
Address	Type ^	Default Gateway	Subnet Mask	Reverse DNS	
192.168.233.56	IPv4 - Private		255.255.128.0		...
139.177.203.114	IPv4 - Public	139.177.203.1	255.255.255.0	139-177-203-114.ip.linodeusercontent.com	...
fe80::f03c:93ff:fe5f:03ae	IPv6 - Link Local	fe80::1	ffff:ffff:ffff:ffff::		
2600:3c02::f03c:93ff:fe5f:03ae	IPv6 - SLAAC	fe80::1	ffff:ffff:ffff:ffff::		...

Figura 9.1 Dirección IP privada para nuestra instancia de máquina virtual

Ahora tenemos tres atributos clave sobre nuestra máquina virtual:

- La dirección IP pública (para nuestra sesión SSH)
- La dirección IP privada (para nuestro nodo de servidor Docker Swarm)
- La etiqueta del nodo del servidor Docker Swarm Manager (por ejemplo, rk8s-swarm-manager-server)

Con estos atributos en mente, preparemos nuestra máquina virtual como un nodo administrador de Docker Swarm.

9.1.1 Preparación del Docker Swarm Manager

Para nuestro clúster Swarm, tendremos un total de tres máquinas virtuales: un administrador y dos nodos de trabajo. Tras configurar los nodos de trabajo, usaremos el administrador como nuestro agente (o nodo) principal para administrar el clúster mediante un archivo `compose.yaml` específico para Swarm. En otras palabras, solo realizamos cambios en Swarm a través del nodo administrador; los nodos de trabajo simplemente hacen lo que se les indica.

Acceda por SSH a nuestra máquina virtual recién creada usando la dirección IP pública e instale Docker. El listado 9.1 muestra estos pasos brevemente. Asegúrese de reemplazar `<public-ip-address>` con la dirección IP pública de su máquina virtual.

Listado 9.1 Instalación del motor Docker en Ubuntu

```
ssh root@<dirección IP pública>#1 curl  
-fsSL https://get.docker.com | sudo bash#2
```

#1 Muestra el comando SSH estándar #2 Este es un acceso directo de una línea para instalar Docker a través de un script oficial.

Una vez completados estos pasos, verifique que Docker funcione correctamente ejecutando `docker ps`. Debería ver que no hay contenedores activos. Si esta parte falla, revise los capítulos anteriores.

Con Docker instalado, configuraremos tres servicios: un balanceador de carga NGINX, nuestra aplicación Python en contenedores (`codingforentrepreneurs/rk8s-py`) y nuestra aplicación Node.js en contenedores (`codingforentrepreneurs/rk8s-js`). Cada uno de estos servicios tendrá los siguientes nombres:

- Web: nuestra aplicación
- Python API: nuestra aplicación
- Node.js Nginx: nuestro balanceador de carga NGINX

En Docker Compose, declararemos estos nombres como nombres de servicio, como ya hicimos. El solucionador DNS integrado de Docker Compose permite la detección de servicios utilizando el nombre del servicio como nombre de host. Dicho de otro modo, se puede acceder al servicio web mediante `http://web:8080` (suponiendo que se usa el puerto 8080) desde cualquier contenedor administrado por Docker Compose.

Dado que podemos usar nombres de servicio como nombres de host DNS, configuraremos NGINX para distribuir el tráfico a cada servicio y equilibrar la carga según lo previsto. Si usa nombres de servicio diferentes, asegúrese de que la configuración de NGINX y la de Docker Compose coincidan.

Desde su sesión SSH y directamente en la máquina virtual, cree un archivo llamado `nginx.conf` con el contenido del listado 9.2. Normalmente, recomendaría usar Git y una canalización de CI/CD para implementar este archivo de configuración, pero para simplificar, lo haremos directamente en la máquina virtual.

Listado 9.2 Archivo de configuración de NGINX para equilibrar la carga de los servicios de Docker Compose

```

backend web ascendente
    { servidor web:8080;#1
    }

servidor
    { escuchar 80; #2

    ubicación /
        { contraseña_proxy http://web_backend;
        encabezado_conjunto_proxy Host $host;#3
        encabezado_conjunto_proxy X-Real-IP $dirección_remota;
        encabezado_conjunto_proxy X-Reenviado-Para $dirección_proxy_x_reenvío
ed_para; }

    ubicación /api
        { contraseña_proxy http://api:3000;#4
        encabezado_conjunto_proxy Host
        $host; encabezado_conjunto_proxy X-Real-IP
        $dirección_remota; encabezado_conjunto_proxy X-Reenviado-Para $dirección_proxy_x_re
ed_para; }

    }

```

#1 Web es el nombre del servicio Docker Compose para nuestra aplicación Python.

#2 El puerto NGINX predeterminado es

80. #3 web_backend reenviará el tráfico a todos los servidores definidos en el bloque ascendente del mismo nombre.

#4 Api es el nombre del servicio Docker Compose para nuestra aplicación Node.js.

Si bien esta configuración de NGINX será efectiva, es una de las más sencillas por diseño para garantizar que nos centremos en Docker Swarm. Docker Swarm también cuenta con mecanismos integrados de equilibrio de carga que pueden funcionar en conjunto con la imagen del contenedor NGINX.

Ahora cree el archivo `compose.yaml` en su servidor con Docker Compose instalado. La siguiente lista nos muestra exactamente eso con una configuración que vimos anteriormente en este libro.

Listado 9.3 Archivo Docker Compose antes de Docker Swarm

```
servicios:
  nginx:
    imagen: nginx
    puertos:
      - 80:80
    volúmenes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf depende_de: - web - api

  web:
    imagen: codingforentrepreneurs/rk8s-py entorno:
      - PUERTO=8080

  API:
    imagen: codingforentrepreneurs/rk8s-js
```

Verifique esta configuración ejecutando `docker compose up` o `docker compose -f compose.yaml up` (para una versión más detallada). Una vez iniciado cada servicio, acceda a la dirección IP de su servidor en el navegador web de su equipo local (p. ej., `http://<your-ip-address>`). Acceder a la ruta raíz (`/`) debería mostrar la aplicación Python en contenedor (p. ej., `codingforentrepreneurs/rk8s-py`). Acceder a la ruta `/api` debería mostrar la aplicación Node.js en contenedor (p. ej., `codingforentrepreneurs/rk8s-js`).

Como recordatorio, NGINX es una herramienta potente con numerosas opciones de configuración. En nuestro ejemplo sencillo, NGINX reenvía las solicitudes y la ruta solicitada a...

Servicio apropiado. Esto significa que la ruta `/api` en NGINX reenvía a la ruta `/api` en nuestra aplicación Node.js. Hay maneras de modificar este comportamiento mediante la configuración de NGINX, pero hacerlo queda fuera del alcance de este libro.

Suponiendo que nuestra configuración de Docker Compose funciona correctamente, la desactivamos con `Control + C` o el comando `docker compose down`.

Verificamos primero el funcionamiento de Docker Compose porque Docker Swarm es básicamente una versión mejorada de Docker Compose para un clúster de máquinas virtuales.

Docker Swarm utiliza una versión heredada del archivo de configuración de Docker Compose, por lo que necesitaremos realizar algunas modificaciones en nuestro archivo `compose.yml` para que funcione con Docker Swarm.

9.1.2 Docker Compose para Docker Swarm

El archivo Docker Compose que necesitamos para Docker Swarm requiere algunas capas adicionales de configuración.

El primero es cómo vinculamos los archivos locales a un servicio. En nuestro caso, ¿cómo vinculamos el archivo `nginx.conf` al servicio `nginx` :? Debemos modificar `volumes`: de `./nginx.conf:/etc/nginx/conf.d/default.conf` a lo que se indica en el listado 9.4.

Listado 9.4 Definición de volumen para Docker Swarm

```
versión: '3.8' # versión heredada de Docker Compose servicios: nginx:
```

```
...
```

```
  volúmenes:
```

```
    - tipo: enlace
```

```
      origen: ./nginx.conf
```

```
      destino: /etc/nginx/conf.d/default.conf
```

Esta modificación puede funcionar con Docker Compose porque es una sintaxis heredada para la versión 3 del archivo Compose. Revise la documentación de Docker Compose

(<https://docs.docker.com/compose/compose-file/compose-file-v3/>) para obtener más información sobre la sintaxis de la versión 3.

Dado que Docker Swarm es una herramienta de orquestación de contenedores, podemos definir el número de réplicas (o instancias) de cada servicio que queremos ejecutar en la directiva "deploy: ". Cuantas más instancias se ejecuten, mayor capacidad de procesamiento necesitará el clúster, por lo que podemos seguir añadiendo nodos a nuestro enjambre.

En su instancia de Docker Swarm Manager, cree el archivo `compose-swarm.yaml`, que incluye los bloques de la directiva `deploy:` y los recuentos de réplicas que desea para cada servicio.

Listado 9.5 Bloques de implementación para Docker Swarm

```
versión: '3.8' # versión heredada de Docker Compose servicios: nginx: imagen:
nginx puertos:
  - 80:80
  volúmenes:

  - tipo: enlazar
    origen: ./nginx.conf destino: /etc/
    nginx/conf.d/default.conf implementar: réplicas: 1 web: imagen:

    codingforentrepreneurs/
rk8s-
  py entorno:

  - PUERTO=8080
  implementar:
    réplicas: 5 api:

imagen: codingforentrepreneurs/cfe-nginx implementar: réplicas: 2
```

Como puede ver, este archivo heredado de Docker Compose, compatible con Swarm, no es muy diferente de lo que vimos antes. De hecho, podría ejecutar `docker compose -f compose-swarm.yaml` y funcionaría perfectamente, aunque las réplicas en la directiva "deploy: " simplemente se ignorarían.

Ahora que tenemos nuestro archivo Docker Compose listo para Docker Swarm, iniciemos el clúster Swarm.

9.1.3 Iniciar Docker Swarm

No lo olvidemos: Docker Swarm sirve para ejecutar múltiples contenedores en varias máquinas virtuales. En este punto,

Solo tenemos una máquina virtual que designamos como Docker Swarm Manager.

Para iniciar un enjambre, necesitamos nuestra dirección IP privada para que otras instancias puedan unirse y comunicarse con esta máquina virtual. Podríamos usar una dirección IP pública, pero esto podría suponer un riesgo de seguridad. Además, sería más costoso, ya que la comunicación saldría de la red, pasaría por internet y luego volvería a la red.

Así es como inicializamos el enjambre:

1. SSH en la instancia de Docker Swarm Manager.
2. Ejecute `docker swarm init --advertise-addr <ip-privada-dirección>`.

Eso es todo. Después de ejecutar este comando, debería ver un mensaje como el de la figura 9.2. Este resultado indica cómo otros nodos pueden unirse al enjambre con `docker swarm join --token <some-token> <private-ip-address>:2377`. También puede obtener este token ejecutando el trabajador `docker swarm join-token` en la instancia de Docker Swarm Manager.

```
root@localhost:~# docker swarm init --advertise-addr 192.168.216.115
Swarm initialized: current node (9hbuymz7lvr9yg9508f0jxjpe) is now a
manager.
```

To add a worker to this swarm, run the following command:

```
docker swarm join --token SWMTKN-1-2xg2lpl05p3qjgtzs71qi05jucpxt8
3msmmvmordwixpx745w-cw1hgh0lzyjt1on1roupstxx4 192.168.216.115:2377
```

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

Figura 9.2 Salida de inicialización de Docker Swarm

Ahora que tenemos un enjambre inicializado, agreguemos algunos nodos al clúster. Para ello, necesitamos hacer lo siguiente:

1. Cree dos nuevas máquinas virtuales en la misma región (necesaria para la comunicación IP privada) con direcciones IP privadas y etiquetadas como rk8s-swarm-worker-1 y rk8s-swarm-worker-2.
2. Conéctese por SSH a cada una de las nuevas máquinas virtuales.
3. Instale Docker Engine en cada una de las nuevas máquinas virtuales con `curl -fsSL https://get.docker.com | sudo bash`.
4. Use `docker swarm join --token <some-token> <private-ip-address>:2377` para unirse al enjambre. Al ejecutar este comando, se obtendrá una declaración como: Este nodo se unió a un enjambre como trabajador. Repita este paso para cada una de las nuevas máquinas virtuales.

La cantidad de nodos de trabajo depende de usted. Creo que un buen punto de partida para su clúster son tres nodos (incluido el administrador). Los pasos anteriores son los mismos para cada nodo que desee agregar al clúster.

Ahora que tenemos nuestros nodos de administrador y de trabajo configurados y nuestro archivo de composición listo para enjambre, es hora de implementar nuestro Docker Swarm Stack.

9.1.4 Implementar una pila Docker Swarm

Una pila Docker Swarm es muy similar a ejecutar Docker Compose Up, pero para un clúster de máquinas. Las opciones disponibles son las siguientes:

- `docker stack deploy -c <compose-file> <stack-name>`: Este comando desplegará una pila en el enjambre. El indicador `-c` indica el archivo de composición que se usará y `<stack-name>` es el nombre de la pila. El nombre de la pila se usa para identificarla en el enjambre; debe elegir un nombre usted mismo.

- `docker stack ls`: este comando listará todas las pilas que se están ejecutando en el enjambre. Sí, puedes tener varias.
- `docker stack ps <stack-name>`: este comando listará todas las tareas (o contenedores) que se ejecutan en la pila. Esta es una buena forma de ver si su pila se está ejecutando correctamente. `docker stack rm <stack-name>`: este comando eliminará la pila del enjambre. Esto no eliminará las imágenes de los nodos, solo la pila. `docker ps`: este comando seguirá listando todos los contenedores que se ejecutan en el nodo.
- Este comando funciona tanto para el nodo administrador como para el nodo trabajador. `docker exec <container-id> /bin/bash`: así es como puede ingresar al shell Bash de cualquier contenedor
- en ejecución. Este comando es el mismo que vimos antes. La razón clave por la que se incluye es para mencionar que debe ejecutar este comando en el nodo en el que se ejecuta la tarea (o contenedor).

Ahora, en su sesión SSH en la instancia de Docker Swarm Manager, ejecute `docker stack ls`. No debería ver ninguna pila ejecutándose. Ahora ejecute `docker stack deploy -c compose-swarm.yaml rk8s`. El resultado debería ser como se muestra en la figura 9.3.

```
root@localhost:~# docker stack deploy -c compose-swarm.yaml rk8s
Creating network rk8s_default
Creating service rk8s_nginx
Creating service rk8s_web
Creating service rk8s_api
```

Figura 9.3 Salida de la implementación de Docker Swarm Stack

Con esta pila implementada, puede ejecutar `docker stack ls` y ver la pila `rk8s` listada con tres servicios. También puede ejecutar `docker stack ps rk8s` para ver las tareas (o contenedores) que se ejecutan en la pila.

Si visita la dirección IP pública de cualquiera de las máquinas virtuales en su clúster Docker Swarm, debería ver los mismos resultados que vio solo con Docker Compose, solo que esta vez tendrá muchas más instancias de cada servicio en ejecución.

Cuando necesite realizar cambios en esta pila de Docker Swarm, puede ajustar el archivo `compose-swarm.yaml` y volver a ejecutar `docker stack deploy -c compose-swarm.yaml rk8s`. Docker Swarm solo actualizará los servicios modificados.

Docker Swarm es una excelente manera de crear rápidamente contenedores en diversas máquinas virtuales. Docker Swarm se basa en el conocimiento de Docker Compose, por lo que es recomendable familiarizarse con Docker Compose antes de usarlo.

La gran ventaja que tiene Docker Swarm sobre Kubernetes es lo sencillo que es migrar de Docker Compose a Docker Swarm, ya que utilizan gran parte de la misma configuración.

Esta configuración también es sencilla y no requiere una curva de aprendizaje pronunciada. Dicho esto, Kubernetes, y en especial Kubernetes administrado, ofrece la enorme ventaja de aprovisionar rápidamente direcciones IP estáticas, balanceadores de carga, volúmenes de almacenamiento persistente y otros recursos en la nube. Docker Swarm no incorpora esta capacidad. Kubernetes también cuenta con un amplio ecosistema de herramientas específicas de Kubernetes que no están disponibles para Docker Swarm.

El reto que les dejo es cómo automatizar la configuración de nuevas instancias en su enjambre. Les doy una pista: tiene que ver con GitHub Actions. Ahora veamos otro enfoque para orquestar contenedores con HashiCorp Nomad.

9.2 Nómada de HashiCorp

Nomad se aleja considerablemente de las diversas herramientas que hemos utilizado a lo largo de este libro. Si bien Nomad está diseñado para ejecutar contenedores, su configuración se basa en el lenguaje de configuración propio de HashiCorp (HCL) en lugar de YAML. HCL se asemeja más a JSON que a YAML, pero sigue siendo un lenguaje declarativo utilizado en muchas herramientas populares como HashiCorp Terraform y HashiCorp Vault.

Para algunos, el uso de HCL supone un impedimento para adoptar Nomad. Para otros, encajará perfectamente con sus proyectos HashiCorp existentes (por ejemplo, Terraform, Vault, Consul).

Para nuestros propósitos, veremos un ejemplo básico del uso de Nomad después de configurar un clúster de máquinas virtuales.

9.2.1 Preparación de nuestro clúster Nomad

Antes de crear nuestro clúster, veamos algunos términos clave relacionados con Nomad:

- **Servidor:** El servidor es el agente principal que administra el clúster. Es responsable de mantener el estado del clúster y de programar las tareas (o contenedores) que se ejecutan en los nodos cliente.
- **Cliente:** El cliente es la máquina virtual que ejecuta las tareas (o contenedores) que el servidor le indica. Un cliente no puede ejecutarse sin un servidor.
- **Tarea:** Este es el término que Nomad utiliza para referirse a un contenedor en ejecución. El servidor le indica al cliente qué tarea (contenedor) ejecutar. Una vez definida, una tarea permanece en el mismo nodo y no se mueve de un nodo a otro.

- Trabajo: Un trabajo es un conjunto de tareas implementadas en el clúster. Se define en un archivo de configuración de Nomad.

Para relacionar estos términos con Kubernetes, podemos pensarlos de la siguiente manera:

- Nomad Server es como el plano de control de Kubernetes. Al igual que usamos Linode Kubernetes Engine (como plano de control de K8s) para gestionar tareas de Kubernetes como la programación de pods y la configuración de nodos, Nomad Server desempeña funciones similares, como la programación de tareas y la gestión de clústeres, dentro del entorno Nomad.
- El Cliente Nomad es como el Nodo de Kubernetes. El Cliente Nomad ejecuta las tareas (o contenedores) que el Servidor Nomad le indica. El Nodo de Kubernetes ejecuta los Pods (o contenedores) que el Plano de Control de Kubernetes le indica.
- Nomad Task es como un pod de Kubernetes. Nomad Task es un contenedor en ejecución. El pod de Kubernetes es un contenedor en ejecución.
- Un trabajo Nomad es similar a una implementación de Kubernetes. El trabajo Nomad es un conjunto de tareas (o contenedores) implementadas en el clúster. La implementación de Kubernetes es un conjunto de pods (o contenedores) implementados en el clúster.

Para preparar nuestro clúster Nomad, necesitamos crear tres máquinas virtuales: una que actúe como servidor Nomad y dos como clientes Nomad. Estos son los pasos a seguir:

1. Cree una cuenta en Akamai Connected Cloud en linode.com/rk8s (los lectores del libro también reciben un crédito promocional).

2. Inicie sesión en Akamai Connected Cloud (anteriormente Linode; <https://linode.com/>).
3. Haga clic en el menú desplegable y seleccione Crear > Linode.
4. Utilice la siguiente configuración:
 - Distribución: Ubuntu 22.04 LTS (o la última versión LTS)
 - Región: Seattle, WA (o la región más cercana a usted)
 - Plan—CPU compartida > Linode 2 GB (o más)
 - Etiqueta: rk8s-nomad-1 (aumentar según sea necesario)
 - Etiquetas: <opcional>
 - Contraseña root: Establece una buena.
 - Claves SSH : <recomendado>. Consulta el apéndice D para aprender a crear una y añadirla a Linode.
 - Adjuntar una VLAN—<omitir>
 - - Complementos: Copias de
 - seguridad—<opcional> IP privada—Seleccionada
5. Haga clic en Crear Linode.

Después de unos minutos, deberías tener tres máquinas virtuales con lo siguiente:

- Instancias etiquetadas como rk8s-nomad-1, rk8s-nomad-2 y rk8s-nomad-3
- Una dirección IP pública y una privada por instancia

9.2.2 Instalación de Nomad

A diferencia de Docker, HashiCorp no cuenta con un script oficial de acceso directo para instalar Nomad en una máquina virtual. Por ello, creé una versión de código abierto de este libro basada en los pasos de instalación oficiales de la documentación de HashiCorp Nomad ([https://](https://developer.hashicorp.com/nomad/docs/install)

developer.hashicorp.com/nomad/docs/install). Para revisar el script, puedes visitar el Camino a Kubernetes

Repositorio Nomad (<https://github.com/jmitchel3/roadtok8s-nomad>).

Como mínimo, necesitamos instalar Nomad y Docker Engine en cada una de nuestras máquinas virtuales. Para ello, siga estos pasos:

1. Acceda por SSH a la máquina virtual.
2. Instale Nomad ejecutando

```
curl -fsSL https://raw.githubusercontent.com/jmitchel3/roadtok8s-nomad/main/install-nomad.sh | sudo bash
```

3. Instalar Docker Engine <https://get.docker.com> con rizo -fsSL | sudo bash.
4. Verifique que Nomad esté instalado con la versión Nomad y que Docker Engine esté instalado con la versión Docker.

Repita este proceso para cada máquina virtual. Una vez que tenga Nomad y Docker Engine instalados en cada máquina virtual, estará listo para configurar Nomad.

9.2.3 Configuración del servidor Nomad

El servidor Nomad es el cerebro del clúster Nomad. Para configuraciones más avanzadas, podemos distribuir este cerebro entre varias máquinas virtuales, incluso entre regiones, pero para nuestros fines, solo usaremos una máquina virtual como servidor Nomad.

La configuración predeterminada de Nomad, suponiendo que esté instalada correctamente, debería estar disponible en `/etc/nomad.d/nomad.hcl`. Esta configuración predeterminada es más bien un marcador de posición para las diversas configuraciones que pueda necesitar. No entraremos en los detalles de todas estas opciones; en cambio,

Cubriremos los pocos puntos críticos para garantizar que nuestro clúster esté configurado. Antes de reemplazar el suyo, puede considerar revisarlo para ver cómo es la configuración predeterminada y qué opciones están disponibles para usted.

Antes de ver la configuración completa, aquí están las declaraciones que haremos y lo que significan:

- `data_dir = "/opt/nomad/data"`—Esta es la ubicación predeterminada donde Nomad almacena datos sobre el clúster.
- `bind_addr = "0.0.0.0"`: Usar 0.0.0.0 en cualquier máquina significa que escuchará en todas las direcciones IP disponibles (esto aplica a cualquier proceso vinculado a esta misma dirección IP). Este valor garantiza que Nomad se ejecute en cualquier dirección IP pública o privada. `name = "nomad-server-1"`: Puede elegir el nombre de su servidor Nomad.
- Recomendando incrementar el nombre a medida que agregue más servidores. `advertise` (nuestra dirección IP privada): Esta es la dirección IP que otros nodos del clúster usarán para
- comunicarse con este nodo; usaremos tres subatributos (`http`, `rpc`, `serf`) para garantizar que nuestros clientes Nomad puedan comunicarse con este servidor Nomad.
- servidor con `enabled = true` y `bootstrap_expect = 1`—
Estas configuraciones garantizarán que esta instancia de Nomad se ejecutará como un servidor y que solo debe esperar un Nomad servidor.
- cliente con `enabled = false`;—Si bien puedes ejecutar un servidor Nomad como cliente, no se recomienda.

Con esto en mente, comencemos a crear nuestro primer archivo de configuración HCL para Nomad. Estos son los pasos:

1. Acceda por SSH a la instancia del servidor Nomad.

2. Cree el directorio `/opt/nomad/data` con `mkdir -p /opt/nómada/datos`.
3. Eliminar el valor predeterminado configuración con `rm /etc/nomad.d/nomad.hcl`.
4. Cree el archivo `/etc/nomad.d/nomad.hcl` con el contenido del listado 9.6. Asegúrese de reemplazar la dirección IP privada 192.168.204.96 por la dirección IP privada de su servidor Nomad.

Listado 9.6 Configuración del servidor Nomad

```
# Ubicado en /etc/nomad.d/nomad.hcl data_dir = "/opt/
nomad/data"

dirección_de_enlace = "0.0.0.0"

nombre = "servidor-nómada-1"

anunciar {
  http = "192.168.204.96" rpc =
  "192.168.204.96" serf =
  "192.168.204.96"
}

servidor
{ habilitado = verdadero
  bootstrap_expect = 1 # número de servidores
}

cliente
{ habilitado = falso
}
```

Si está familiarizado con JSON (Notación de Objetos JavaScript), puede ver parte de la inspiración para HCL. HCL usa llaves (`{}`) para indicar un bloque de configuración, corchetes (`[]`) para indicar una lista de valores y el signo `=` para

denotan un par clave-valor. YAML, por otro lado, utiliza sangría para indicar un bloque de configuración, un guion (-) para indicar una lista de valores y dos puntos (:) para indicar un par clave-valor.

Con este archivo creado, podemos ejecutar Nomad de dos maneras diferentes:

- Como servicio en segundo plano con `systemctl` (recomendado)
- Como servicio en primer plano con agente `nómada`

Al comparar la CLI de Nomad con la CLI de Kubernetes (`kubectl`), es importante tener en cuenta que la CLI de Nomad no es tan completa como `kubectl`. La CLI de Nomad se centra en la programación de trabajos, la gestión de tareas y la monitorización básica. `kubectl`, por otro lado, permite realizar diversas operaciones complejas, así como diversos complementos que permiten ampliar su funcionalidad.

Ejecutar Nomad como servicio en primer plano es una excelente manera de probar la configuración, pero no se recomienda para producción. Ejecutar Nomad como servicio en segundo plano es la forma recomendada para ejecutar Nomad en producción. Afortunadamente, la configuración del servicio en segundo plano ya está incluida:

1. Habilite el servicio en segundo plano con `sudo systemctl enable nomad`. Es necesario habilitarlo una vez.
2. Inícielo con `sudo systemctl start nomad`.
3. Si realiza cambios en Nomad, ejecute `sudo systemctl restart nomad`. (Si los cambios no se propagan, ejecute `sudo systemctl daemon-reload` antes de reiniciar).
4. Verifique el servicio en segundo plano con `sudo systemctl status nómada`.

5. Una forma de ayudar a diagnosticar problemas con el servicio Nomad en segundo plano es ejecutar `sudo journalctl -u nomad`.

Con Nomad ejecutándose en segundo plano, ejecute `nomad server members` y debería ver un resultado similar a la figura 9.4.

```
root@localhost:~# nomad server members
Name          Address      Port  Status  Leader  Raft Version  Build  Datacenter  Region
nomad-server-1.global 192.168.204.96 4648  alive   true    3             1.6.1  dc1         global
```

Figura 9.4 Salida de los miembros del servidor Nomad

Los valores clave a buscar son los siguientes:

- Nombre: debe ser el nombre que declaró en la configuración con `.global` agregado al mismo.
- Dirección: debe ser la dirección IP privada de su servidor Nomad (probablemente no 192.168.204.96 como se muestra).
- Estado: este valor debe estar activo; de lo contrario, tenemos un problema con nuestra configuración.
- Líder: Este valor se establecerá como verdadero, ya que es el único servidor Nomad que tenemos. Si tuviéramos más de uno, cada miembro del servidor elegiría automáticamente un líder, y este valor sería falso para todos los miembros del servidor excepto uno.
- Centro de datos: podemos modificar este valor, pero `dc1` es el valor predeterminado y lo que debería ver aquí.

Antes de configurar nuestros clientes Nomad, accedamos a la interfaz de usuario del servidor Nomad para verificar su correcto funcionamiento. Para ello, necesitamos la dirección IP pública del servidor Nomad y acceder al puerto 4646, ya que es el predeterminado. Abra su navegador en `http://<dirección-ip-pública>:4646/` y será redirigido a `/ui/jobs`, donde verá la interfaz de usuario de Nomad, como se muestra en la figura 9.5.

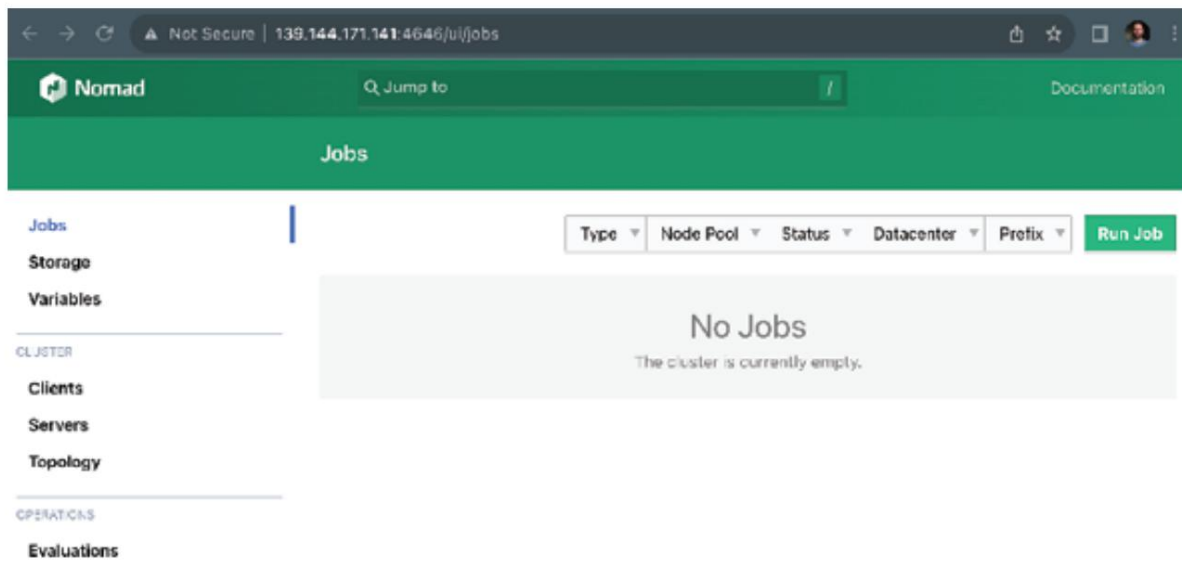


Figura 9.5 Interfaz de usuario del servidor Nomad

La interfaz de usuario de Nomad es una excelente manera de administrar nuestro clúster de Nomad. Pronto usaremos la interfaz para crear nuestro primer trabajo y ejecutar nuestros primeros contenedores con Nomad. También podríamos usar la línea de comandos, y en algunos casos, sin duda lo haríamos, ya que la interfaz de usuario de Nomad es pública por defecto. Para este sencillo ejemplo, basta con usar la interfaz. Antes de crear trabajos, necesitamos configurar nuestros clientes de Nomad. Hagámoslo ahora.

9.2.4 Configuración de los clientes Nomad

Los clientes Nomad tienen el mismo proceso de instalación que el servidor Nomad (p. ej., se instalan tanto Nomad como Docker), pero la configuración de `nomad.hcl` es ligeramente diferente. La siguiente lista muestra el archivo de configuración de nuestro primer cliente Nomad, al que llamaremos `nomad-client-a`.

Listado 9.7 Configuración del cliente Nomad

```
# Ubicado en /etc/nomad.d/nomad.hcl

directorio_de_datos = "/opt/nomad/datos"

dirección_de_enlace = "0.0.0.0"

nombre = "cliente nómada"

servidor
{
  habilitado = falso
}

cliente
{
  habilitado = verdadero
  opciones =
  {
    "driver.raw_exec.enable" = "1"
    "docker.privileged.enabled" = "verdadero"
  }
  servidores = ["192.168.204.96:4647"] unión_al_servidor
  {
    reintentar_unirse =
    {
      ["192.168.204.96:4647"]
    }
  }
}

ui
{
  habilitado = falso
}
```

Tenemos dos nuevas declaraciones en este archivo de configuración:

- cliente con los subatributos habilitados
 - = verdadero: esto es necesario para que el cliente se ejecute.
 - opciones = {}
 - driver.raw_exec.enable = true: esto es necesario para que el cliente se ejecute.

- `docker.privileged.enabled = true`: Esto es necesario para que el cliente se ejecute.
 - `servers = ["<nomad-server-private-ip>:4647"]`: Esta es la dirección IP privada del servidor Nomad. Así es como el cliente se comunicará con el servidor.
 - `server_join: retry_join = ["<nomad-server-private-ip>:4647"]`: De nuevo, esta es la dirección IP privada del servidor Nomad.
 - Esto garantizará que el cliente siga intentando conectarse al servidor hasta que lo consiga.
 - `server` con `enabled` siendo `false`, por lo que no intentará ejecutar esta instancia como servidor.
- - `ui` con el valor "habilitado" como falso, por lo que no se expone la interfaz de usuario en esta instancia. Úselo en cualquier servidor o cliente Nomad donde desee deshabilitar la interfaz de usuario.

Con estos conceptos claros, configuremos nuestro cliente Nomad. Estos son los pasos:

1. Inicie sesión mediante SSH en su instancia de cliente Nomad.

2. Instalar Nomad con

```
curl -fsSL https://raw.githubusercontent.com/jmitchel3/roadto-k8s-nomad/main/install-nomad.sh | sudo bash
```

3. Instalar Docker con `curl -fsSL https://get.docker.com | sudo bash`.

4. Cree el directorio `/opt/nomad/data` con `mkdir -p /opt/nomad/data`.

5. Elimine la configuración predeterminada con `/etc/nomad.d/nomad.hcl`. rm

6. Cree el archivo `/etc/nomad.d/nomad.hcl` con el contenido del listado 9.7.

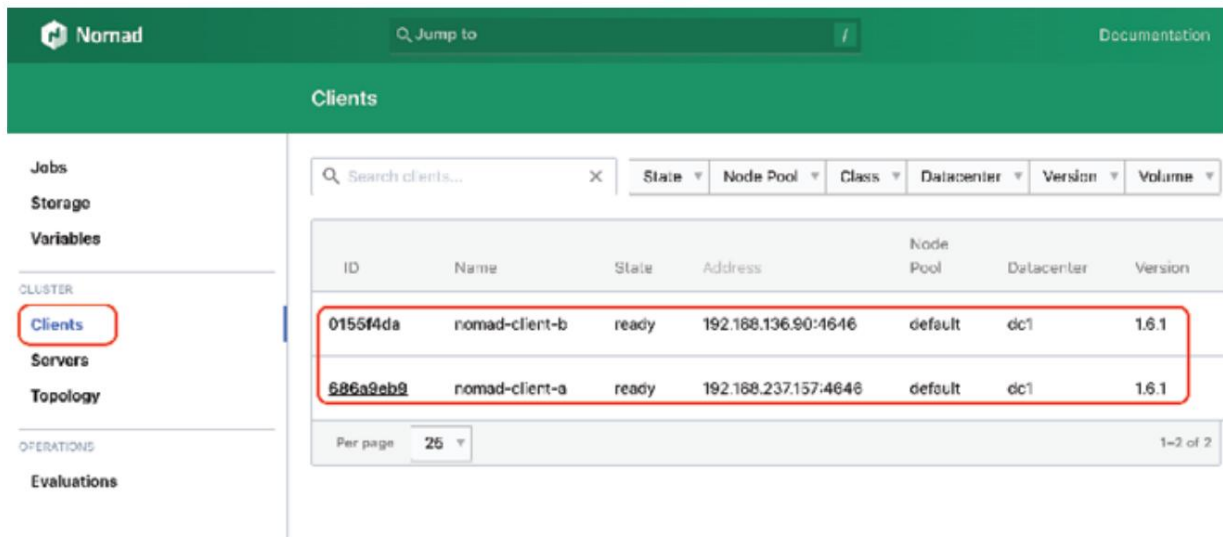
- Cambie el nombre="nomad-client" a nombre="nomad-client-a" (o nomad-client-b para el segundo cliente).
 - Reemplace la dirección IP privada indicada de 192.168.204.96 con la dirección IP privada de su servidor Nomad.
7. Habilite e inicie el servicio en segundo plano con `sudo systemctl enable nomad` y `sudo systemctl start nomad`.

Después de completar este paso con ambos clientes, acceda a su servidor Nomad por SSH y ejecute el comando "nomad node status". La Figura 9.6 muestra el resultado de este comando.

```
root@localhost:~# nomad node status
ID           Node Pool  DC   Name           Class  Drain  Eligibility  Status
0155f4da    default   dc1  nomad-client-b  <none> false  eligible    ready
686a9eb9    default   dc1  nomad-client-a  <none> false  eligible    ready
```

Figura 9.6 Salida del estado del nodo del servidor Nomad

También puede verificar estos datos en la interfaz de usuario de Nomad (por ejemplo, <http://<nomad-server-public-ip-address>:4646/ui/>) y navegar a la pestaña Clientes como se ve en la figura 9.7.



ID	Name	State	Address	Node Pool	Datacenter	Version
0155f4da	nomad-client-b	ready	192.168.136.90:4646	default	dc1	1.6.1
686a9eb9	nomad-client-a	ready	192.168.237.157:4646	default	dc1	1.6.1

Figura 9.7 Clientes de la interfaz de usuario del servidor Nomad

Ahora que tenemos el servidor Nomad y los clientes Nomad, es hora de crear nuestro primer trabajo Nomad.

9.2.5 Ejecución de contenedores con trabajos Nomad

Para ejecutar contenedores con Nomad, debemos definir una tarea mediante HCL. Las definiciones de tarea se encuentran dentro de las definiciones de grupo, y las definiciones de grupo se encuentran dentro de las definiciones de trabajo de Nomad (de nuevo, como HCL). Dicho de otro modo, cada trabajo puede tener varios grupos, y cada grupo dentro de un trabajo puede tener varias tareas (o contenedores). El diseño general se muestra en la siguiente lista.

Listado 9.8 Definición del trabajo nómada

```
trabajo "un_nombre_de_trabajo_único"
{ grupo "grupo_único_por_trabajo" {
  tarea "tarea única por grupo" { controlador =
    configuración "docker"
    { imagen =
      "codingforentrepreneurs/rk8s-py"
    }
  }
}
```

Las definiciones en el listado 9.8 son un trabajo válido, pero faltan algunas configuraciones clave (más sobre esto pronto).

Como era de esperar, necesitamos un entorno de ejecución de Docker (como Docker Engine) para ejecutar el contenedor, así como una imagen de contenedor válida y de acceso público (p. ej., `codingforentrepreneurs/rk8s-py`); ambos elementos se declaran en la definición de la tarea. Las imágenes de contenedor privadas también pueden ejecutarse, pero requieren una configuración adicional que queda fuera del alcance de este libro.

Para probar esta configuración de trabajo, creemos un nuevo trabajo en nuestro servidor Nomad. Estos son los primeros pasos:

1. Abra su servidor Nomad en `http://<nomad-server-public-dirección IP>:4646/ui/`.
2. Haga clic en la pestaña Trabajos.
3. Haga clic en el botón Ejecutar trabajo.

En este punto, puedes pegar el código del listado 9.8, como puedes ver en la figura 9.8.

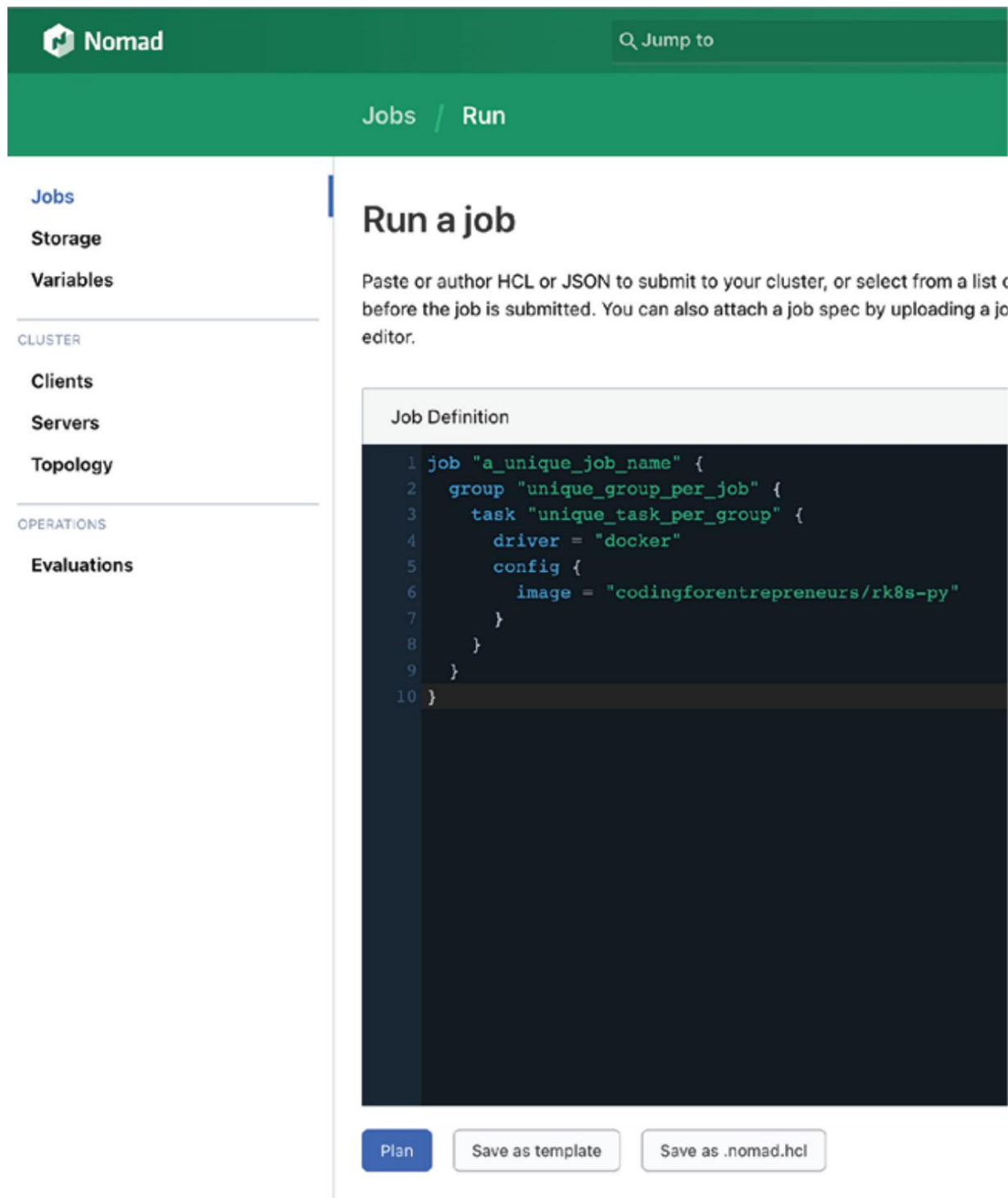
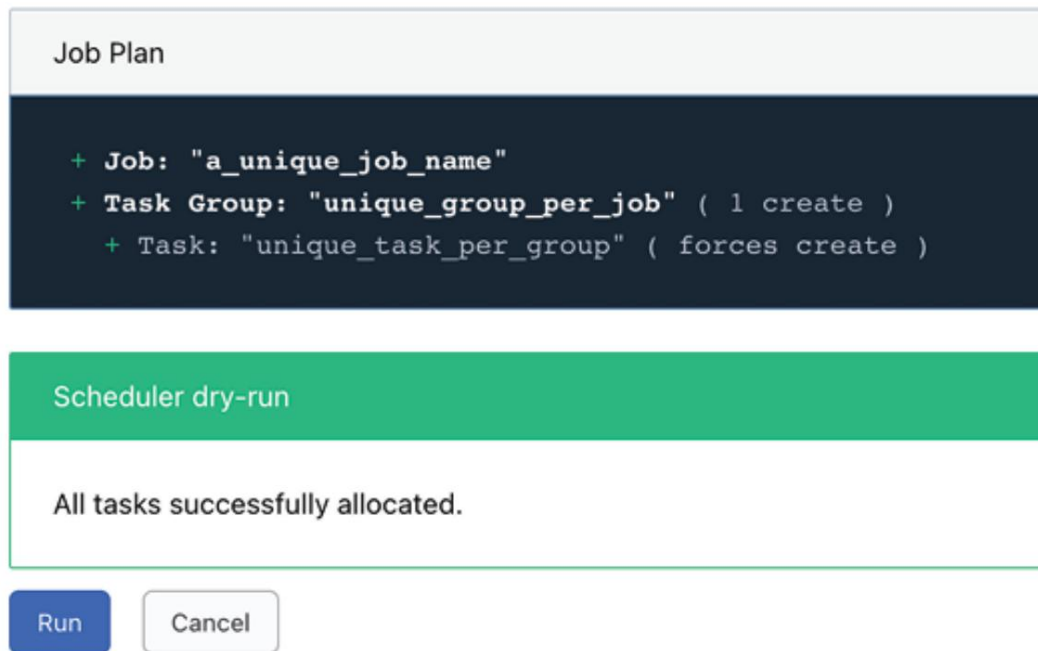


Figura 9.8 Tarea de ejecución de la interfaz de usuario del servidor Nomad

Después de hacer clic en Planificar, verá el resultado de lo que el trabajo pretende hacer. Las herramientas de HashiCorp como Terraform son excelentes para...

Le informamos de lo que sucederá antes de que acepte hacerlo. Este sencillo plan se muestra en la figura 9.9.



The screenshot displays the Nomad user interface for job planning. It features a 'Job Plan' section with a dark background showing a job definition in a code-like font. Below this is a green 'Scheduler dry-run' section indicating a successful allocation. At the bottom, there are 'Run' and 'Cancel' buttons.

```
+ Job: "a_unique_job_name"
+ Task Group: "unique_group_per_job" ( 1 create )
  + Task: "unique_task_per_group" ( forces create )
```

Scheduler dry-run

All tasks successfully allocated.

Run Cancel

Figura 9.9 Trabajo de planificación de la interfaz de usuario del servidor Nomad

Ahora haga clic en "Ejecutar" y observe cómo Nomad inicia el proceso de intentar ejecutar esta imagen de contenedor. Si se realiza correctamente, verá el mismo resultado que en la figura 9.10.

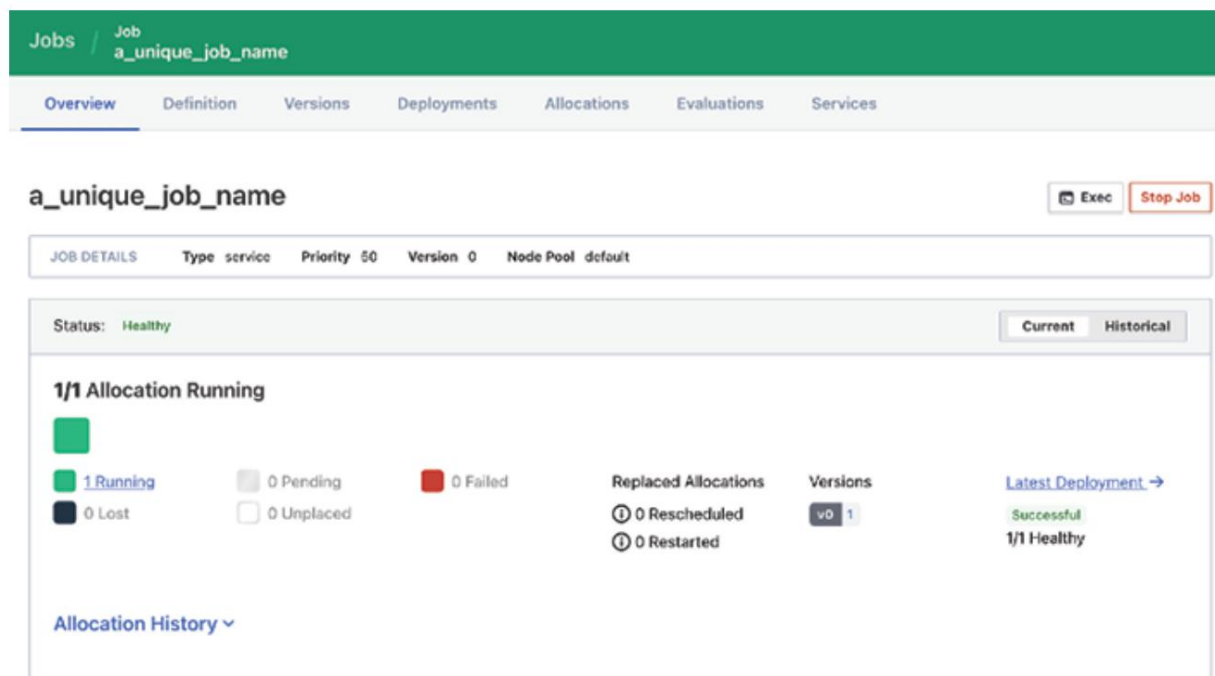


Figura 9.10 Detalle del trabajo de la interfaz de usuario del servidor Nomad

Aprender los detalles de la interfaz de usuario del servidor Nomad está fuera del alcance de este libro, pero te animo a que lo explores.

La interfaz de usuario del servidor Nomad es una excelente manera de iterar rápidamente sobre los trabajos y aprender más sobre el clúster Nomad.

Ahora que tenemos un trabajo en ejecución, eliminémoslo por completo de la interfaz de usuario de Nomad. Los pasos son los siguientes:

1. Vaya a la interfaz de usuario de su servidor Nomad en `http://<nomad-server-public-ip-address>:4646/ui/`.
2. Haga clic en la pestaña Trabajos.
3. Busque el trabajo que desea detener (es decir, `a_unique_job_name`).
4. Haga clic en Detener trabajo y haga clic en Sí, detener trabajo para confirmar.
5. Haga clic en Purgar trabajo para eliminar el trabajo de Nomad servidor.

Si está interesado en el equivalente de este proceso en línea de comandos, son los siguientes pasos:

1. Conéctese por SSH a su servidor Nomad.
2. Cree un archivo HCL de Nomad, como `nomad-job.hcl`, con el contenido de su definición de trabajo (consulte el listado 9.8).
3. Ejecute el plan de trabajo nómada `nomad-job.hcl` para ver el plan.
4. Ejecute `nomad job run nomad-job.hcl` para ejecutar el trabajo.
5. Ejecute `nomad job status <job-name>` para ver el estado del trabajo.
trabajo.
6. Ejecute `nomad job stop <job-name>` para detener el trabajo; incluya el indicador `-purge` para eliminarlo completamente de Nomad. Si no lo purga, puede ejecutar `nomad job start <job-name>` para reiniciarlo.
7. Repita este proceso para cada trabajo que desee ejecutar o administrar.

Crear y eliminar trabajos es sencillo y es una buena forma de practicar.

Ahora, creemos otro trabajo con un contenedor adicional y dos réplicas de cada contenedor. La siguiente lista muestra la nueva definición del trabajo.

Listado 9.9 Definición de trabajo con dos tareas para dos contenedores

```
trabajo "web"
  { grupo "apps" { recuento
    = 1 tarea
    "tarea_única_por_grupo" { controlador =
      configuración "docker"
      { imagen =
        "codingforentrepreneurs/rk8s-py"
      }
    }
  }
  tarea "js-app"
    { controlador = "docker"
      config
        { imagen = "codingforentrepreneurs/rk8s-js"
        }
    }
  }
}
```

Como podemos ver, tenemos dos tareas dentro de un grupo. Este grupo también tiene un recuento de 1, que es el valor predeterminado. Lo incluí aquí para que podamos practicar la modificación de un trabajo preexistente.

Cambiamos el valor a count = 2 y veamos qué sucede.

Estos son los pasos para hacerlo:

1. Abra su servidor Nomad en <http://<nomad-server-public-dirección IP>:4646/ui/>.
2. Haga clic en la pestaña Trabajos.
3. Seleccione el trabajo que desea modificar (por ejemplo, web).
4. Busque la pestaña Definición.
5. Después de hacer clic en Definición , debería ver el trabajo anterior. definición.
6. Haga clic en Editar.
7. Edite la definición como considere oportuno (por ejemplo, count = 2).
8. Haga clic en Plan.

9. Haga clic en Ejecutar.

Tras unos instantes, el plan modificado debería ejecutarse correctamente.

Ahora, nuestros dos contenedores deberían ejecutarse dos veces

cada uno en nuestras instancias cliente de Nomad.

Para verificar esto, me conecté por SSH a mis dos clientes Nomad, ejecuté `docker ps` y encontré el resultado que se muestra en la figura 9.11.

```
root@localhost:~# docker ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATED
TED            STATUS                                PORTS                                NAMES
4e4a464be16d   codingforentrepreneurs/rk8s-js     "docker-entrypoint.s..."           1 se
cond ago      Up Less than a second              js-app-8cad8477-21bd-5e4c-b2cf
-612b2d95ae3e
6a5588f760ce   codingforentrepreneurs/rk8s-py     "./entrypoint.sh"                   20 s
econds ago    Up 19 seconds                      unique_task_per_group-8cad8477
-21bd-5e4c-b2cf-612b2d95ae3e

root@localhost:~# docker ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATED
STATUS                                PORTS                                NAMES
369b6765f65f   codingforentrepreneurs/rk8s-py     "./entrypoint.sh"                   35 seconds
ago          Up 34 seconds                      unique_task_per_group-2e06adfe-2e17-d648-4a44-5827
ebca3e30
0c4b6d844c22   codingforentrepreneurs/rk8s-js     "docker-entrypoint.s..."           38 seconds
ago          Up 37 seconds                      js-app-2e06adfe-2e17-d648-4a44-5827ebca3e30
```

Figura 9.11 Nomad Client Docker PS

Al añadir "count = 2" a nuestro grupo, Nomad debía ejecutar cada tarea (contenedor) dos veces en nuestro clúster, de modo que hubiera un total de cuatro instancias de contenedor en ejecución. En este caso, Nomad descubrió que la forma más eficaz de hacerlo era distribuir una imagen de cada contenedor a ambos clientes de Nomad. Esto no siempre es así, pero sí lo es en este ejemplo.

Un componente importante que falta en nuestras tareas es la asignación de puertos que queremos que tengan nuestros contenedores. Para ello, podemos introducir un nuevo bloque en nuestra definición de grupo, «red», como se muestra en la siguiente lista.

Listado 9.10 Trabajo Nomad con mapeo de puertos de red

```
trabajo "web"  
  { grupo "aplicaciones" {  
    ...  
    red { puerto  
      "http-py" { a = 8080  
  
      }  
      puerto "http-js" { a = 3000  
  
    }  
  }  
}
```

Al usar el bloque de red dentro de nuestro grupo, cada tarea dentro de ese grupo accede a esos valores. Veamos esto en acción actualizando nuestras tareas con la asignación de puertos correcta, como se muestra en la siguiente lista.

Listado 9.11 Definición de trabajo Nomad con dos tareas para dos contenedores con mapeo de puerto de red

```
trabajo "web"
{ grupo "aplicaciones" {
  ...
  tarea "py-app" {
    ...

    configuración { imagen = "codingforentrepreneurs/
      rk8s-py" puertos = ["http-py"]
    }

  } tarea "js-app" {
    ...

    configuración { imagen = "codingforentrepreneurs/
      rk8s-js" puertos = ["http-js"]
    }
  }
}
}
```

Como siempre, mapear correctamente los puertos internos de los contenedores es fundamental para su funcionamiento eficaz, pero esto plantea una nueva pregunta: ¿Cómo podemos acceder a estos contenedores? Sí, podríamos ejecutar sesiones SSH en cada cliente Nomad para acceder a los contenedores, pero queremos poder acceder a ellos públicamente.

Para exponer nuestras tareas (contenedores) a internet con Nomad y nuestra configuración actual, tenemos dos opciones: un servicio a nivel de grupo o un servicio a nivel de tarea. Para nuestras tareas, debemos usar el servicio a nivel de tarea debido a nuestros valores de puerto.

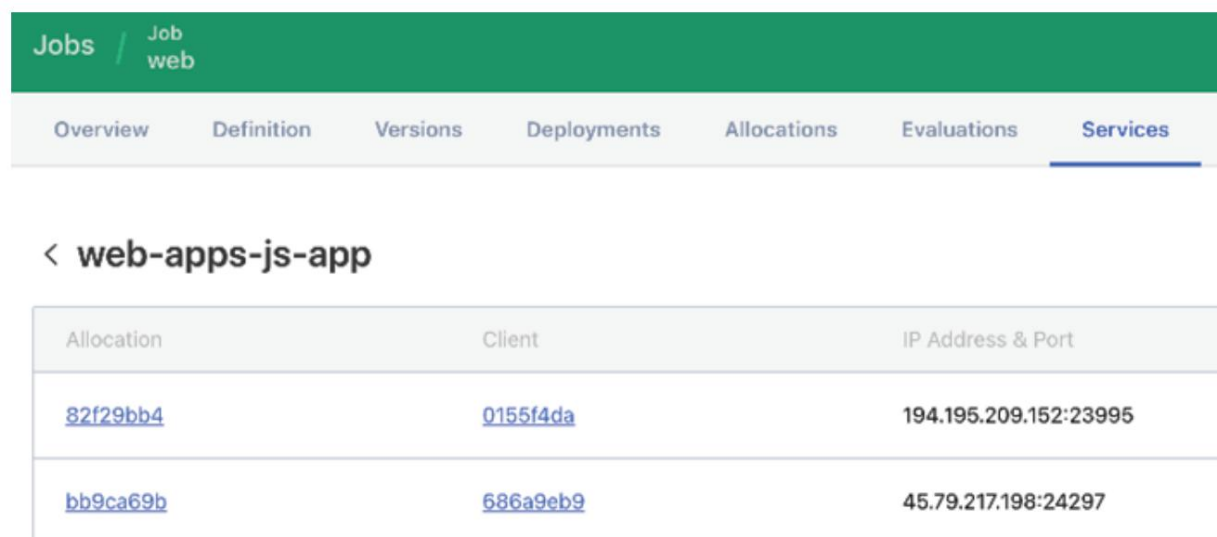
Si nuestros valores de puerto fueran los mismos para cada servicio, entonces podríamos usar un servicio a nivel de grupo.

Un servicio asignará automáticamente un puerto (al azar) que se asigna al puerto del contenedor (p. ej., http-py y http-js) asignado a cada instancia de la tarea en ejecución. Esto significa que si tenemos dos instancias de la tarea `unique_task_per_group`, tendremos dos puertos asignados a esa tarea. Lo mismo ocurre con la tarea `js-app`. Para crear un servicio, necesitamos agregar un nuevo bloque a nuestra definición de tarea, "service", como se muestra en el listado 9.12.

Listado 9.12 Tareas nómadas con definiciones de servicio

```
trabajo "web"
{ grupo "aplicaciones" {
  ...
  tarea "py-app" {
    ...
    servicio
      { puerto = "http-py"
        proveedor = "nomad"
      }
  }
  tarea "js-app" {
    ...
    servicio
      { puerto = "http-js"
        proveedor = "nomad"
      }
  }
}
}
```

Una vez actualizada la definición del trabajo, vaya a la pestaña "Servicios" de este trabajo y verá dos servicios nuevos con dos asignaciones. Haga clic en cualquiera de los servicios y verá el puerto y la dirección IP expuestos de cada servicio y tarea, como se muestra en la figura 9.12.



Jobs / Job web		
Overview	Definition	Versions
Deployments	Allocations	Evaluations
Services		
< web-apps-js-app		
Allocation	Client	IP Address & Port
82f29bb4	0155f4da	194.195.209.152:23995
bb9ca69b	686a9eb9	45.79.217.198:24297

Figura 9.12 Detalles del servicio de la interfaz de usuario del servidor Nomad

En esta pantalla vemos los valores

- 194.195.209.152:23995
- 45.79.217.198:24297

Estas son las direcciones IP públicas de nuestros clientes Nomad y los puertos que Nomad asignó al servicio de cada tarea. Estos puertos se asignan dinámicamente para garantizar que las definiciones de nuestros trabajos no entren en conflicto entre sí ni con el número de grupos (p. ej., número = 2). Si aumenta el número de grupos o añade más grupos o trabajos, Nomad es lo suficientemente inteligente como para asignar direcciones de puerto disponibles para cualquier servicio.

Si utiliza otras herramientas de HashiCorp, como Terraform o Vault, es lógico usar Nomad antes de empezar a usar Kubernetes. Nomad es una herramienta mucho más sencilla de usar en general y ofrece mejor compatibilidad entre regiones y centros de datos que la configuración predeterminada de Kubernetes. La interfaz de usuario de Nomad es, en mi opinión, mucho más fácil de usar que el panel de control de Kubernetes. Usar Kubernetes administrado, como se mencionó con Docker Swarm, permite aprovisionar la nube rápidamente.

Recursos como balanceadores de carga, volúmenes de almacenamiento persistente y direcciones IP estáticas. Nomad no incorpora esta función, pero si se combina con la herramienta Terraform de HashiCorp, sería más que posible.

Hasta ahora, hemos cubierto los fundamentos de Nomad para la gestión e implementación de varios contenedores en varias máquinas. Para aprovechar al máximo el potencial de Nomad, necesitamos integrarlo estrechamente con otro servicio de HashiCorp llamado Consul. Consul permite a Nomad realizar descubrimiento de servicios de forma similar a las funciones integradas de Kubernetes y Docker Swarm. La configuración de Consul queda fuera del alcance de este libro, pero te animo a explorarla por tu cuenta.

Resumen

- Docker Swarm es muy parecido a Docker Compose, pero se ejecuta en múltiples máquinas virtuales.
- Docker Swarm y Docker Compose son casi intercambiables, pero Docker Swarm requiere más configuración.
- Docker Swarm tiene un descubrimiento de servicios incorporado, lo que facilita la ejecución de múltiples contenedores en múltiples máquinas virtuales.
- HashiCorp Nomad tiene una interfaz de usuario intuitiva que facilita la gestión del clúster, los trabajos y los clientes.
- HashiCorp tiene un conjunto de herramientas que utilizan HCL, lo que convierte a Nomad en una excelente opción para aquellos que ya utilizan las herramientas de HashiCorp.
- La asignación dinámica de IP de Nomad permite que cada instancia de contenedor en ejecución sea accesible públicamente en segundos.

Apéndice A Instalación Python en macOS y Ventanas

En el capítulo 2, creamos una aplicación web en Python. Esta aplicación se transformará a lo largo del libro para su implementación en Kubernetes.

Antes de empezar a crear nuestra aplicación, necesitamos instalar Python en nuestra máquina. Este apéndice nos ayudará a hacerlo en macOS y Windows. En el capítulo 2, explicamos cómo instalar Python en Linux. Para consultar mi guía más actualizada sobre la instalación de Python, visite mi blog en <https://cfe.sh/blog/>. Si le interesa la documentación oficial para instalar Python, visite <https://docs.python.org/>. —

A.1 Instalación de Python para macOS

Con macOS, podrías verte tentado a usar herramientas de instalación de terceros como Homebrew o MacPorts. Aunque estas herramientas son excelentes, recomiendo usar el instalador oficial de Python del sitio web de Python, ya que nos asegurará tener la versión exacta que necesitamos, cuando la necesitamos.

Podemos tener múltiples versiones de Python instaladas sin ningún problema.

Para empezar, necesitamos determinar dos cosas:

- Versión de Python
- Procesador: Intel o Apple

Versión A.2 de Python

Python 3 es la versión actual de Python. La versión principal actual (enero de 2023) es Python 3.11.

A lo largo de los años, Python ha mejorado, pero la sintaxis sigue siendo en gran medida la misma.

Para este libro, recomiendo Python 3.10, con una versión mínima de Python 3.8.

A.2.1 Procesador: Intel o Apple

Para versiones más recientes de Python, puede pasar a la siguiente sección. Para versiones anteriores de Python, necesita saber con qué tipo de procesador está trabajando. Para determinar este valor, haga lo siguiente:

1. Haga clic en el ícono de Apple en la esquina superior izquierda.
2. Seleccione Acerca de esta Mac.
3. Busca Chip. Será Apple o Intel.

Si tiene un procesador Apple (p. ej., M1, M2, M3, etc.), debe usar el instalador universal 2 de macOS de 64 bits. Si tiene un procesador Intel, puede usar el instalador universal 2 de macOS de 64 bits o el instalador exclusivo para Intel de macOS de 64 bits. Es posible que versiones anteriores de Python no sean compatibles con su equipo actual.

A.2.2 Descargar Python

Ahora que sabemos qué versión de Python queremos, 3.10, y qué procesador tenemos, Intel o Apple, finalmente podemos Descargar el instalador correcto:

1. Visita la versión oficial de Python para macOS (<https://www.python.org/downloads/macos/>). Lanzamientos
2. En Versiones estables, busque la última versión de Python 3.10 que tiene instalador universal2 para macOS de 64 bits como opción. Al momento de escribir esto, la versión es 3.10.9. La voluntad se parecerá enlace <https://www.python.org/ftp/python/3.10.9/python-3.10.9-macos11.pkg>.
3. Haga clic en el enlace para descargar el instalador.
4. Una vez completada la descarga, navegue hasta el Carpeta de descargas en su máquina.
5. Busque el instalador y ábralo. Debe llamarse algo como python-3.10.9-macos11.pkg.
6. Ejecute todas las opciones de instalación predeterminadas para el instalador. La personalización de la instalación queda fuera del alcance de este libro.

Con el instalador completo, podemos verificar la instalación. Trabajó en nuestra aplicación de línea de comandos llamada Terminal (ubicado en Aplicaciones / Utilidades), como se muestra en la Siguiente listado. Abra la terminal y escriba lo siguiente. comandos,

Listado A.1 Aplicación de terminal

```
Python 3.10 -V#1
python3 -V#2
```

#1 Dado que instalamos Python 3.10, este comando debería funcionar. Si Si instalamos Python 3.9, escribiríamos python3.9 -V.
#2 Debido a que acabamos de instalar una nueva versión de Python, python3 Probablemente la versión predeterminada sea esa. Si instalamos Python 3.9, escribiríamos python3 -V, que devolvería Python 3.9.7.

CREANDO PYTHON DESDE LA FUENTE

Construir Python a partir del código fuente es una excelente manera de compilar la versión exacta de Python que desea en su máquina, pero está fuera del alcance de este libro.

La idea general es que descargues el código fuente de casi cualquiera de las versiones de macOS (<https://www.python.org/downloads/macos/>) y ejecutes varios comandos de terminal para compilarlo e instalarlo en tu máquina.

La gran mayoría de los lectores omitirán la compilación desde el código fuente porque podría complicar demasiado el proceso de instalación. Si eres de los que compilan software desde el código fuente, probablemente ni siquiera hayas abierto este apéndice.

Ahora que tenemos Python instalado, podemos pasar a la creación de un entorno virtual en la sección 3.5.2.

A.3 Instalación de Python para Windows

La mejor manera de instalar Python en Windows es hacerlo directamente desde el sitio web de Python (<http://python.org/>). La razón es que el sitio web de Python nos garantizará tener la versión exacta de Python que necesitamos, cuando la necesitamos. Podemos tener varias versiones de Python instaladas sin problemas.

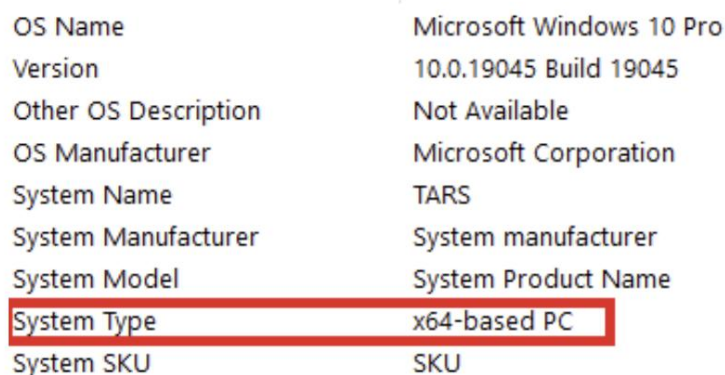
Esta sección cubrirá algunos aspectos de la instalación de Python en Windows. Si le interesa la versión oficial

documentación para instalar Python, visite <https://docs.python.org/3/using/windows.html>.

A.3.1 Verificar la información del sistema

Antes de instalar Python, debemos verificar si nuestro sistema Windows es de 64 o 32 bits. Si tiene una máquina más reciente, probablemente usará un sistema de 64 bits. Así es como podemos comprobarlo:

1. Abra el menú Inicio.
2. Escriba Información del sistema y presione Entrar.
3. Busque la sección Tipo de sistema (ver figura A.1).
4. Si ves una PC basada en x64, tienes un sistema de 64 bits. Si ves una PC basada en x86, tienes un sistema de 32 bits.



OS Name	Microsoft Windows 10 Pro
Version	10.0.19045 Build 19045
Other OS Description	Not Available
OS Manufacturer	Microsoft Corporation
System Name	TARS
System Manufacturer	System manufacturer
System Model	System Product Name
System Type	x64-based PC
System SKU	SKU

Figura A.1 Información del sistema de Windows

A.3.2 Instalar Python

Ahora que sabemos si tenemos un sistema de 64 o 32 bits, podemos instalar Python:

1. Vaya a <https://www.python.org/downloads/windows>.
2. Elija Python 3.10.X (reemplace X con el número más alto disponible).

3. Para la versión de Python seleccionada, verifique la lista de Windows Installer que coincida con su tipo de sistema (por ejemplo, 64 bits o 32 bits).
4. Seleccione descargar la versión de 64 bits o 32 bits para que coincida su sistema (ver figura A.2.).
5. Haga clic en el enlace para descargar el instalador.



Figura A.2 Instalador de Python para Windows de 64 bits en Python 3.10.9

Después de descargar el instalador, ábralo, ejecútelo y haga lo siguiente:

1. Seleccione Agregar Python 3.10 a PATH solo si esta es la primera versión de Python que ha instalado.
2. Seleccione Personalizar instalación y luego Siguiente.
3. Seleccione Instalar pip y luego Siguiente.
4. En Opciones avanzadas, utilice al menos lo siguiente Configuración (ver figura A.3): Instalar
 - para todos los usuarios

- Agregar Python a las variables de entorno
- Crear accesos directos para aplicaciones instaladas
- Precompilar la biblioteca estándar Personalizar la ubicación de instalación para que sea C:\Python310, donde 310 representa 3.10 ya que los puntos (.) no pueden estar en un nombre de carpeta en este caso. Si está instalando Python 3.9, deberá utilizar C:\Python39 en su lugar.

5. Haga clic en Instalar

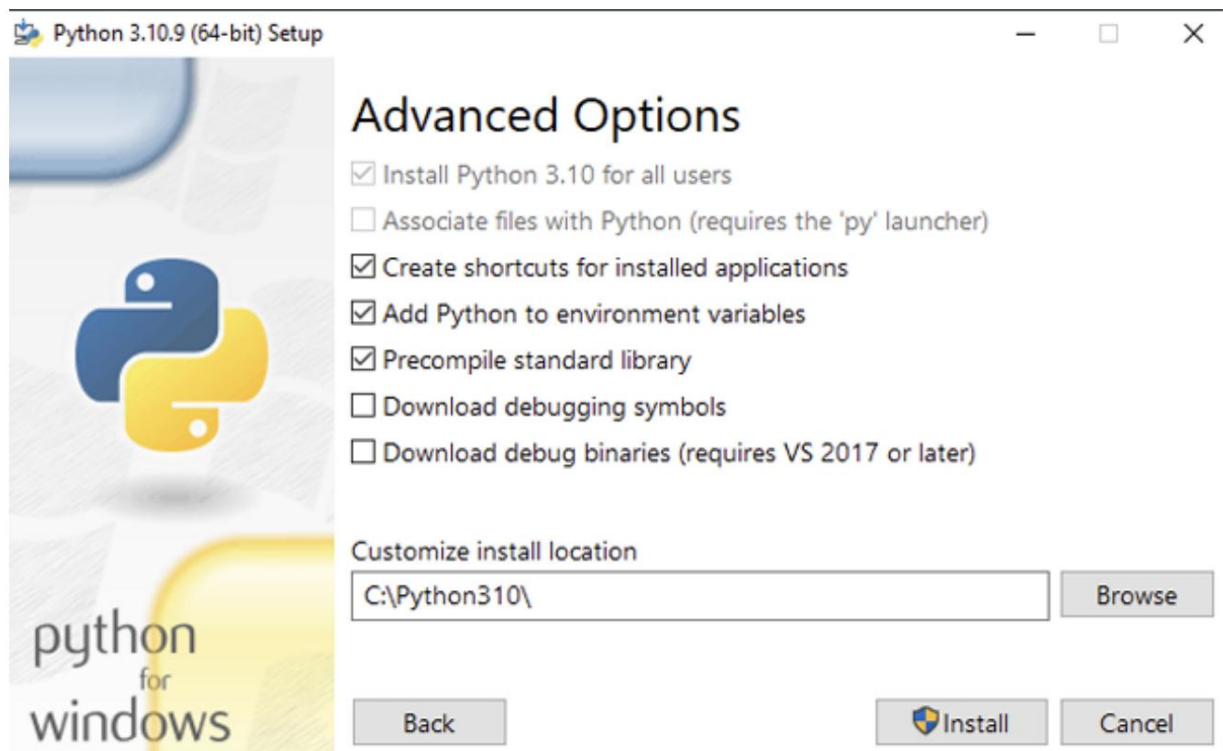


Figura A.3 Opciones de instalación avanzada de Python

Seleccionar "Añadir Python 3.10 a PATH" también es opcional, pero le permitirá ejecutar Python desde la línea de comandos sin especificar la ruta completa del ejecutable. Abra el símbolo del sistema y escriba los siguientes comandos en la lista que aparece a continuación.

Listado A.2 Uso del acceso directo Agregar Python 3.10 a PATH

Python -V#1

#1 Esto debería generar su versión más reciente de Python 3 instalada que marcó la opción Agregar Python 3.10 a PATH.

Usar la instalación personalizada para C:\Python310 es opcional, pero se recomienda, ya que así podrá tener fácilmente varias versiones de Python en su equipo. Por ejemplo, si tiene Python 3.10 y Python 3.9 instalados, puede ejecutar cualquiera de las dos abriendo el símbolo del sistema y escribiendo los siguientes comandos.

Listado A.3 Python 3.10 y Python 3.9 instalados

```
C:\Python39\python.exe -VC:  
\Python310\python.exe -V
```

Ahora que tiene Python instalado en su máquina Windows, es momento de crear un entorno virtual en la sección 3.5.2.

A.4 Instalación de Python para Linux

Al usar Linux, existen muchas distribuciones diferentes. Para instalar Python, asumiré que usa un sistema Linux basado en Debian, concretamente Ubuntu. Ubuntu tiene Python 3 instalado por defecto. Abra la terminal y escriba los comandos de la siguiente lista.

Listado A.4 Ubuntu con Python 3

```
python3 -V
```

Si ve Python 3.8 o una versión posterior, puede pasar a la sección 3.5.2. Si ve Python 3.8 o una versión anterior (incluyendo Python 3.6 o Python 2.7), deberá instalar Python 3.8 o una versión posterior. Si necesita una guía más detallada sobre la instalación de Python en Linux, consulte el capítulo 6, ya que instalamos Python 3 en Linux para nuestro entorno de producción inicial de nuestra aplicación Python.

A.5 Entornos virtuales y Python

Los entornos virtuales son una forma de ayudar a aislar los proyectos de Python entre sí y del sistema en general.

Los entornos virtuales se pueden crear, activar, desactivar y destruir fácilmente. Considérelos efímeros, ya que nunca los guardaremos en nuestros repositorios de Git, imágenes de Docker ni clústeres de Kubernetes; siempre los recreamos cuando nuestro código se mueve.

Para crear un entorno virtual, usaremos un módulo integrado de Python llamado venv. En mi opinión, venv es la forma más sostenible de gestionar paquetes de Python de terceros, como FastAPI, Django, solicitudes y muchos otros. Existen muchas otras maneras de gestionar paquetes de Python, pero quedan fuera del alcance de este libro.

A.5.1 Creación de una carpeta de proyecto de prueba

Creemos una única ubicación para albergar nuestro proyecto de prueba, como se muestra en los siguientes listados.

Listado A.5 Creación de una carpeta de proyecto en macOS/Linux

```
mkdir -p ~/Dev/k8s-appendix-test#1 cd ~/dev/  
k8s-appendix-test#2
```

#1 La mayoría de mis proyectos de software residen en el directorio Dev, en la carpeta raíz del usuario (~). k8s-appendix-test es el nombre del directorio donde crearemos nuestro entorno virtual y proyecto. Modifíquelos como considere oportuno.

#2 Navega al director recién creado

Listado A.6 Creación de una carpeta de proyecto en Windows

```
mkdir -p ~\Dev\k8s-appendix-test #1 cd  
~\dev\k8s-appendix-test#2
```

#1 Las rutas de la línea de comandos son ligeramente diferentes para Windows.

#2 La mayoría de mis proyectos de software residen en el directorio ~/Dev. k8s-appendix-test es el nombre del directorio en el que crearemos nuestro entorno virtual y proyecto.

A.5.2 Creación de un entorno virtual

Cree un entorno virtual en el directorio actual con la versión de Python 3 recién instalada. Para ello, usaremos el gestor integrado de Python llamado venv con la opción `-m venv <your-environment-name>`, como se muestra en el listado A.7. Usaremos el nombre venv para nuestro entorno virtual. Puede usar el nombre que desee, pero le recomiendo usar uno de los siguientes: venv, env, _venv o .venv. Si cambia el nombre de venv, muchos aspectos de este libro y los apéndices no funcionarán.

Listado A.7 Creación de un entorno virtual en macOS/Linux

```
cd ~\Dev\k8s-appendix-test python3  
-m venv venv
```

`python -m` (o `python3 -m`) es la forma de llamar a módulos integrados en Python; otra forma popular es usar el módulo `pip` con `python -m pip` (o `python3 -m pip`, dependiendo de su sistema). Esta es la forma preferida de llamar a `venv` o `pip`, ya que garantizará que esté utilizando la versión correcta de Python, como se muestra en la siguiente lista para Windows.

Listado A.8 Creación de un entorno virtual en Windows

```
cd ~\Dev\k8s-appendix-test python  
-m venv venv
```

En este caso, usar `python -m` debería funcionar si instaló Python 3 correctamente en Windows y se agregó Python a su `PATH` (como se mencionó anteriormente). En algunos casos, podría necesitar usar la ruta absoluta al ejecutable de la versión específica de Python en equipos Windows (por ejemplo, `C:\Python310\python.exe -m venv`).

A.5.3 Activación de un entorno virtual

Activar un entorno virtual nos permite usar los paquetes de Python que instalamos en él. Podemos activar un entorno virtual abriendo la terminal y escribiendo los comandos de la siguiente lista.

Listado A.9 Activación de un entorno virtual

```
cd ~/Dev/k8s-appendix-test fuente  
venv/bin/activate
```


El comando `source venv/bin/activate` puede ser nuevo para usted.
Así es como se desglosa:

- El código fuente está integrado en macOS y Linux. Se utiliza para ejecutar un script o programa. `venv` es el nombre del directorio que contiene todos los archivos de nuestro entorno virtual.
- `bin` es el directorio que contiene todos los archivos ejecutables de nuestro entorno virtual. También encontrará una versión de Python aquí, que puede verificar con `venv/bin/python -V`.
- `activate` es el archivo ejecutable que activa nuestro entorno virtual.

Abra PowerShell y escriba los comandos en la siguiente lista.

Listado A.10 fuente `venv/bin/activate`

```
cd ~\Dev\k8s-appendix-test .  
.\venv\Scripts\activate
```

Quizás no sepas usar `.\venv\Scripts\activate`. Aquí te explicamos cómo funciona:

- Ejecute un script en la línea de comando usando un punto (.) al comienzo del comando. `venv` es el nombre del directorio que contiene todos los archivos de nuestro entorno virtual.
- `Scripts` es el directorio que contiene todos los archivos ejecutables de nuestro entorno virtual. También encontrará una versión de Python aquí, que puede verificar con `venv\Scripts\python -V`. `activate` es el archivo ejecutable que activa nuestro entorno virtual.
-

La activación del entorno virtual es probablemente la principal diferencia que encontrarás al usar Python en macOS/Linux y Windows. El resto de los comandos son prácticamente iguales.

A.5.4 Instalación de paquetes de Python

Con nuestro entorno virtual activado, podemos instalar paquetes de Python usando pip abriendo la terminal y escribiendo los comandos como se muestra en el siguiente listado.

Listado A.11 Uso de pip para instalar paquetes de Python

```
$(venv) python -V $  
(venv) python -m pip install fastapi $(venv) pip  
install uvicorn
```

Entendamos mejor qué está pasando aquí:

- `$(venv)` denota que nuestro entorno virtual está activo. Es posible que veas `$(venv)` o `(venv)` o `(venv) >` dependiendo de tu aplicación de línea de comandos.
- ¿Observas que falta el 3 en Python? Esto se debe a que nuestro entorno virtual activado utiliza Python del entorno virtual por defecto. En macOS/Linux, `python3` debería seguir haciendo referencia a Python del entorno virtual activado. `python -m pip install` es la forma preferida de usar pip para instalar paquetes de Python
- de terceros desde <http://pypi.org/>. `pip install` técnicamente funciona, pero no es la forma preferida de instalar paquetes de Python de terceros porque el comando pip podría
- estar apuntando a una versión incorrecta de Python.

A.5.5 Desactivación y reactivación de entornos virtuales

Al cerrar la línea de comandos, el entorno virtual se desactivará por defecto. Si necesita desactivarlo manualmente, puede hacerlo abriendo la terminal y escribiendo los comandos de la siguiente lista.

Listado A.12 Desactivación de un entorno virtual

```
$(venv) desactivar #1
```

#1 Desactivar está disponible en un entorno virtual activado

A menudo es necesario desactivar un entorno virtual cuando necesitas cambiar de proyecto y/o de versión de Python.

Cuando necesite reactivar, es tan sencillo como utilizar uno de los comandos de los siguientes listados.

Listado A.13 Reactivación de un entorno virtual, opción 1

```
cd ~/Dev/k8s-appendix-test#1 fuente  
venv/bin/activate#2
```

#1 Navega al directorio que contiene nuestro entorno virtual

#2 Activa nuestro entorno virtual con el comando de activación original

Listado A.14. Reactivación de un entorno virtual, opción 2

```
cd ~\Dev\k8s-appendix-test#1 .  
\venv\Scripts\activate#2
```

#1 Navega al directorio que contiene nuestro entorno virtual

#2 Activa nuestro entorno virtual con el comando de activación original

Ahora veamos cómo podemos recrear entornos virtuales.

A.5.6 Recreación de entornos virtuales

En este punto, quiero retarte a recrear tu entorno virtual desde cero. Es decir, elimina todo el directorio venv y empieza de nuevo. Uno de los elementos clave que probablemente necesitarás agregar es requirements.txt para realizar un seguimiento de los paquetes de terceros. Usamos requirements.txt con más detalle a partir del capítulo 2.

Apéndice B Instalación de Node.js en macOS y Windows

En el capítulo 2, creamos una aplicación web Node.js. Esta aplicación se transformará a lo largo del libro para su implementación en Kubernetes.

Antes de empezar a crear nuestra aplicación, necesitamos instalar Node.js en nuestro equipo. Este apéndice nos ayudará a hacerlo en macOS y Windows. En el capítulo 3, explicamos cómo instalar Node.js en Linux. Para consultar mi guía más actualizada sobre la instalación de Node.js, visite mi blog en <https://cfe.sh/blog/>. Si le interesa la documentación oficial para instalar Node.js, visite <https://nodejs.org/>.

Para este libro, solo queremos asegurarnos de usar la última versión de soporte a largo plazo (LTS) de Node.js. La forma más sencilla de descargarla es visitando <https://nodejs.org> y siguiendo el proceso de instalación predeterminado. El resto de este apéndice te ayudará a instalar Node.js en macOS y Windows.

B.1 Instalación de Node.js para macOS

Recomiendo instalar Node.js de una de dos maneras:

- Directamente desde [Nodejs.org](https://nodejs.org)

- Uso del administrador de versiones de nodo (nvm; <https://github.com/nvm-sh/nvm>)

Hay otras herramientas que pueden instalar Node.js, pero a menudo resultan más difíciles a largo plazo que las dos opciones anteriores.

A diferencia de Python, no necesitamos saber qué procesador utiliza nuestra computadora Apple, ya que la última versión LTS de Node.js ya es compatible con Apple Silicon (M1, M2, etc.) y procesadores Intel.

Para instalar nvm, utilice el siguiente comando:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.5/install.sh | bash
```

Este comando proviene directamente de la documentación oficial de nvm en <https://github.com/nvm-sh/nvm>.

Tras instalar nvm, abra una nueva sesión de terminal y ejecute el comando `nvm install --lts`. Esto instalará la última versión LTS de Node.js. Una vez finalizado, podrá ejecutar `node --version` y ver la versión de Node.js instalada.

B.2 Instalación de Node.js para Windows

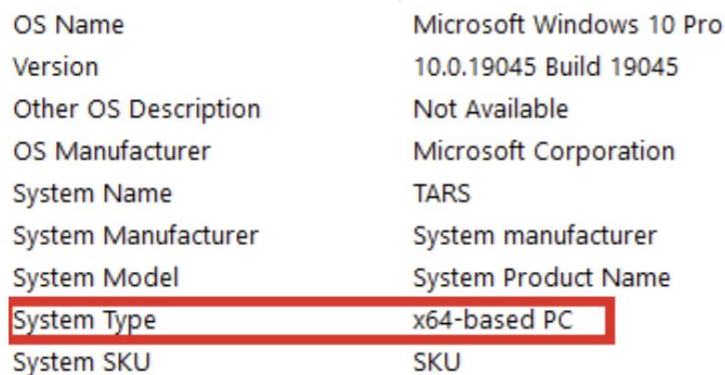
La instalación más eficiente de Node.js en Windows es directamente desde el sitio web de Node.js en <http://nodejs.org>. Node.js ha simplificado la descarga de todo tipo de versiones de nodos, incluida la última LTS, que se recomienda para la mayoría de usuarios.

La versión LTS actual es 18.13.0, pero la gran mayoría del código Node.js que crearemos en este libro debería ser compatible con versiones anteriores y futuras. Antes de instalar Node.js, verifiquemos la información de nuestro sistema de la misma manera que hicimos con Python.

B.2.1 Verificación de la información del sistema

Antes de instalar Python, debemos verificar si nuestro sistema Windows es de 64 o 32 bits. Si tiene una máquina más reciente, probablemente usará un sistema de 64 bits. Así es como podemos comprobarlo:

1. Abra el menú Inicio.
2. Escriba Información del sistema y presione Entrar.
3. Busque la sección Tipo de sistema (ver figura B.1).
4. Si ves una PC basada en x64, tienes un sistema de 64 bits. Si ves una PC basada en x86, tienes un sistema de 32 bits.



OS Name	Microsoft Windows 10 Pro
Version	10.0.19045 Build 19045
Other OS Description	Not Available
OS Manufacturer	Microsoft Corporation
System Name	TARS
System Manufacturer	System manufacturer
System Model	System Product Name
System Type	x64-based PC
System SKU	SKU

Figura B.1 Información del sistema de Windows

Ahora que sabemos en qué tipo de sistema estamos, 64 bits o 32 bits, podemos instalar Node.js.

B.2.2 Descarga e instalación de Node.js

Para la gran mayoría de usuarios, la versión recomendada de Node.js es la LTS, ya que seguirá siendo la versión con mayor soporte y estabilidad durante muchos años. La versión LTS actual de Node.js es la 18.13.0.

Si tiene una máquina de 64 bits, puede instalar Node.js con lo siguiente:

1. Abra un navegador web y navegue a <http://nodejs.org/>.
2. Haga clic en el botón Descargar para la versión LTS de Node.js.
3. Haga clic para descargar el archivo Dfile a su computadora.
4. Haga doble clic en el archivo para iniciar la instalación.
5. Siga las instrucciones predeterminadas para instalar Node.js.

Si tiene una máquina de 32 bits, puede instalar Node.js con lo siguiente:

1. Abra un navegador web y navegue a <http://nodejs.org/>.
2. Haga clic en el botón Descargar para la versión LTS de Node.js.
3. Haga clic en Otras descargas.
4. En Windows Installer (.msi), haga clic en el enlace de 32 bits.
5. Haga clic para descargar el archivo a su computadora.
6. Después de la descarga, haga doble clic para ejecutar el instalador.
7. Siga las instrucciones predeterminadas para instalar Node.js.

Si necesita instalar varias versiones de Node.js en su equipo Windows, considere usar la herramienta nvm-windows , cuyas instrucciones se encuentran en <https://github.com/coreybutler/nvm-windows>. Dado que nvm solo es compatible con macOS/Linux, nvm-windows es una alternativa eficaz que le ofrece varias de las mismas funciones. La instalación de nvm-windows queda fuera del contexto de este libro.

Apéndice C Configuración

Claves SSH para acceder al servidor sin contraseña

En este apéndice, aprenderemos a usar un protocolo de conexión llamado Secure Shell (SSH) para que nuestra máquina local pueda conectarse a una computadora remota. En este caso, una computadora remota puede ser cualquier cosa, desde otra computadora en casa hasta una máquina virtual alquilada a un proveedor de nube.

SSH nos da acceso a la línea de comandos de otro ordenador, lo que nos permite gestionar la línea de comandos de otro ordenador de forma similar a la de nuestro ordenador local. A menudo usamos SSH para acceder a sistemas operativos Linux, lo que significa que los comandos de nuestro ordenador local pueden ser ligeramente diferentes a los de Linux. Esto es especialmente cierto para los usuarios de Windows, ya que Linux y macOS comparten muchas características en sus sistemas operativos.

Para conectarse a una computadora remota con SSH, a menudo necesitará uno de los siguientes:

- Un nombre de usuario y contraseña válidos
- Una clave SSH pública instalada

Dada la naturaleza de este apéndice, nos prepararemos para el uso de claves SSH, ya que pueden permitir el ingreso sin contraseña a una computadora remota.

Las claves SSH se componen de dos componentes:

- Clave pública: una clave compartible que ayuda a verificar nuestra identidad
- Clave privada: una clave secreta, no compartible, que verifica que nuestra clave pública es, de hecho, nuestra.

Podemos instalar nuestra clave pública manualmente en una gran cantidad de servidores, usar `ssh-copy-id` (hablaremos más sobre esto en breve) o contratar a un proveedor de nube para que lo haga por nosotros. En el capítulo 3, mostramos un ejemplo de cómo el proveedor de nube Akamai Connected Cloud (anteriormente Linode) instalará automáticamente nuestra clave SSH pública en cualquier máquina virtual que creemos.

Si su clave SSH falla, normalmente necesitará un nombre de usuario y una contraseña (como ocurre con muchos programas) para iniciar sesión en el dispositivo remoto. Este nombre de usuario y esta contraseña pueden usarse en lugar de claves SSH válidas o junto con ellas. Si tiene un nombre de usuario y una contraseña, pero su clave SSH pública no está instalada, puede usar el comando `ssh-copy-id` para instalarla en el dispositivo remoto. Encontrará más información sobre `ssh-copy-id` en la sección 3.4.2.

C.1 Generación de claves SSH multiplataforma

Dado que SSH se usa tan ampliamente, crear claves SSH es algo habitual. Te animo a que te acostumbres a hacerlo y, aún más importante, a que te acostumbres a eliminar estas claves y a empezar desde cero siempre que sea posible.

Estos son los comandos principales que probablemente tendrá su línea de comandos:

- `ssh`—Se utiliza para todas las conexiones SSH.

- `ssh-keygen`: una forma estandarizada y multiplataforma de generar una clave SSH pública y privada.
- `ssh-copy-id`: una forma estandarizada de copiar claves SSH públicas a una computadora a la que tiene acceso privado.

Verifiquemos que tenemos acceso para usar estos comandos con

```
comando -v ssh#1 comando  
-v ssh-keygen #2 comando -v ssh-copy-id
```

El comando #1 `-v <algún comando cli>` es una forma de verificar si un comando existe o no.

#2 La mayoría de las líneas de comandos modernas tienen SSH disponible por defecto. Algunas máquinas Windows más antiguas requieren la descarga de un programa llamado PuTTY para usar SSH y comandos relacionados.

C.1.1 Navegación a su directorio raíz SSH

De forma predeterminada, nuestras claves SSH y configuración se almacenarán en el directorio `~/.ssh`. Verifiquemos que `~/.ssh` exista y naveguemos hasta él:

```
mkdir -p ~/.ssh cd ~/.ssh#1
```

#1 Trabajar en el directorio `.ssh` de nuestro usuario hace que el resto de esta guía sea simple.

C.1.2 Generación manual de claves SSH con `ssh-keygen`

Para crear un par de claves SSH, usamos el comando `ssh-keygen` con opciones estándar como esta:

```
ssh-keygen -t rsa -b 4096
```

Deberías ver algo como:

Generando par de claves RSA públicas/privadas.

Introduzca el archivo en el que guardar la clave (/Users/justin/.ssh/id_rsa):

Si en algún momento necesita detener este proceso de creación, simplemente ejecute Control+C. Ingrese el nombre id_rsa_rk8s y presione Enter. Deberías ver:

Ingrese el archivo en el que guardar la clave (/Users/justin/.ssh/id_rsa): id_rsa_rk8s Ingrese la frase de contraseña
(vacíe si no hay frase de contraseña):

Para mayor seguridad, puede agregar una frase de contraseña a la clave SSH. Piense en esto como una contraseña para la clave SSH privada.

Para simplificar, lo dejaremos en blanco. Presiona Enter/Return y verás

Ingrese la misma contraseña nuevamente:

Presiona Enter/Return nuevamente y verás

```

Su identificación ha sido guardada en /Users/justin/.ssh/id_r
sa_rk8s
Su clave pública se ha guardado en /Users/justin/.ssh/id_rsa_r
k8s.pub
La huella clave es:
SHA256:..algunaclave.. justin@justins-laptop.lan
La imagen aleatoria de la clave es:
+---[RSA 4096]-----+
|=. .= .. +**o.|
|.. .. +. |   ==
|p . oE | B oo |p. .
+ = |
|.. ..=. S |
|. o o+ o |
|.. ..o= |
|. .=.+o |
||      .oO=.
+----[SHA256]-----+

```

Es importante tener en cuenta que puedes ejecutar ssh-keygen en cualquier lugar en su computadora, y creará y almacenará la clave/par en el directorio de trabajo a menos que especifique una ruta exacta a una ubicación de almacenamiento. Usamos ~/.ssh para mitigar los problemas al generar nuestras claves.

C.1.3 Automatización de ssh-keygen

Los siguientes comandos crearán un nuevo par de claves SSH sin una frase de contraseña usando la ruta de archivo ~/.ssh/id_rsa_k8s como hicimos antes. La única diferencia es que estos comandos no requieren ninguna entrada del usuario:

- macOS/Linux: `ssh-keygen -t rsa -b 4096 -C ~/.ssh/id_rsa_k8s -q -N`
- Windows: `ssh-keygen -b 2048 -t rsa -f ~/.ssh/id_rsa_k8s -q -N`

Ahora que hemos creado las claves, verifiquemos que se crearon en el lugar correcto.

C.1.4 Verificación de las claves

Dependiendo de las rutas de archivo que hayas usado, tus claves podrían estar en una ubicación diferente. Verifiquemos que las hayamos almacenado en el directorio SSH estándar de nuestro usuario:

```
ls ~/.ssh
```

Es posible que veas algo como esto:

```
configuración
hosts_conocidos
id_rsa id_rsa.pub
id_rsa_rk8s id_rsa_rk8s.pub
```

O, al menos, ahora lo veréis.

```
id_rsa_rk8s
id_rsa_rk8s.pub
```

Estas son las dos claves que forman su par de claves SSH.

La clave pública: Compartiremos nuestra clave pública (id_rsa_rk8s.pub) con nuestros hosts remotos, Linode, GitHub o cualquier otro lugar donde necesitemos usar SSH para una autenticación más rápida. Debemos mantener la clave privada (id_rsa_rk8s) segura para evitar ataques no deseados. acceso.

C.2 Sobrescritura de claves SSH

La razón por la que optamos por utilizar una nueva clave SSH para este libro es para garantizar que no eliminemos accidentalmente el acceso a otros servidores y/o expongamos las claves incorrectas en los lugares equivocados. Lo que estás aprendiendo es que es increíblemente común eliminar o sobrescribir tus claves SSH antiguas. Practiquemos. ahora:

```
cd ~/.ssh  
ssh-keygen -t rsa -b 4096
```

Deberías ver algo como

```
Generando par de claves RSA públicas/privadas.  
Introduzca el archivo en el que guardar la clave (/Users/justin/.ssh/id_rs a):
```

Nuevamente, ingrese el nombre id_rsa_rk8s y presione Enter. Debería ver

```
Introduzca el archivo en el que guardar la clave (/Users/justin/.ssh/id_rs a):  
id_rsa_rk8s  
id_rsa_rk8s ya existe.  
¿Sobrescribir (y/n)?
```

Siéntete libre de responder sí o no para experimentar lo que sucede. No hay ninguna respuesta incorrecta. También puedes eliminar el par de claves SSH con

```
rm id_rsa_rk8s rm  
id_rsa_rk8s.pub
```

Si comienzas a pensar en las claves SSH como en realidad dos claves, estás en el camino correcto.

C.3 Copia de la clave pública

Ahora copiaremos la clave SSH pública del directorio ~/.ssh/. Es importante copiar el contenido de un archivo que tenga la extensión ".pub" al final del nombre (por ejemplo, id_rsa_rk8s.pub). La clave pública se parece a esta cadena:

```
ssh-rsa <valor codificado> justin@justins-laptop.lan#1
```

#1 Se han omitido datos irrelevantes, pero el valor completo (incluido <valor codificado>) debe estar en una línea.

La clave privada se parece a esta cadena:

```
----intento
-----INICIO CLAVE PRIVADA OPENSsh-----
b3BIbnNzaC1rZXktdjEAAAABAG5vbmUAAAABbm9uZQAAAAAAAAABAAACFwAAA
Adzc2gtcn #1
...
...
...
-----FIN DE LA CLAVE PRIVADA OPENSsh-----
```

#1 Se han omitido datos irrelevantes. Este archivo tiene unas 50 líneas.

Un atajo para copiar la clave pública es ejecutar el comando en los siguientes listados, dependiendo de su sistema.

Listado C.1 Copia de la clave pública SSH en macOS/Linux

```
gato ~/.ssh/id_rsa_rk8s.pub | pbcopy
```

Listado C.2 Copia de la clave pública SSH en Windows

```
gato ~\.ssh\id_rsa_rk8s.pub | clip
```

Ahora que podemos copiar la clave pública, podemos compartirla donde sea necesario o instalarla directamente.

C.4 Instalación de la clave pública SSH en un host remoto

Para esta sección, necesitamos un host remoto activo (como una máquina virtual Linode) donde podamos copiar nuestras claves SSH. Instalaremos estas claves mediante dos métodos diferentes.

C.4.1 Instalación manual de claves SSH con copiar y pegar

Si su máquina tiene el comando `ssh-copy-id`, puede considerar saltar al siguiente paso:

1. Copie la clave pública (como en el listado C.1 o C.2).
2. Host SSH: en `root@<dirección_IP_de_su_linode>` Se le solicitará `ssh` contraseña.
3. En el host remoto, navegue a `~/ssh/`: `cd ~/ssh` 4. Cree un nuevo archivo llamado `authorized_keys`: `touch claves_autorizadas`
5. Abra el archivo: `nano claves_autorizadas` 6. Pegue el contenido de su clave pública en el archivo.
7. Cierre y guarde el archivo con `CTRL + X` y luego `Y`, y luego presione `Enter`.
8. Revise los cambios con `cat ~/ssh/authorized_keys` 9. Salga de la sesión SSH: `exit`

C.4.2 Instalación automática de claves SSH con el comando `ssh-copy-id`

El comando es simple:

```
ssh-copy-id -i ~/.ssh/id_rsa_rk8s.pub root@<dirección IP de su servidor>
```

Deberías verlo

```
/usr/bin/ssh-copy-id: INFORMACIÓN: Origen de la(s) clave(s) que se instalarán: "/Users/justin/.ssh/id_rsa_rk8s.pub" /usr/bin/ssh-copy-id: INFORMACIÓN: intentando iniciar sesión con la(s) nueva(s) clave(s), para filtrar las que ya estén instaladas /usr/bin/ssh-copy-id: INFORMACIÓN: Quedan 1 clave(s) por instalar - - si se le solicita ahora es para instalar las nuevas claves contraseña de root@<your_server_ip_address>:
```

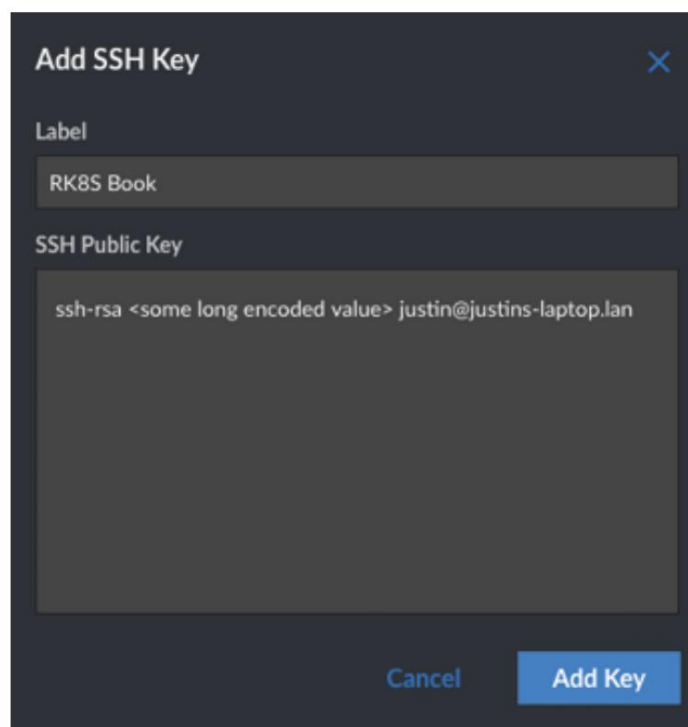
Una vez ingresada la contraseña, se deberá instalar la clave.

C.4.3 Instalación automática en máquinas virtuales en Linode

1. Vaya a Claves SSH en la consola Linode

(<https://cloud.linode.com/profile/keys>)

2. Haga clic en el botón Agregar una clave SSH .
3. En el campo Etiqueta , ingrese RK8S Book.
4. En el campo "Clave pública SSH" , pegue la clave id_rsa_rk8s.pub de la sección anterior (ver figura C.1). La clave real será más larga que la que se muestra aquí.
5. Haga clic en Agregar clave.
6. Verifique que se haya agregado la clave del libro RK8S (figura C.2).



Add SSH Key

Label

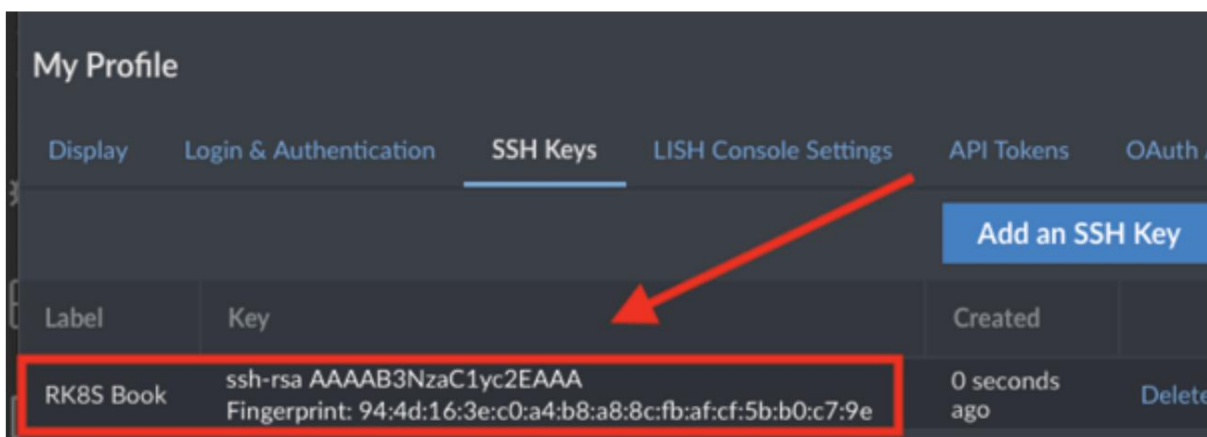
RK8S Book

SSH Public Key

ssh-rsa <some long encoded value> justin@justins-laptop.lan

Cancel Add Key

Figura C.1 Ejemplo de clave SSH en la consola de Linode



My Profile

Display Login & Authentication **SSH Keys** LISH Console Settings API Tokens OAuth A

Add an SSH Key

Label	Key	Created	
RK8S Book	ssh-rsa AAAAB3NzaC1yc2EAAA Fingerprint: 94:4d:16:3e:c0:a4:b8:a8:8c:fb:af:cf:5b:b0:c7:9e	0 seconds ago	Delete

Figura C.2 Almacenamiento de claves SSH en la consola de Linode

Al igual que al crear estas claves en su equipo local, puede agregarlas o eliminarlas de Linode cuando lo desee. También puede agregar varias claves a su cuenta de Linode. Esta es una excelente manera de garantizar el acceso a su cuenta de Linode desde varios equipos.

Cuando necesite instalar esta clave en cualquier máquina virtual, asegúrese de seleccionar agregar RK.

C.5 Conectarse al host remoto con una clave SSH específica

Independientemente de cómo haya instalado su clave pública SSH utilizando los métodos anteriores (secciones 3.4.1, 3.4.2 y 3.4.3), ahora deberíamos poder usar la entrada sin contraseña a nuestra máquina virtual:

```
ssh -i ~/.ssh/id_rsa_rk8s root@<su_dirección_IP_de_linode>
```

Algunas cosas a tener en cuenta son las siguientes:

- Si su clave SSH tenía una frase de contraseña, se le solicitará la misma.
- Usamos `-i ~/.ssh/id_rsa_rk8s` y no `-i ~/.ssh/id_rsa_rk8s.pub` con el comando SSH . Las claves privadas se usan para comprobar la validez de una clave pública.
- Debemos pasar `-i ~/.ssh/id_rsa_rk8s` hasta que actualicemos la configuración de SSH en la siguiente sección.

C.6 Actualización o creación de configuración SSH

Escribir nuestra clave privada cada vez que necesitamos conectarnos al mismo servidor se vuelve aburrido rápidamente. Claro, si usáramos la clave SSH predeterminada `~/.ssh/id_rsa`, podríamos omitir por completo `-i ~/.ssh/id_rsa_rk8s` , pero eso podría significar que usáramos la misma clave pública en todas partes, lo cual perjudica la seguridad.

En su lugar, actualicemos nuestro archivo de configuración ssh para que podamos garantizar que la dirección IP remota específica use la misma clave cada vez. Usando un usuario administrador, abra o edite ~/.ssh/config:

```
Anfitrión rk8s-book-vm#1
    Nombre de host <su_dirección_ip_de_linode>#2 Usuario
    root#3 IdentityFile
    ~/.ssh/id_rsa_rk8s#4

Host <su_dirección_IP_de_linode>#5 Nombre de
    host <su_dirección_IP_de_linode>#6 Usuario root#7 Archivo
    de identidad ~/.ssh/
    id_rsa_rk8s#8
```

#1 El host puede ser un apodo o una

dirección IP específica.

#2 HostName debe ser la dirección IP o el nombre de dominio asignado a la dirección IP.

#3 El usuario debe tener acceso SSH. El usuario root lo tiene por defecto.

#4 IdentityFile debe ser una ruta válida a una clave privada SSH.

#5 El host puede ser un apodo o una

dirección IP específica.

#6 HostName debe ser la dirección IP o el nombre de dominio asignado a la dirección IP.

#7 El usuario debe tener acceso SSH. El usuario root lo tiene por defecto.

#8 IdentityFile debe ser una ruta válida a una clave privada SSH.

Una vez que se actualiza ~/.ssh/config , ahora podemos conectarnos a nuestro host remoto usando el apodo que creamos o la dirección IP sin el indicador -i para nuestra clave privada:

```
ssh root@rk8s-book-vm ssh
root@<su_dirección_IP_de_linode>
```

Apéndice D Instalación y uso de ngrok

Desarrollamos aplicaciones web para poder compartirlas con otros. Para compartirlas, necesitamos que sean accesibles en internet.

Pasar de las aplicaciones actuales a un entorno de producción real requerirá algunos capítulos más para comprender completamente el proceso, pero eso no significa que no podamos publicar nuestras aplicaciones en internet ahora mismo. Podemos hacerlo usando una herramienta llamada ngrok (se pronuncia en-grok). ngrok nos permite exponer nuestras aplicaciones web locales a internet. Es una herramienta excelente para pruebas y desarrollo, así como para compartir nuestro progreso con amigos o compañeros de trabajo.

D.1 Instalación de ngrok

Podemos instalar ngrok con las instrucciones en su sitio web en <https://ngrok.com/download>. En esta ubicación, hay una excelente guía paso a paso para hacer que ngrok funcione en su máquina local.

D.2 Verificar la instalación

Suponiendo que haya instalado ngrok correctamente, debería poder ejecutar el siguiente comando:

```
ngrok --versión #1
```

#1 La salida debe ser la versión ngrok 3.1.0.

D.3 Ejecución de ngrok

Ahora que hemos verificado que ngrok está instalado, ejecutémoslo:

```
ngrok http 3011
```

El comando se descompone así:

- `ngrok http`: inicia un ngrok con un túnel HTTP.
- `3011`—El puerto que queremos exponer, que es exactamente asignado a `http://localhost:3011`.

Esto generará algo como lo siguiente:

```
salida estándar
Estado de la sesión en línea
Versión de Justin Mitchel (Plan: Gratis)
la cuenta 3.1.0
Latencia Estados Unidos (nosotros)
de la región 65 ms
Reenvío de interfaz http://127.0.0.1:4040
web https://1f69-192-63-174-214.ngrok.io -> http://localhost:3011

Conexiones p50 TTL abierto rt1 rt5
p90
0 0 0.00 0.00
0.00 0.00
```

Las opciones para detener/cancelar la ejecución del túnel ngrok son las siguientes:
Sigue:

- Salir de la línea de comandos/ventana de terminal (lo más fácil).

- Utilice CTRL+C en la terminal donde se ejecuta el programa (recomendado).
- Utilice kill -9 <PID> donde <PID> es el ID del proceso de la instancia del túnel ngrok (avanzado).

índice

SÍMBOLOS

[--bandera de puerto](#)

[--recargar bandera](#)

[Archivo .dockerignore, segundo](#)

[llamar de vuelta](#)

[método http](#)

[cadena de ruta](#)

A

[Máquina basada en ARM](#)

[apt. gestor de paquetes](#)

[declaración apta](#)

[Un récord, 2º](#)

[Nube conectada de Akamai \(ACC\)](#)

[creando nueva VM en, 2da, 3ra](#)

[barra de navegación](#)

[descripción general](#)

[AWS Elastic Beanstalk](#)

Administrador de paquetes APT

Manual de estrategias de Ansible

Aplicación Node.js, 2.^a

Aplicación de Python, 2.^a

como archivo con formato YAML

creando, 2do _____

Creación de aplicaciones para Node.js

variable playbook dir _____

vs. Flujo de trabajo de GitHub Actions

aplicaciones

en contenedores, 2º _____

contenerización

Implementar como tarea compleja _____

etapa de desarrollo _____

Internet _____

Múltiples etapas en el proceso de preparación de solicitudes _____

portátil _____

etapa de producción _____

renderizado del lado del servidor (SSR) _____

puesta en escena

[etapa de puesta en escena](#)

[con estado](#)

[paquetes de terceros y desarrollo de aplicaciones](#)

[Comando apt-get install nginx](#)

[Ansible, archivos declarativos y](#)

[automatización, repositorio Git](#)

[Andreesen, Marc](#)

[AWS Secrets Manager, como alternativa a los secretos de Kubernetes](#)

[Interfaz de puerta de enlace asíncrona \(ASGI\)](#)

[repetibilidad automatizada](#)

Ansible

[repetibilidad automatizada](#)

[módulo de comando](#)

[copia, módulo incorporado](#)

[estado actual, 2º](#)

[programación declarativa](#)

[Implementando la aplicación Node.js con, 2do](#)

[Implementando una aplicación Python con, 2da](#)

[descrito](#)

[estado deseado](#)

Enfoque basado en documentos para la automatización

manipuladores

recurso para revisar las opciones de configuración

Ejecutando con GitHub Actions

conexión de shell segura (SSH)

resultado de la configuración

módulo de sincronización

estado actualizado

automatización de máquinas virtuales con, 2da

B

repositorio vacío

creando

creando un repositorio remoto y privado

descrito

metadatos del proyecto

paquetes de compilación

creando imágenes de contenedores

descrito

shell bash

do

[interfaz de línea de comandos \(CLI\) _____](#) .

creando

[Nuevo secreto del repositorio _____](#)

[claves SSH especialmente diseñadas](#)

[Integración continua y entrega continua \(CI/CD\) _____ -- _____](#) .

[instancia de contenedor, escalamiento](#)

configurando

[Cientes nómadas _____](#)

[Servidor nómada _____](#)

[Orquestación de contenedores, herramientas alternativas, 2.º _____](#)

[plano de control _____](#)

[Cónsul, servicio HashiCorp _____](#)

[Canalizaciones de CI/CD](#)

[Acciones de GitHub y, 2.º _____](#)

[historial de confirmaciones](#)

[contenedores, 2º, 3º _____](#)

[-it, terminal interactiva _____](#)

[Flujo de trabajo de GitHub Actions para la creación, 2.º _____](#)

Kubernetes y

como máquinas virtuales ligeras

shell bash, 2do

Construyendo y ejecutando sin Kubernetes. 2do.

Construyendo con GitHub Actions

constructor de contenedores

imagen del contenedor

registro de contenedores

tiempo de ejecución del contenedor, 2.º

comunicación entre contenedores

definido

pod eliminable

Implementación en Kubernetes. 2.º

descrito, 2º

ingresando al contenedor NGINX

naturaleza efímera de

puerto interno de contenedores

limitaciones de

Proporcionar una red para la comunicación entre

escalado con Docker Compose

[puesta en escena. 2ª](#)

[con estado, 2º](#)

[apátrida. 2º, 3º](#)

[dependencia a nivel de sistema](#)

[volúmenes](#)

[volúmenes y con estado, 2º](#)

orquestración de contenedores

[Docker Compose como primer paso hacia](#)

[descrito](#)

[computación en la nube](#)

grupos

[definido](#)

[espacios de nombres y gestión de recursos del clúster](#)

Conectando a través de

[ejemplo de conexión](#)

[constructor de contenedores](#)

[Aplicaciones en contenedores. implementación. 2.º](#)

[despliegue continuo](#)

[Botón Crear repositorio](#)

[Mapas de configuración](#)

como pares clave-valor

Personalización de NGINX

definido

uso diferente de

variables de entorno con, 2º

configuración de variables de entorno

estado actual, 2º

mensajes de confirmación

Razones para escribir bien

verboso

integración continua (IC)

pruebas automatizadas

definido

archivo compose prod yml

imágenes de contenedores

dockerignore

NGINX

Construyendo con Docker Compose

Diseñando nuevo, 2do

Problema al usar Docker Hub para compilar

[preconstruido, en Docker](#)

[Empujando a Docker Hub, 2do](#)

[Imagen de contenedor de Python en ejecución](#)

[Ejecutando desde Dockerfile](#)

[comando curl](#)

[entrega continua \(CD\)](#)

[definido](#)

[entorno de puesta en escena](#)

D

Estibador

[Comando ADD](#)

[Comando COPIAR](#)

[como herramienta declarativa](#)

[herramienta buildx](#)

[administrador de procesos integrado](#)

[imágenes de contenedores, 2da](#)

[archivos declarativos en](#)

[descrito](#)

[variables de entorno](#)

[Ignorar archivos no deseados](#)

[Instalación en Ubuntu](#)

[Instalación en VM con GitHub Actions](#)

[puertos](#)

[herramientas de acción de GitHub de terceros](#)

[variaciones de los comandos de compilación y envío](#)

[Administrador de Docker Swarm, 2.º](#)

[bases de datos](#)

[Motor Docker](#)

[descrito](#)

[Instalación en Ubuntu](#)

Implementación directa en Ubuntu

[Detención de contenedores y publicación del puerto](#)

[Enjambre de Docker](#)

[Docker Compose](#)

[ventaja sobre Kubernetes](#)

[Descubrimiento de servicios integrado, 2.º](#)

[configuración](#)

[orquestración de contenedores con, 2º](#)

[a partir de](#)

[terminología](#) . .

[archivo de configuración predeterminado](#)

[TOKEN DE DOCKERHUB](#)

Escritorio Docker

[instalando](#)

[guía de instalación oficial](#) _____

[bases de datos, contenedores y](#) _____

[Centro de Docker](#)

[Etiquetas de versión de Node.js](#)

[Etiquetas de versión de Python](#)

[como registro oficial de Docker](#) _____ .

[Iniciar sesión a través de Docker Login](#) _____

[Imágenes de contenedores oficiales y prediseñadas](#) _____

[página del repositorio](#) _____

[Servicios de Docker Compose](#) _____

[Plataforma de compilación Docker](#)

[pérdida de datos](#)

[estado deseado. 2º](#) _____

[dependencias. 2ª. 3ª](#) _____

[específico de la aplicación](#) _____

[específico del hardware](#)

[Instalación de NGINX en Ubuntu](#)

[Instalación de dependencias de aplicaciones, 2.º](#)

[Sistema operativo](#)

[tercero, 2do](#) .

[varios](#)

[versión específica](#)

archivos declarativos

[en Docker](#)

[en Kubernetes](#)

[herramientas de infraestructura como código y](#)

despliegue

[computación en la nube](#)

[basado en contenedores, límites de](#)

[software](#)

[Comando docker compose down, segundo](#)

[dotenv, archivo de variable de entorno, 2do](#)

[Archivos Docker](#)

[Ejemplo de creación de un archivo de configuración para un contenedor](#)

[Comando docker compose up, segundo](#)

[valor del diccionario](#)

Implementar aplicaciones en producción

[Servicios de balanceador de carga](#)

[pasos para](#)

[Registro de Docker](#) .

[Comando de compilación de Docker Compose](#)

[Docker Compose, 2º](#) _____

[Servicio Redis en](#)

[Automatizar comandos para crear y ejecutar contenedores](#)

[Imagen de contenedor de construcción con](#)

[gestor de paquetes apt integrado](#) _____

[descubrimiento de servicios integrado](#)

[gestión de volumen incorporada](#) _____

[compose.yaml, archivo de configuración](#) _____

[configuración, opciones clave](#) _____

[bases de datos](#)

[descrito](#)

[para Docker Swarm, 2º](#) _____

[para aplicaciones Node.js, 2º](#) _____

[para aplicaciones Python, 2º](#) _____

[para múltiples entornos](#) _____

[para producción Node.js](#) _

[para producción Python](#) _____

[instalación en Ubuntu](#) _____

[Instalación en VM con GitHub Actions](#) _____

[Limitaciones de uso para producción](#) _____

[desaparecido](#)

[redes, 2º](#) _____

[acceso remoto](#) _____

[contenedores de escalado](#)

[host único](#) _____

[etiquetado de imágenes](#)

[dos formas de parar](#)

[Usando dotenv con](#) _____

[con múltiples Dockerfiles](#) _____

[REPOSITOR DOCKERHUB](#)

[entorno de desarrollo](#) _____

[flexibilidad](#)

[programación declarativa](#) _____

Archivo Docker

[Imagen de un contenedor de construcción de](#)

[creando_2do _____](#)

[Diseño de nuevas imágenes de contenedores](#)

[Diferentes Dockerfiles para múltiples entornos](#)

[sintaxis](#)

[despliegues](#)

[como grupos de vainas](#)

[Etiqueta de contenedor e implementación de producción](#)

[declaración y montaje de volúmenes](#)

[definido](#)

[descripción general](#)

[Docker Swarm Stack implementación](#)

mi

[script de punto de entrada_2º_3º](#)

[variables de entorno_2.ª_3.ª](#)

[ConfigMap para](#)

[Valor del puerto](#)

[abstraído del código de la aplicación](#)

[definiendo](#)

[archivos dotenv](#)

[con ConfigMaps y secretos, 2do](#)

ambiente

[Diferentes Dockerfiles para múltiples entornos](#)

[entorno de puesta en escena](#)

Express.js

[como marco web del lado del servidor](#)

[creando una respuesta JSON con, 2da](#)

[creando aplicación, 2da](#)

[descrito](#)

[manejo de rutas con](#)

[instalando](#)

[Requisitos para el uso](#)

[devolviendo datos JSON con](#)

[servidor web en ejecución](#)

[utilizado como microservicio](#)

[Comando EOF \(Fin de archivo\)](#)

F

FastAPI, 2.º

Creación de la aplicación FastAPI, 2.º

descrito

Manejo de múltiples rutas con, 2º

puerto 8011

Usando uvicorn para correr

cortafuegos

fallido, estado del pod

archivos

Protocolo de transferencia de archivos (FTP)

Estado de Git y

confirmando cambios

creando un archivo posterior a la recepción

agregar git

postergación

seguimiento

sin seguimiento

GRAMO

Comando git commit, 2do

[gunicorn](#)

[configuración de línea base para](#)

[descrito](#)

Secreto de GitHub Actions

[añadiendo dirección IP](#)

[añadiendo token de acceso personal a](#)

[creando, 2do](#)

[complemento login action](#)

[comando git init](#)

[Repositorio de GitHub](#)

[Google Secrets Manager, como alternativa a los secretos de Kubernetes](#)

GitHub

[Configurando el repositorio git local para](#)

[creando cuenta](#)

[creando repositorio en](#)

[Empujando código a, 2do](#)

[Administrador secreto de Google](#)

[comando git checkout](#)

[Flujo de trabajo de GitHub Actions](#)

[Ansible, 2º](#) _____

[ejemplo, 2do](#) _____

[flexibilidad de](#) _____

[para Node.js y Ansible Playbook](#) _____

[para la construcción de contenedores](#)

[Instalación de NGINX en un servidor remoto con](#) _____

[elementos por definir](#) _____

[gestión de configuraciones, 2º, 3º](#) _____

[revisando la salida](#) _____

[Sintaxis para usar secretos almacenados](#) _____

[vs. Ansible Playbook](#) _____

Acciones de GitHub

[Pestaña Acciones](#)

[Canalizaciones de CI/CD, 2.º](#)

[Recursos de Kubernetes](#)

[Lista desplegable Ejecutar flujo de trabajo](#)

[Construyendo y empujando contenedores, 2º, 3º, 4º](#) _____

[construcción de contenedores con](#)

[programación declarativa](#) _____

[Despliegue de contenedores de producción, 2.º](#) _____

descrito

Ejecutando Docker Compose en producción

Preocupación por la seguridad y el almacenamiento de secretos en

barra lateral

complementos de terceros

GitLab

Git

como herramienta de código abierto

historial de confirmaciones

Confirmar cambios con el comando git commit

características de

Git add y seguimiento de archivos Python

Descripción general de los comandos fundamentales de Git, 2.º

reiniciando repositorios Git

compartir códigos hacia y desde computadoras

estado

cambios de código de seguimiento, 2do

utilizando en proyectos, 2do

verificando que esté instalado

control de versiones

[comando de estado git](#)

repositorio Git

[pocos beneficios en comparación con los servicios de terceros](#)

[inicializando](#)

[enviando código local al repositorio remoto](#) _____ .

[remoto autoalojado, 2do](#) _____

H

[HEAD, creando repositorios Git vacíos](#) _____

[implementaciones híbridas](#)

manipuladores

[Implementando](#)

[supervisor de recarga](#)

[Código fuente HTML](#) _____

[encabezados, devolviendo datos JSON con Express.js](#) _____

[controles de salud](#)

[directorio de ganchos](#)

[Corporación Hashi](#)

[HashiCorp Vault, como alternativa a los secretos de Kubernetes](#) _____

[héroe ku](#)

[Nómada de HashiCorp](#)

[HCL, lenguaje de configuración, 2º](#) _ _ _

[Definición de trabajo nómada](#)

[Comparación de Nomad CLI y Kubernetes CLI](#) _____

[Instalación de Nomad](#) _____

[interfaz de usuario intuitiva](#) _____

[descripción general](#)

[preparando el clúster Nomad](#) _____

[Ejecución de contenedores con trabajos Nomad. 2.º](#) _____

[terminología](#) . .

[Configuración del trabajo de prueba, ejemplo](#) _____

|

[imágenes, etiquetado](#)

instalación para Windows

[Descargar e instalar Node.js](#) _____ _

[Verificando la información del sistema. 2da](#) _____

[initContainers, creando StatefulSet y](#) _____

[gestión de entrada](#) _____

[archivo de inventario](#)

[programación imperativa](#)

[Infraestructura como Código \(IaC\), 2.º](#)

instalando

[binario de kubectl en Linux](#)

[con Chocolatey en Windows](#)

[con Homebrew en macOS](#)

J

[Formato de datos JSON](#)

JavaScript

[como lenguaje amigable para principiantes](#)

[basado en navegador](#)

[del lado del servidor](#)

K

[kubectl, CLI de Kubernetes](#)

[Comparando con Nomad CLI](#)

[configurando](#)

[plano de control](#)

[instalación, 2do](#)

[ejecutando el pod de eliminación en cualquier momento](#)

[comando de implementación de kubectl](#)

Clústeres de Kubernetes

[aprovisionamiento](#)

[velocidad en la creación](#)

Servicios de Kubernetes

[creando primero, 2do](#)

[servicio Kubernetes predeterminado](#)

[descrito](#)

[tipos fundamentales](#) . [_____](#)

[archivo kubeconfig](#)

[Variable KUBECONFIG](#)

[configuración en VSCode](#)

Kubernetes

[Docker Compose](#) [_____](#)

[Acciones de GitHub e implementación en](#) [_____](#)

[Tablero del K8, 2.º](#) [_____](#)

[modos de acceso](#)

[como herramienta declarativa](#)

[conectando a, 2do](#) [_____](#)

[plano de control](#)

[conceptos y componentes básicos, 2º](#) _____

[comunicación entre contenedores](#) _____

[comunicación entre contenedores](#) _____

[archivos declarativos en](#) _____

[Implementación de aplicaciones en producción, 2.º](#) _____

[Desplegando contenedores a, 2do](#) _____

[ruta de implementación](#) _____

[recurso de implementación](#) _____

[descrito](#) _____

[variables de entorno](#) _____

[Desafíos fundamentales con el uso, 2º](#) _____

[Aprovisionamiento de clúster de Kubernetes, 2.º](#) _____

[Razones para usar Kubernetes administrado basado en la nube, 2.º](#) _____

[tipos de recursos](#) _____

L

[Motor de Kubernetes Linode \(LKE\)](#) _____ .

[login action, complemento de acciones de GitHub](#) _____

[entrada de registro, información en](#) _____

[equilibrio de carga, 2º, 3º](#) _____

Servicios de LoadBalancer, 2.º

METRO

etiquetas de metadatos

Manifiestos

colas de mensajes, contenedores y

nodo administrador

se manifiesta

aplicar etiquetas de metadatos a los recursos

descrito

características principales

soporte para múltiples máquinas virtuales

Implementación manual con máquinas virtuales, 2.º

norte

Administrador de versiones de nodo (nvm)

instalando

Instalación de dependencias de aplicaciones

verificando que esté instalado

Archivo Docker de Node.js

Administrador de paquetes de nodos (npm)

[Instalación de dependencias de la aplicación, ejemplo de dependencias del paquete](#)

[Variable bash NVM VERSION](#)

herramienta ngrok

[descrito](#)

[Instalación y uso, 2º](#)

[grupos de nodos](#)

Aplicaciones Node.js

[Docker Compose](#)

[archivo compose.yaml](#)

[contenerización, 2º](#)

[script de punto de entrada](#)

[apátrida](#)

[Servicio NodePort](#)

[nodo, definido](#)

NGINX

[configuración para Python y Ansible](#)

[imágenes de contenedores](#)

[comunicación entre contenedores](#)

[Personalización con ConfigMaps](#)

[Implementación directa en Ubuntu](#)

[Diferentes formas de eliminar](#)

[máquinas virtuales efímeras](#)

[Instalación en Ubuntu](#)

[Instalación en sistemas Ubuntu](#)

[Instalación en un servidor remoto con el flujo de trabajo de GitHub Actions, 2.º](#)

[Instalación con Ansible](#)

[equilibrio de carga](#)

[modificar la página HTML predeterminada](#)

[mi servicio nginx](#)

[descripción general](#)

[configuración de proxy inverso](#)

[Sirviendo múltiples aplicaciones](#)

[Sirviendo sitios web estáticos, 2º](#)

[puerto de nodo](#)

Node.js

[archivo .dockerignore](#)

[Configuración de supervisor para](#)

[Construyendo contenedores con Docker Compose](#)

[instalación para Windows](#)

[instalación para macOS](#)

[JavaScript del lado del servidor](#)

[iniciando un nuevo proyecto](#)

espacios de nombres

[Recursos de Kubernetes](#)

[añadiendo recursos a](#)

[definido](#)

[ejemplos de puntos finales DNS para servicios en varios](#)

[Recurso de espacios de nombres para Redis](#)

[no utilizado ni declarado](#)

[Nuevo secreto del repositorio](#)

[herramienta nvm-windows](#)

Oh

[servidores locales y software implementado en](#)

PAG

[Mapeo de puertos](#)

[lenguajes de programación, contenedores y](#)

[Direcciones IP estáticas públicas, Kubernetes administrado](#)

[Intérpretes de Python](#)

[pendiente, estado del pod](#)

[Python 3, instalación](#)

[Creación de un entorno virtual del lado del servidor](#)

[Ejecutando la aplicación Python, 2.º](#)

[Actualización del gancho de Git para la aplicación Python](#)

vaina

[Basado en NGINX](#)

[como la unidad configurable más pequeña en Kubernetes](#)

[creando nuevo en el panel](#)

[definido](#)

[borrable](#)

[supresión](#)

[despliegues](#)

[diferentes fases](#)

[esquema de nombres, 2º](#)

[ordenando](#)

[lanzamiento](#)

[puertos, 2º](#)

[paquete de efectivo](#)

contenedores de producción

[edificio y alojamiento. 2º](#) _____

[Construido en una máquina diferente a la del host de producción](#)

[Token de acceso personal](#)

[Índice de paquetes de Python \(PyPI\)](#) .

[Plataforma como servicio \(PaaS\)](#) _____ .

[dispositivos personales y software implementado en](#) _____

producción

[tiempo de ejecución del contenedor](#)

[dependencias a nivel de sistema](#) _____

[Usando GitHub Actions para ejecutar Docker Compose en segundo lugar](#)

Herramienta de paquetes de Python (pip)

[Instalación de dependencias de aplicaciones](#)

[Plataforma como servicio \(PaaS\)](#) _____ .

Python y

[instalación de paquetes de Python](#) _____

[Recurso PersistentVolumeClaim \(PVC\)](#) _____

[portabilidad](#) .

[Procfile. archivo de configuración](#)

Pitón

[archivo .gitignore](#)

[Enfoque basado en API](#)

[Ansible, herramienta de línea de comandos](#)

[Configuración de supervisor para](#)

[como lenguaje amigable para principiantes](#)

[Construyendo contenedores con Docker Compose, 2º](#)

[construyendo a partir del código fuente](#)

[versión actual](#)

[contenedores de bases de datos](#)

[ecosistema de desarrolladores](#)

[serialización de diccionarios](#)

[descargando el instalador](#)

[instalación para Linux](#)

[instalación para Windows](#)

[instalación para macOS](#)

[instalar todos los paquetes de terceros](#)

[instalando paquetes](#)

[procesador](#)

[carpeta de desarrollo del proyecto](#)

[enviando la imagen del contenedor a Docker Hub](#)

[Reiniciando repositorios Git, ejemplo de _____](#)

[Revisando el registro del historial de confirmaciones](#)

[Pasos para crear un nuevo proyecto _____](#)

[herramientas de instalación de terceros](#)

[gancho de Git posterior a la recepción](#)

[Supervisor _____](#)

[como script bash _____](#)

[Instalación de dependencias de la aplicación y actualización](#)

[Actualización para la aplicación Python](#)

[verificando](#)

[Administrador de paquetes de Python \(pip\) _____](#)

Aplicaciones de Python

[Docker Compose _____](#)

[contenerización, 2º _____](#)

[script de punto de entrada](#)

[apátrida _____](#)

Archivo Docker de Python

[creando, 2do _____](#)

[para navegar _____](#)

[Actualización para herramientas Redis](#)

[etiquetas de versión](#)

[dirección IP pública, aprovisionamiento](#)

Q

[Sección de configuración rápida](#)

R

servidor remoto

[creando y conectando con, 2do](#)

[Modo de acceso ReadWriteMany](#)

[en ejecución, estado del pod](#)

[repositorio remoto, 2do](#)

[res.json\(\), que devuelve datos JSON](#)

[Modo de acceso ReadOnlyMany](#)

[Modo de acceso ReadWriteOnce](#)

[límite de réplicas](#)

[límite de réplicas](#)

[réplicas](#)

[REDIS_HOST, variable de entorno](#)

reiniciando repositorios Git

[desventaja](#)

[archivos de requisitos](#)

[complejo](#)

[archivos del repositorio](#)

[ciclo de solicitud/respuesta, 2º](#)

[recuento de réplicas](#)

Redis

[Conjunto con estado para](#)

[como aplicación con estado](#)

[control de acceso basado en roles \(RBAC\)](#)

[proxy inverso, 2º](#)

S

[comando sudo apt install nginx](#)

Supervisor

[procesos en segundo plano](#)

[comprobando el estado](#)

[configuración para Python y Ansible](#)

[actualización de configuración](#)

[configuración para aplicaciones, 2º](#)

[creando directorios de registro](#)

[descrito](#)

[instalando](#)

[comandos útiles](#)

Claves SSH

[ACC e instalación de una clave SSH pública](#) .

[y acciones de GitHub](#)

[componentes](#)

[Conectándose al host remoto con una clave SSH específica](#) .

[copiando clave pública](#) .

[Generación de claves SSH de plataforma cruzada](#)

[creando nuevos](#)

[Instalación de la clave pública SSH en el host remoto](#)

[instalación pública](#)

[sobrescribiendo](#)

[entrada al servidor sin contraseña](#) .

[pegar nueva clave SSH](#) .

[permiso denegado](#)

[construido especialmente, 2º](#)

[preocupación por la seguridad](#)

[errores de clave ssh](#)

[generador de claves ssh](#)

[verificando](#)

[valor del selector](#)

misterios

[definido](#)

[para Postgres](#)

[para variables de entorno](#)

[Herramientas de terceros y protección de valores secretos](#)

[control del sistema](#)

[escalabilidad](#)

puesta en escena

[aplicaciones en contenedores](#)

[Contenedores y complejidad de las aplicaciones](#)

[comando sudo, 2do](#)

[pila, terminología de Docker Swarm](#)

[seguridad, repositorios Git autohospedados y](#)

[direcciones IP estáticas](#)

[entornos de puesta en escena](#)

software

[desplegado, 2º](#)

despliegue, 2º _____

cuentas de servicio

comando ssh

comando sudo apt update _____

remoto autoalojado

Iniciar sesión en el servidor con SSH

Conjuntos con estado

definido

Definición de configuraciones clave para crear StatefulSet

para Postgres

características principales

descripción general

selectores, 2º, 3º _____

tenido éxito, estado del pod

Shell seguro (SSH)

Sistema operativo basado en Linux _____

Clave SSH instalada en el host remoto

comprobación de autenticidad

Conectarse a una computadora remota con

Conectando a través de

descrito

Navegando al directorio raíz SSH

Requisitos para realizar SSH

actualizar o crear configuración SSH

T

plantilla

puerto de destino

tarea, terminología de Docker Swarm

complementos de acción de GitHub de terceros

U

uvicornio, 2º

descrito

ejecutando aplicaciones FastAPI

Ubuntu

Docker Compose

Motor Docker

Instalación de Docker

Cortafuegos sencillo (UFW)

desconocido, estado de la cápsula

V

venv

[como administrador de entorno virtual integrado](#)
[descrito](#)

[servicios virtuales y software implementado en](#)
[control de versiones, 2da](#)

[integración continua y](#)

entorno virtual

[Python y](#)

[activando](#)

[creando, 2do](#)

[creando y activando](#)

[desactivación y reactivación](#)

[descrito](#)

[recreando](#)

[del lado del servidor, creando](#)

[volúmenes, 2º, 3º, 4º](#)

[y despliegues, 2º](#)

[volúmenes de almacenamiento en bloque](#)

[definido, 2do](#)

[descrito](#)

[desprendimiento](#)

[comando docker](#)

[variables de entorno](#)

[características principales](#)

[Kubernetes administrado](#)

[gestión con Docker Compose](#)

[Mezclar y combinar varios tipos de volumen](#)

[volúmenes persistentes](#)

[usando con Docker, 2do](#)

[usando con Docker Compose](#)

máquinas virtuales

[Instalación de Docker Compose con GitHub Actions](#)

[Instalación de Docker con GitHub Actions](#)

[aprovisionamiento nuevo, 2º](#)

[VSCode](#)

[volumeClaimTemplates, creando nuevos pods y](#)

máquina virtual (VM)

[Nube conectada de Akamai \(ACC\)](#)

Automatización de Ansible y VM, 2.º

máquinas virtuales efímeras

aprovisionamiento

alquiler

Reemplazar la dirección de mensajería instantánea de la máquina virtual

Código de Visual Studio (VSCode)

ver fuente, código fuente HTML

O

Interfaz de puerta de enlace de servidor web (WSGI)

gunicorn

Tecnología de servidores web, contenedores y

nodo trabajador

Torre de vigilancia

Subsistema de Windows para Linux (WSL)

incógnita

máquina basada en x86

Y

Formato YAML